WARWICK

Advanced Computer Graphics

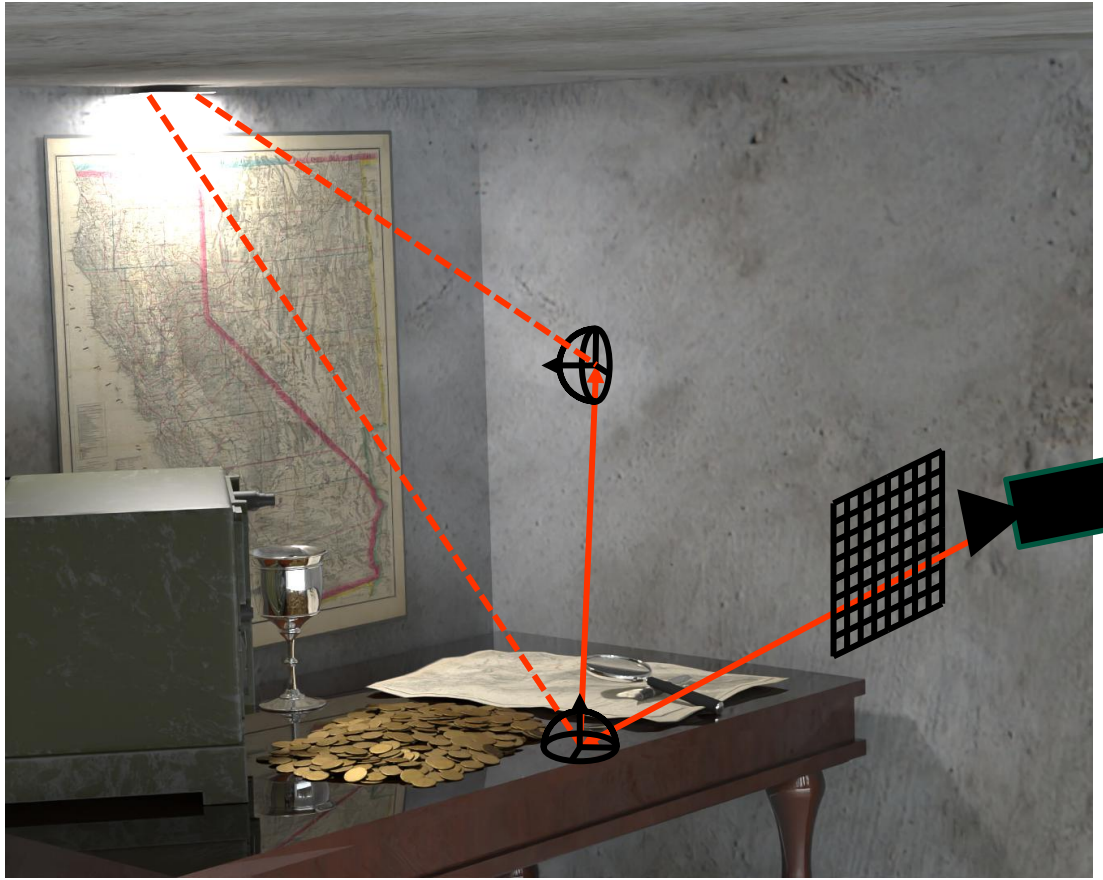
Rendering Algorithms

Dr. Tom Bashford-Rogers

Monte Carlo Rendering

- Path Tracing (Recap)
  - For each pixel
    - Sample a point in the pixel
    - 1. Trace ray
    - Calculate direct lighting
    - Sample indirect direction, update throughput
    - Goto 1. until path terminated

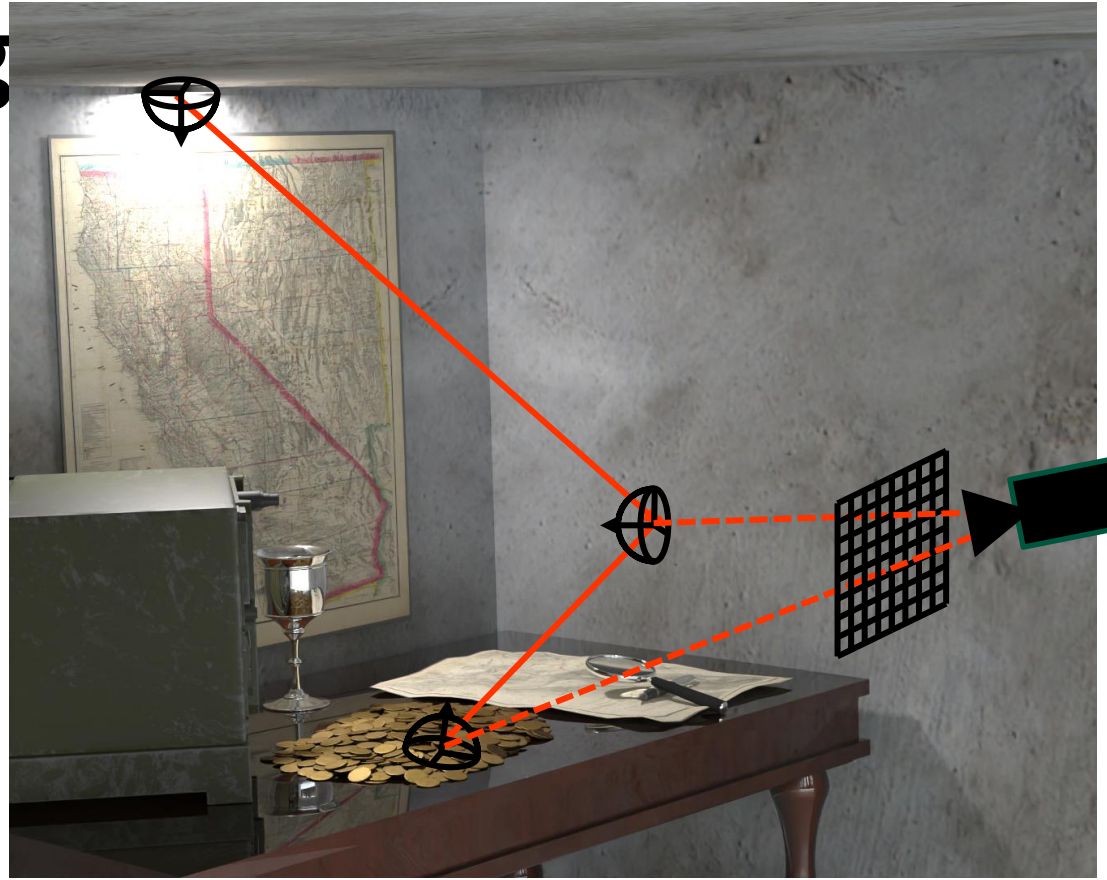
Path Tracing



Light Tracing

- Simulates the natural flow of light
- Start at light sources
- Trace into scene
- NEE to camera
- Bounce and terminate similar to path tracing

Light Tracing



Light Tracing

- Radiance vs Importance
  - Radiance carried from light to camera
    - Describes the actual light energy distribution.

$$L = \frac{d^2\Phi}{dA\,\cos\theta\,d\omega}$$

  - Importance is an adjoint quantity carried from the camera to the light

$$W = \frac{d^2Q}{dA\,\cos\theta\,d\omega}$$

    - Describes the relevance of light paths to the camera

Light Transport

- Monte Carlo Estimate (Light Tracing)
  - Estimate of a single path of length L
  - Note arrow directions

Sample next path vertex

$$\frac{L_e(x_0 \to x_1)}{p_A(x_0)} \left[\prod_{i=1}^{L-2} \frac{1}{1 - q_i} \frac{G(x_{i-1} \leftrightarrow x_i) f_r(x_{i-1} \to x_i \to x_{i+1})}{p_A(x_{i-1} \to x_i)} \right] G(x_{L-1} \leftrightarrow x_L) W_e(x_{L-1} \to x_L)$$

Sample Light

Deterministically connect to camera

Light Tracing

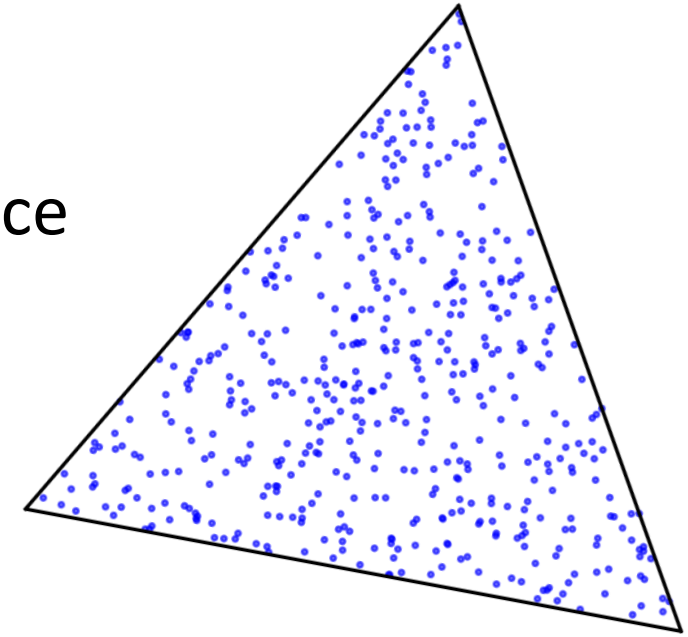
- Algorithm
  - Sample light source
  - Sample point on light source
  - Connect to camera
  - 1. Sample direction, update throughput and trace
  - Connect to camera
  - Goto 1. until path terminated

Light Tracing

- Algorithm
  - Sample light source
  - Sample point on light source

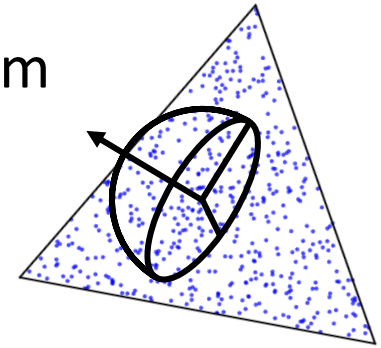
$$pmf(l_i) = \frac{\Phi_i}{\sum_{j=1}^{N_{lights}} \Phi_j}$$

$$\frac{L_e(x_0 \rightarrow x_1)}{p_A(x_0)}$$



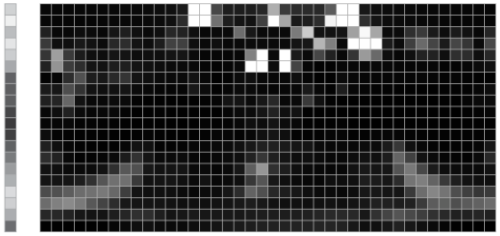
Light Tracing

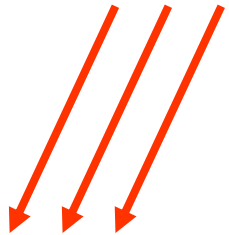
- Algorithm
  - Sample direction from the light
  - Use cosine sampling
    - PDF cancels cosine in first Geometry Term



Light Tracing

- Algorithm
  - Environment Maps
    - 1. Sample direction on sphere
      - Use environment map sampling
      - But need to flip direction to trace into scene

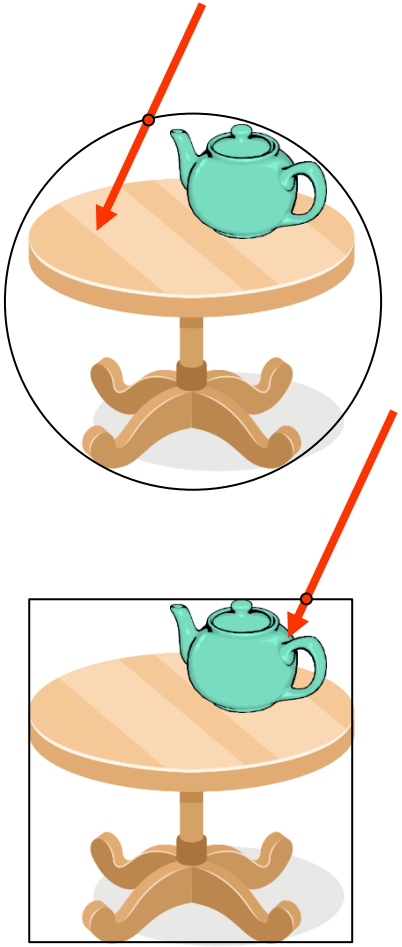




Images from https://pbr-book.org/

Light Tracing

- Algorithm
  - Environment Maps
    - 2. Sample position on the scene
      - Use bounding sphere or box
    - Overall PDF is product of
      - Direction PDF
      - Position on bounds PDF



Light Tracing

- Algorithm
  - Connect to camera
  - Need to define $W_e(x_{L-1} \rightarrow x_L)$
    - Product of filter function and Delta function aligning ray through pixel
    - Can write as $W_e(x, \omega)$ in ray space

Light Tracing

- Algorithm
  - Connect to camera
  - Need to find if point projects onto camera
    - And which pixel
  - Use view and projection matrix!
    - Implemented in Camera class

```
bool projectOntoCamera(const Vec3& p, float& x, float& y)
```

Light Tracing

- Sensor Importance Function $W_e(x_{L-1} \to x_L)$
  - Image from $I = \int W_e(x_{L-1} \to x_L) L(x_{L-1} \to x_L) dA$
  - Estimator $\dfrac{W_e(x_{L-1} \to x_L) L(x_{L-1} \to x_L)}{p_f(x_{L-1} \to x_L)}$
  - Or $\dfrac{W_e(\omega) L(\omega)}{p_f(\omega)}$
  - So $W_e(\omega) \propto p_f(\omega)$

Light Tracing

- Sensor Importance Function $W_e(x_{L-1} \to x_L)$
  - Start with deriving it as normalized PDF in ray space

$$p_f(\omega) = \frac{p_A(x_{L-1} \to x_L)}{G(x_{L-1} \leftrightarrow x_L)}$$

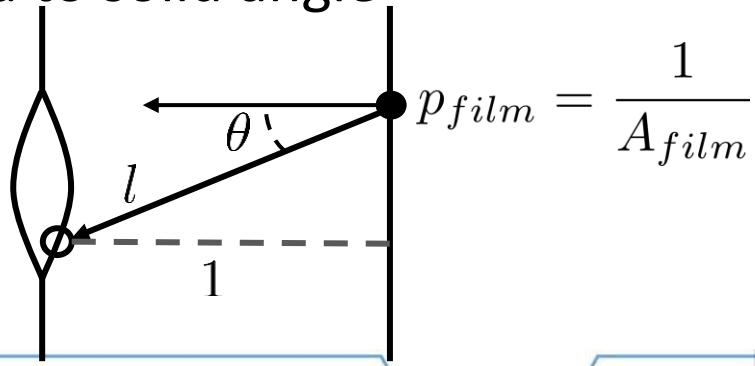
  - Combination of uniformly sampling image plane

$$p_{film} = \frac{1}{A_{film}}$$

Light Tracing

- Sensor Importance Function $W_e(x_{L-1} \rightarrow x_L)$
  - And transport between lens and image plane
    - Recall distance between image plane and lens = 1
    - Change of variables from area to solid angle

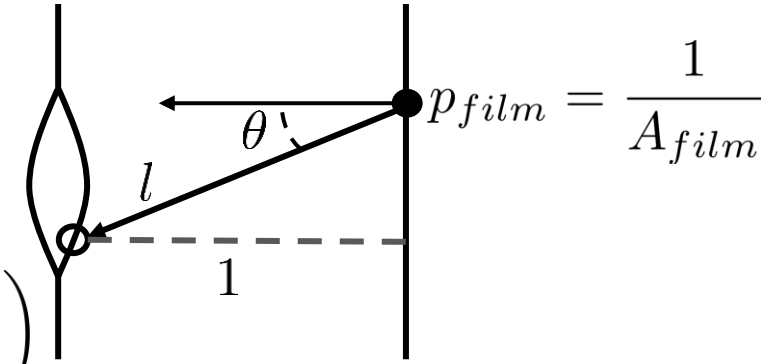
$$p_f(\omega) = p_{film} \frac{l^2}{\cos(\theta)} = \frac{l^2}{A_{film} \cos(\theta)}$$



Light Tracing

- Sensor Importance Function $W_e(x_{L-1} \rightarrow x_L)$
  - And transport between lens and image plane
    - So $l = ||\frac{d}{\cos(\theta)}|| = \frac{1}{\cos(\theta)}$
    - And $p_f(\omega) = \frac{1}{A_{film}\cos(\theta)^3}$
    - Where $A_{film} = W_{film} \cdot H_{film}$

$$W_{film} = 2\tan\left(\frac{fov \cdot \pi}{2 \cdot 180}\right)$$
$$H_{film} = W_{film} \cdot aspect$$



Light Tracing

- Sensor Importance Function $W_e(x_{L-1} \rightarrow x_L)$
  - PDF is defined over directions but $W_e(x_{L-1} \rightarrow x_L)$ defined over $A_{lens} \times \mathcal{S}^2$
  - So can integrate over domain and ensure normalized

$$\int_{A_{lens}} \int_{\mathcal{S}^2} W_e(x, \omega) \cos(\theta) d\omega dA = 1$$

Light Tracing

- Sensor Importance Function $W_e(x_{L-1} \rightarrow x_L)$
  - Can include into definition

$$W_e(x,\omega) = \frac{p_f(\omega)}{A_{lens}\cos(\theta)} = \begin{cases} \frac{1}{A_{film}A_{lens}\cos(\theta)^4} & \text{if ray in frustrum} \\ 0 & \text{otherwise} \end{cases}$$

  - If using a pinhole camera $A_{lens} = 1$
  - If using a realistic camera $A_{lens} = \pi r^2$

Light Tracing

- Monte Carlo Estimate (Light Tracing)
  - Path Throughput $C(\bar{x}_L)$

$$C(\bar{x}_L) = \prod_{i=1}^{L-2} \frac{G(x_{i-1} \leftrightarrow x_i) f_r(x_{i-1} \leftarrow x_i \leftarrow x_{i+1})}{G(x_{i-1} \leftrightarrow x_i) p^{\perp}(x_{i-1} \leftarrow x_i)}$$

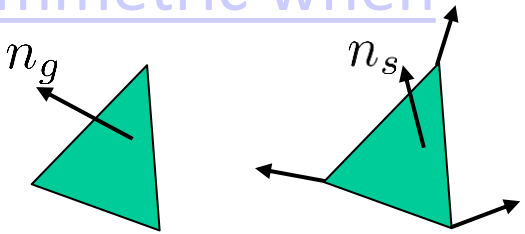
  - Or with the solid angle formulation

$$C(\bar{x}_L) = \prod_{j=1}^{L-2} \frac{f_r(x_j, \omega_{i,j}, \omega_o) \cos(\theta_j)}{p_\Omega(\omega_{i,j})}$$

Use this in practice

Light Tracing

- Non-symmetric scattering
  - Certain operations are non-symmetric when transporting importance
    - Refraction
    - Shading normal n_s vs geometric normal n_g



  - Have to be handed with care
    - Correction factor when transporting importance

$$\frac{|(\omega_o \cdot n_s)|}{|(\omega_o \cdot n_g)|} \frac{|(\omega_i \cdot n_g)|}{|(\omega_i \cdot n_s|)}$$

Light Tracing

- Algorithm
  - Terminate the path with Russian Roulette

- Fast, great for caustics and lighting which is more efficient to sample from the light
- Non uniform over the image plane

Light Tracing

- Implementation
  - Suggest to add three methods
    - Handle connections to camera

      ```
      void connectToCamera(Vec3 p, Vec3 n, Colour col)
      ```

    - Handles starting a light path

      ```
      void lightTrace(Sampler* sampler)
      ```

    - Handles tracing the rest of the light path

```
void lightTracePath(Ray& r, Colour pathThroughput, Colour Le, Sampler* sampler)
```

Light Tracing

```
void connectToCamera(Vec3 p, Vec3 n, Colour col)
```

- Handle connections to camera
  - Check if p is on camera
    - Use `bool projectOntoCamera(const Vec3& p, float& x, float& y)`
  - Compute geometry term between p and camera
    - Camera normal is given by `scene->camera.viewDirection`
    - Camera position is given by `scene->camera.origin`

Light Tracing

```
void connectToCamera(Vec3 p, Vec3 n, Colour col)
```

- Handle connections to camera
  - Need to compute $W_e(x, \omega) = \frac{1}{A_{film} \cos(\theta)^4}$
    - θ is angle between `scene->camera.viewDirection` and direction to camera
    - A_{film} is stored in `scene->camera.Afilm`
  - Splat `col` onto film at coordinates from `projectOntoCamera`

Light Tracing

```
void lightTrace(Sampler* sampler)
```

- Handles starting a light path
  - Sample a light source
  - Check if area light
    - Sample point and direction from on light source

```
p = light->samplePositionFromLight(sampler, pdfPosition);
wi = light->sampleDirectionFromLight(sampler, pdfDirection);
```

    - Connect position to camera
      - `col` is $\frac{L_e(x_0 \rightarrow x_1)}{p_A(x_0)}$: `light->evaluate(-wi) / pdfPosition`

Light Tracing

```
void lightTrace(Sampler* sampler)
```

- Handles starting a light path
  - Create a ray starting at `p` in direction `wi`
  - Then call

```
void lightTracePath(Ray& r, Colour pathThroughput, Colour Le, Sampler* sampler)
```

Light Tracing

- Handles tracing the rest of the light path
  - Call recursively

```
void lightTracePath(Ray& r, Colour pathThroughput, Colour Le, Sampler* sampler)
```

  - Similar to path tracing but no direct lighting
    - Connect each intersection to camera
  - Use direction to camera (need to normalize)

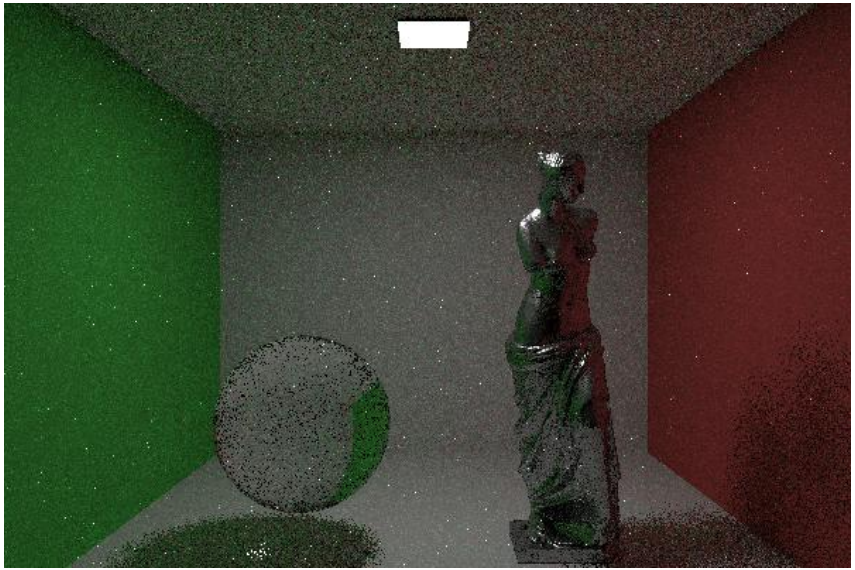
```
Vec3 wi = scene->camera.origin - shadingData.x;
```

  - `col` has value:

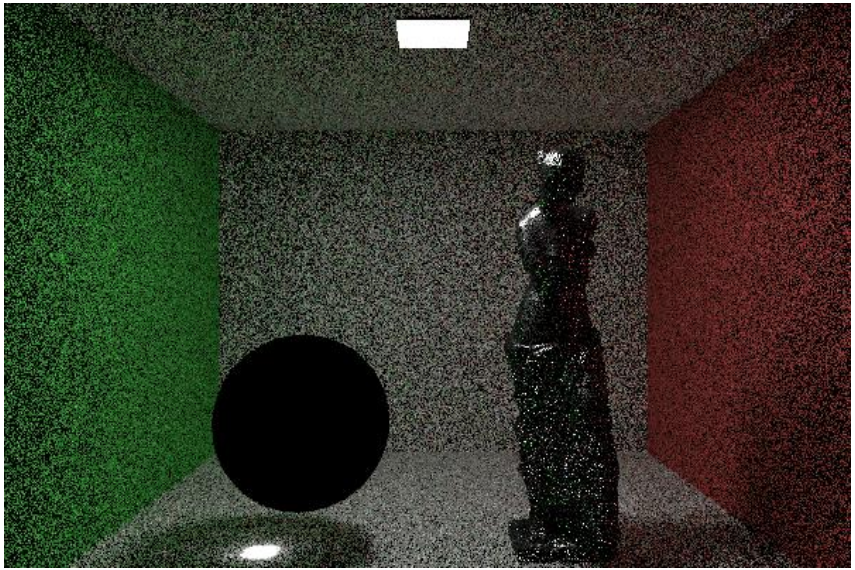
```
pathThroughput * shadingData.bsdf->evaluate(shadingData, wi) * Le
```

Light Tracing

Path Tracing

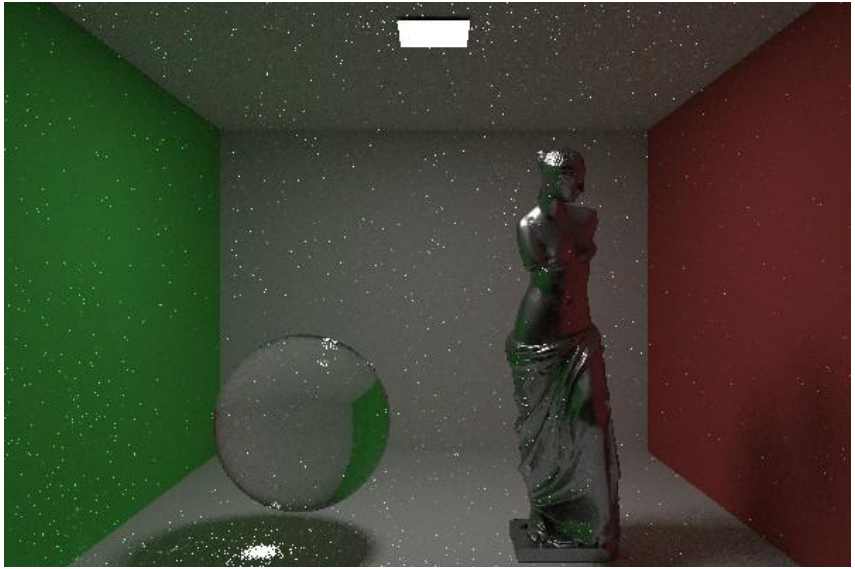


Light Tracing

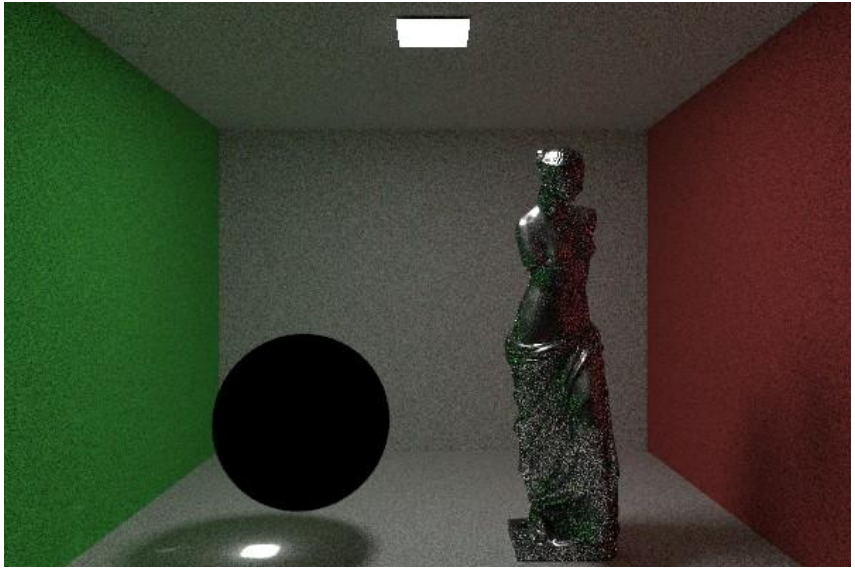


Light Tracing

Path Tracing



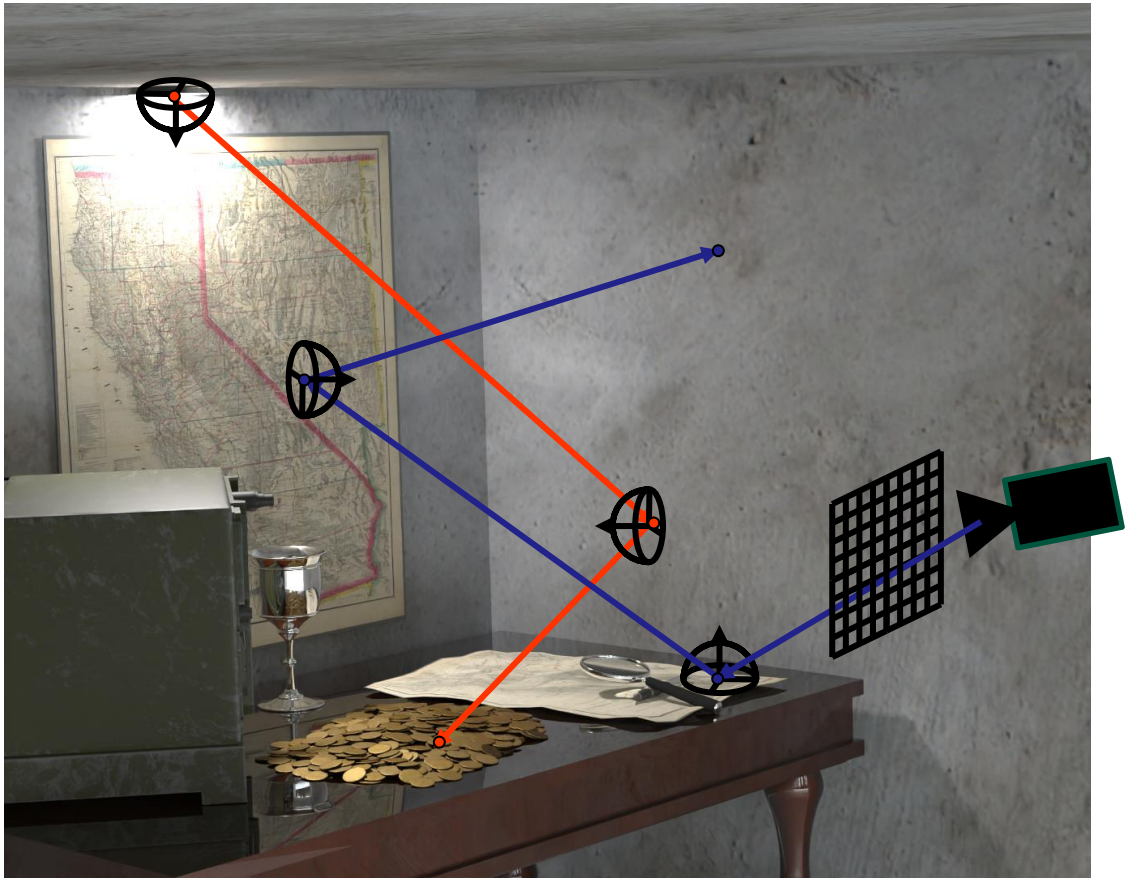
Light Tracing



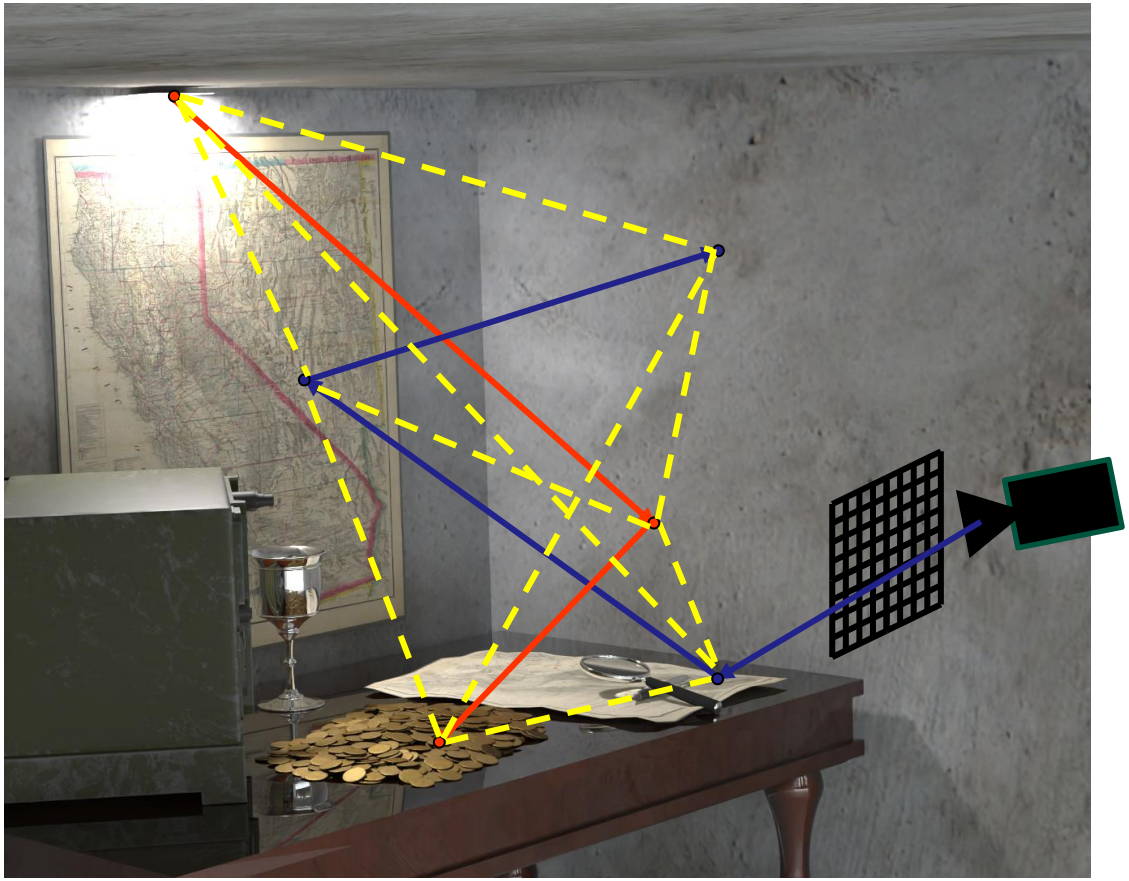
Bidirectional Path Tracing

- Combination of light tracing and path tracing
- Advantages of both
- But slower and more complicated to implement

BPT



BPT



Bidirectional Path Tracing

- Trace light path
  - Store path vertices
- Trace camera path
  - Store path vertices
- Connect each vertex
- Weight

Bidirectional Path Tracing

- Trace light path
  - Same as light tracing
  - But no connections to camera

$$\frac{L_e(x_0 \to x_1)}{p_A(x_0)} \left[\prod_{i=1}^{n_L} \frac{G(x_{i-1} \leftrightarrow x_i) f_r(x_{i-1} \to x_i \to x_{i+1})}{p_A(x_{i-1} \to x_i)} \right]$$

Bidirectional Path Tracing

- Trace camera path
  - Same as path tracing but no connections to light
  - Have relabelled path indices for convenience

$$\frac{W_e(x'_0 \to x'_1)}{p_A(x'_0 \to x'_1)} \left[\prod_{i=1}^{n_e} \frac{G(x'_i \leftrightarrow x'_{i+1}) f_r(x'_{i-1} \to x'_i \to x'_{i+1})}{p_A(x'_i \to x'_{i+1})} \right]$$

Bidirectional Path Tracing

- Connect each vertex
  - Iterate through all vertices on camera path (length t): $\{x'_0, x'_1, ...x'_t\}$
  - Iterate through all vertices on light path (length s): $\{x_0, x_1, ..., x_s\}$
  - Build full path: $\bar{x} = \{x_0, ..., x_s, x'_t, ...x'_0\}$

Bidirectional Path Tracing

- Connect each vertex
  - Need to consider:
    - BSDF at vertex s of light path and vertex t of camera path
      - Check if possible to connect (e.g. not specular BSDFs)
    - Geometry Term including visibility between vertex s of light path and vertex t of camera path

Bidirectional Path Tracing

- Connect each vertex

$$\frac{L_e(x_0 \to x_1)}{p_A(x_0)} \left[\prod_{i=1}^{s} \frac{G(x_{i-1} \leftrightarrow x_i)\, f_r(x_{i-1} \to x_i \to x_{i+1})}{p_A(x_{i-1} \to x_i)} \right]$$

$f_r(x_{s-1} \to x_s \to x'_t)$ ← BSDF at s

$G(x_s \leftrightarrow x'_t)$ ← Geometry Term

$f_r(x_s \to x'_t \to x'_{t-1})$ ← BSDF at t

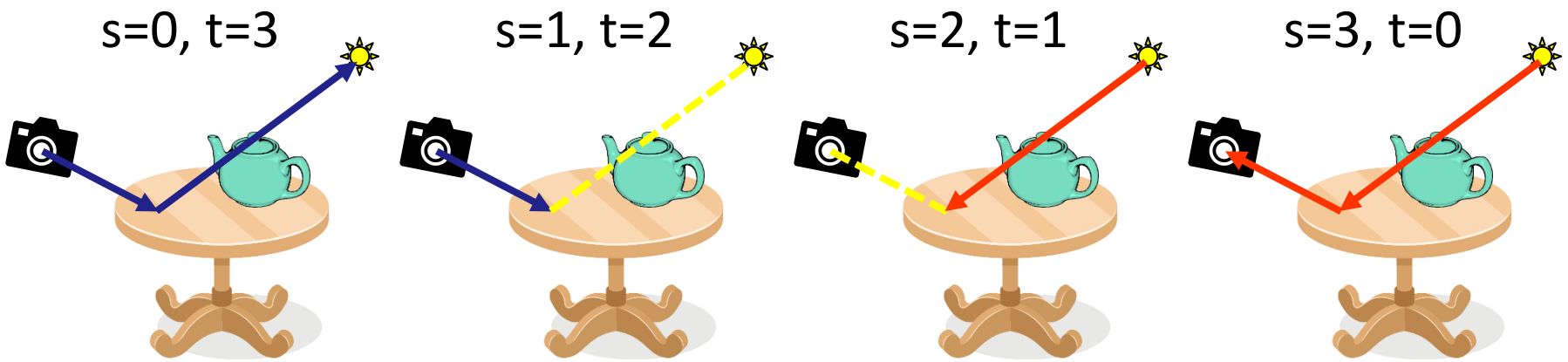
$$\left[\prod_{i=1}^{t} \frac{G(x'_i \leftrightarrow x'_{i+1})\, f_r(x'_{i-1} \to x'_i \to x'_{i+1})}{p_A(x'_i \to x'_{i+1})} \right] \frac{W_e(x'_0 \to x'_1)}{p_A(x'_0 \to x'_1)}$$

Bidirectional Path Tracing

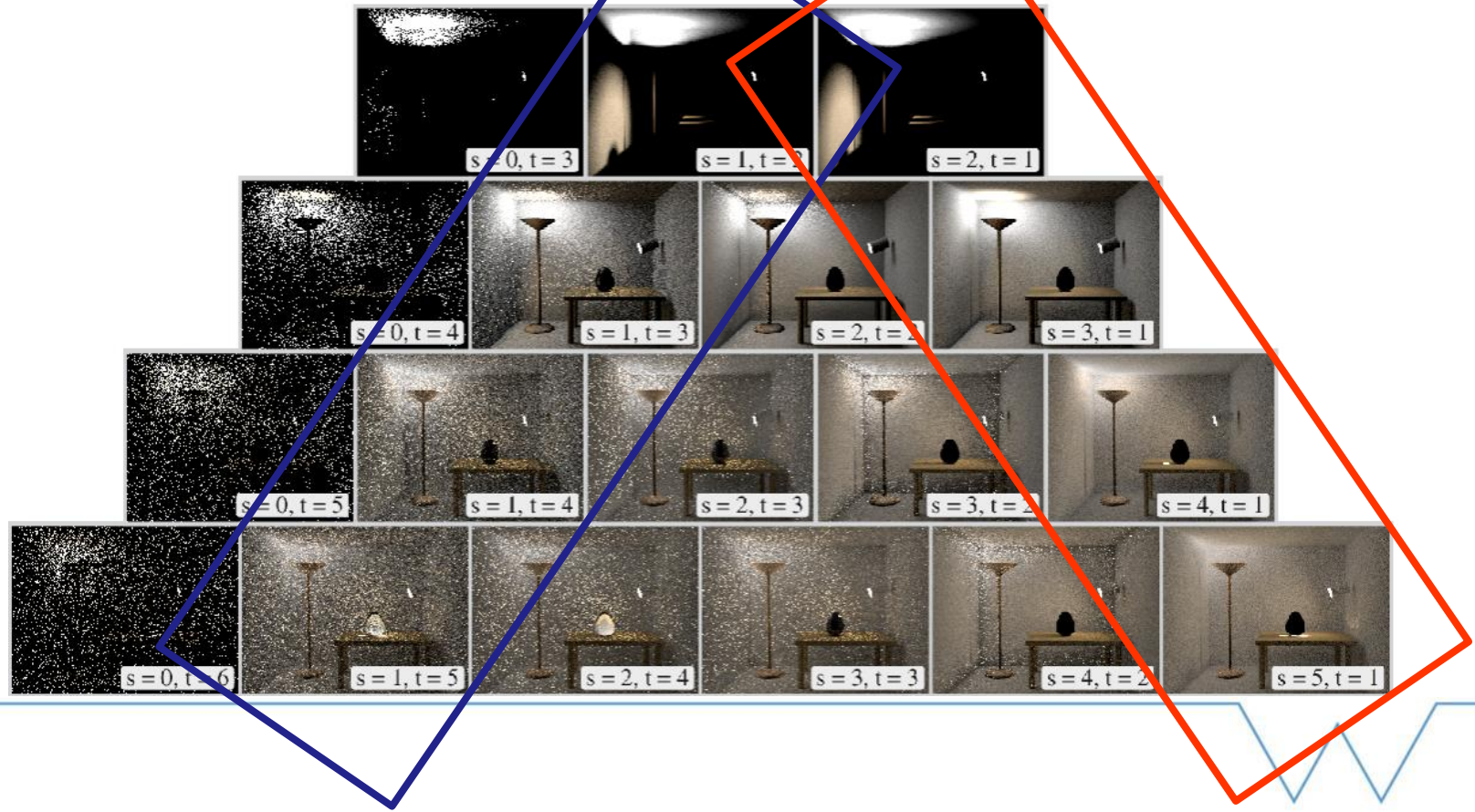
- Weight
  - We have contribution for path length l = s+t
  - But this could have been generated by sampling l = (s-1)+(t+1)
  - Have to take different ways of generating the same path into account
  - Multiple Importance Sampling

Bidirectional Path Tracing

- Weight



Bidirectional Path Tracing



Bidirectional Path Tracing

- Weight
  - PDF for one possible path

$$p_{s,t}(\bar{x}_{s+t}) = p_A(x_0)\left[\prod_{i=1}^{s} p_A(x_{i-1} \to x_i)\right]\left[\prod_{i=1}^{t} p_A(x'_i \to x'_{i+1})\right] p_A(x'_0 \to x'_1)$$

  - MIS Weight $w_{s,t}(\bar{x}_{s+t}) = \dfrac{p_{s,t}(\bar{x}_{s+t})}{\sum_{i=0}^{s+t} p_{i,(s+t)-i}(\bar{x}_{s+t})}$

Bidirectional Path Tracing

- Estimate for one path

$$w_{s,t}(\bar{x}_{s+t}) \cdot \frac{L_e(x_0 \to x_1)}{p_A(x_0)} \left[\prod_{i=1}^{s} \frac{G(x_{i-1} \leftrightarrow x_i)\, f_r(x_{i-1} \to x_i \to x_{i+1})}{p_A(x_{i-1} \to x_i)} \right]$$

$$f_r(x_{s-1} \to x_s \to x'_t)$$

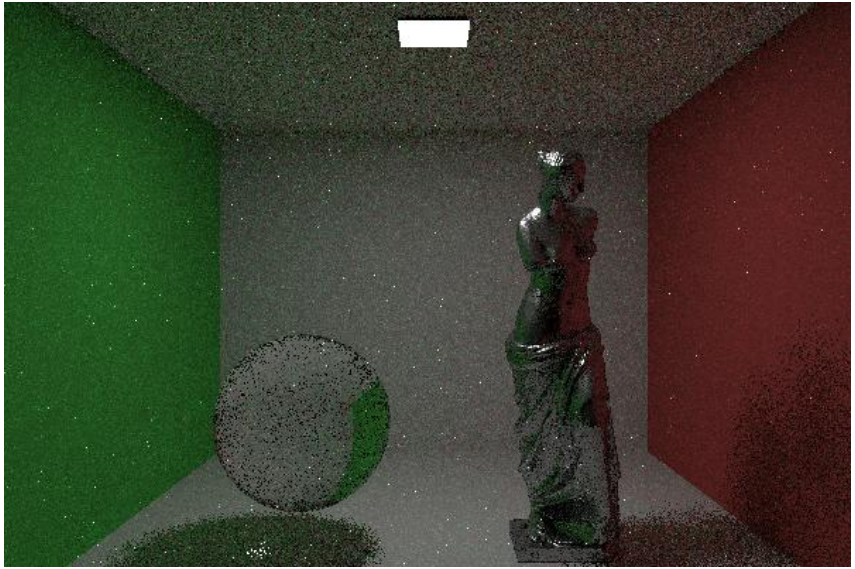
$$G(x_s \leftrightarrow x'_t)$$

$$f_r(x_s \to x'_t \to x'_{t-1})$$

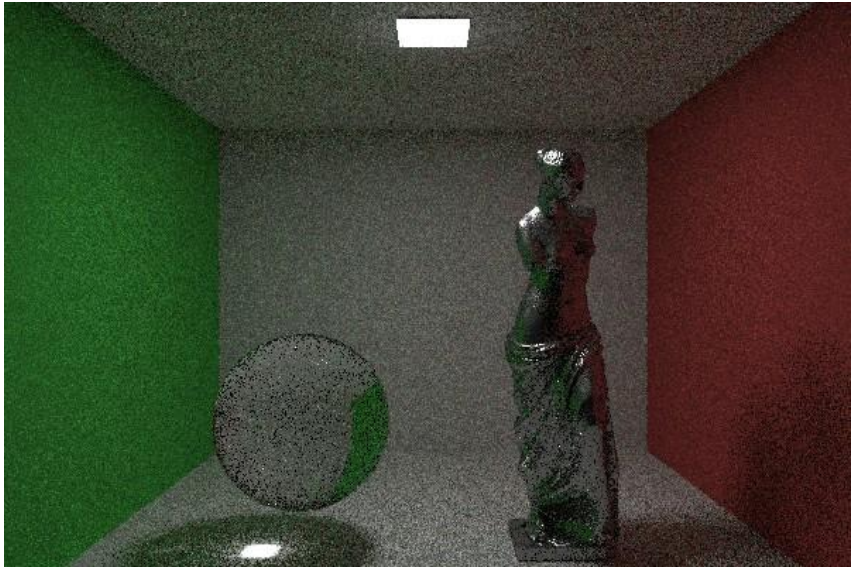
$$\left[\prod_{i=1}^{t} \frac{G(x'_i \leftrightarrow x'_{i+1})\, f_r(x'_{i-1} \to x'_i \to x'_{i+1})}{p_A(x'_i \to x'_{i+1})} \right] \frac{W_e(x'_0 \to x'_1)}{p_A(x'_0 \to x'_1)}$$

Bidirectional Path Tracing

Path Tracing

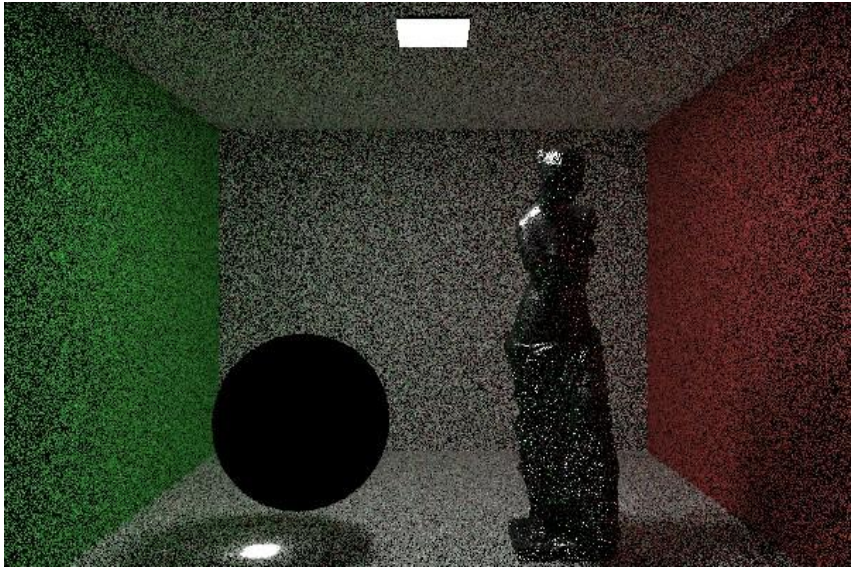


Bidirectional Path Tracing

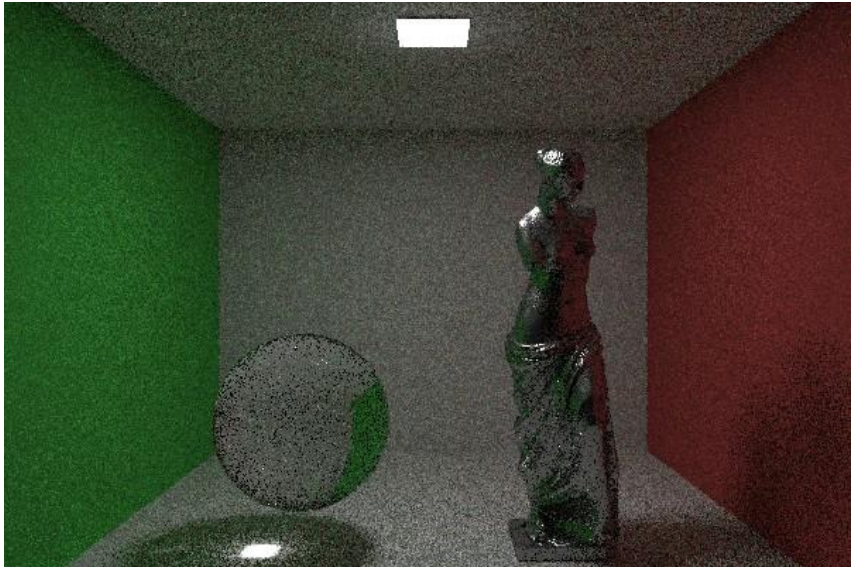


Bidirectional Path Tracing

Light Tracing

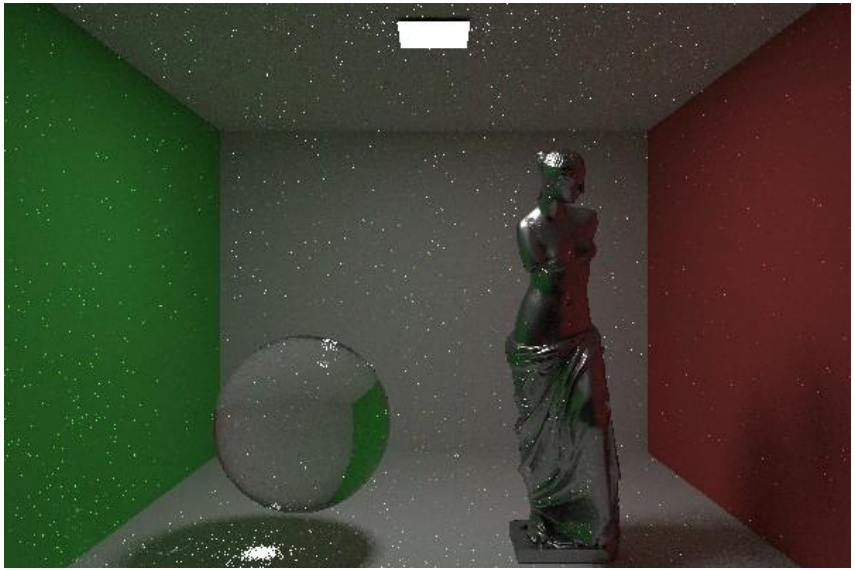


Bidirectional Path Tracing

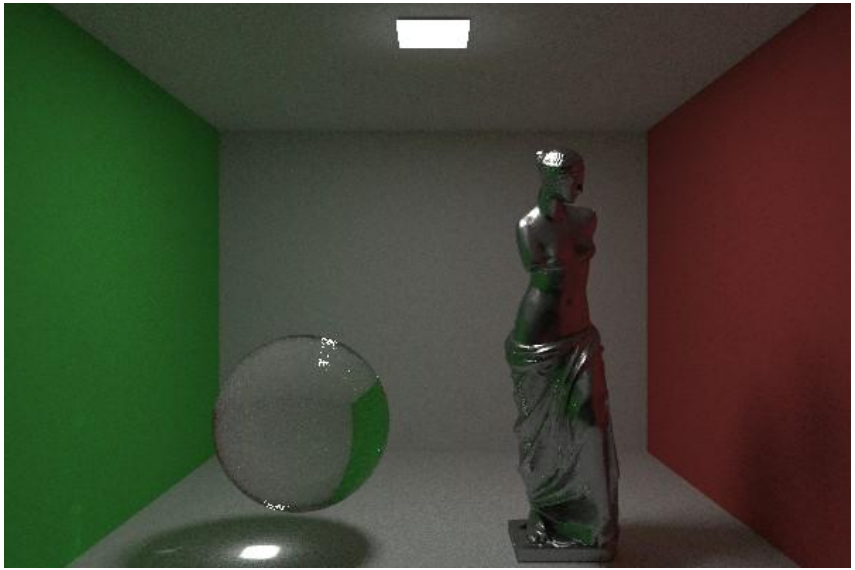


Bidirectional Path Tracing

Path Tracing



Bidirectional Path Tracing



Bidirectional Path Tracing

Path Tracing

Bidirectional Path Tracing

Bidirectional Path Tracing

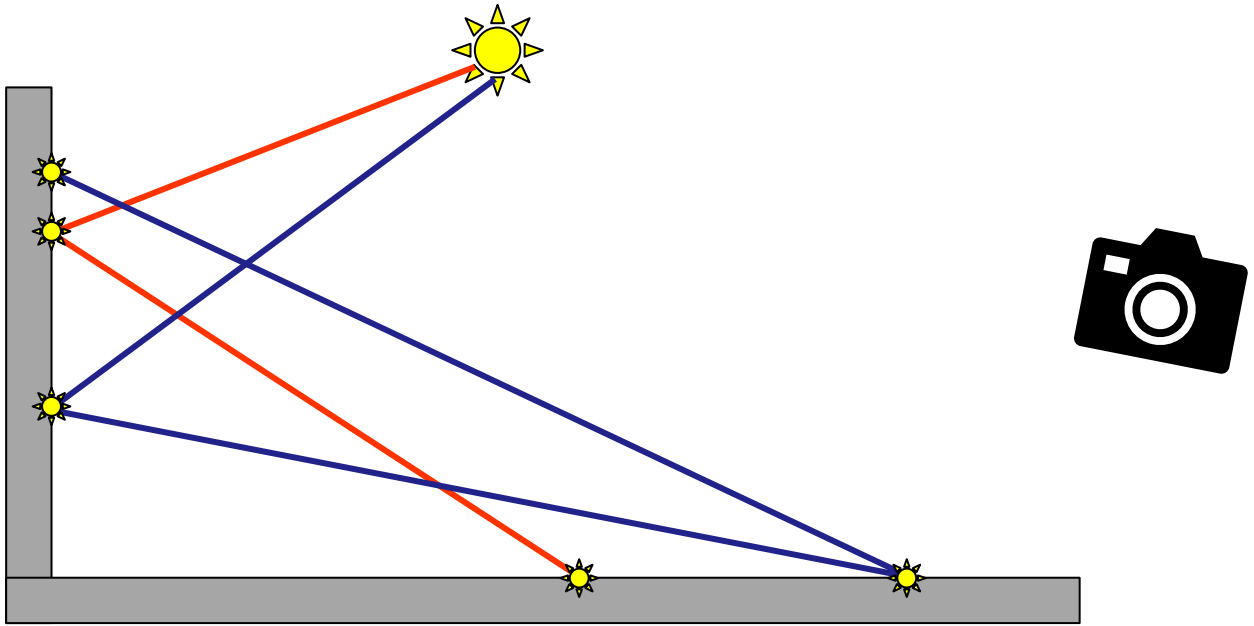
- Combines strengths of light tracing and path tracing
- Extra computation needed
- Optimizations
  - Precompute weights for MIS (complexity per pixel from $\mathcal{O}(n^4)$ to $\mathcal{O}(n^2)$)

Instant Radiosity

- Concept
  - Trace a small number of well distributed paths from the light
  - Store information at each intersection
    - Virtual Point Light (VPL)
  - Calculate contribution from each VPL to points visible from the camera

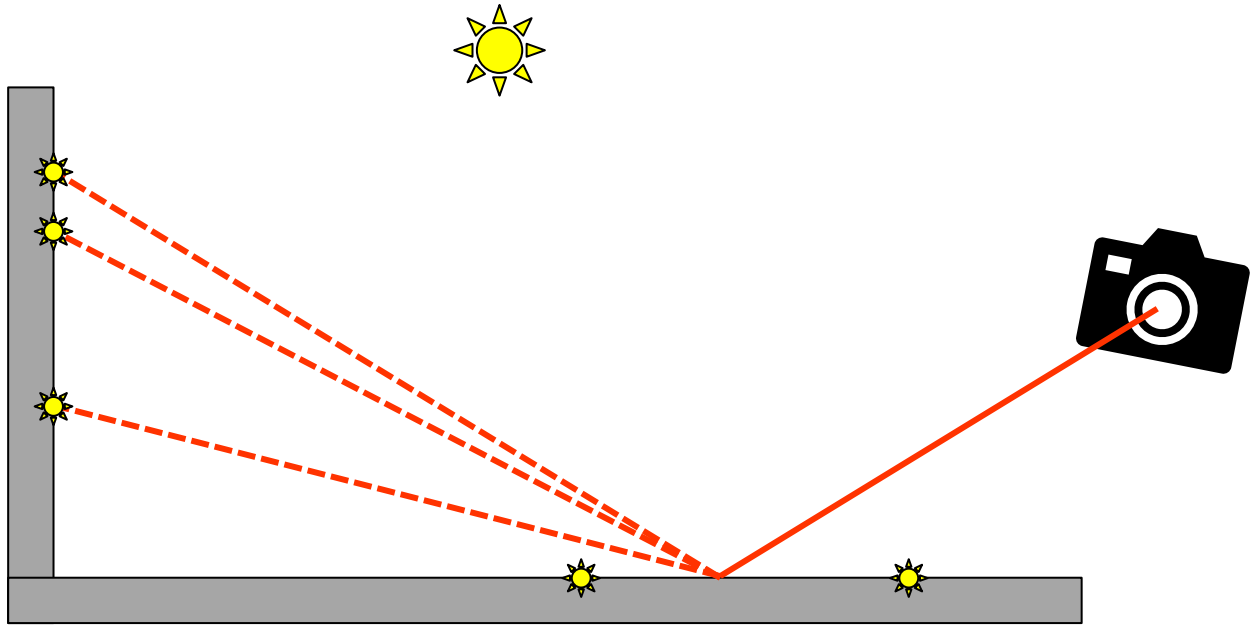
Instant Radiosity

- Pass 1: Trace paths



Instant Radiosity

- Pass 2: Compute Indirect Lighting Estimate



Instant Radiosity

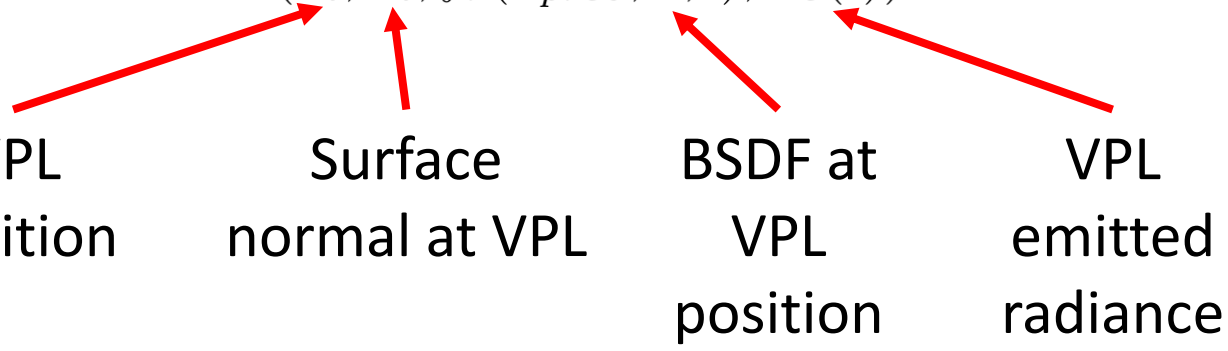
Instant Radiosity

- First pass
  - Trace a small number N_{VPL} of well distributed paths from the light
  - Store a VPL data at each interaction
  - Use quasi random sequences for tracing paths

Instant Radiosity

- Virtual Point Lights
  - Tuple of values

$$(x_i, n_i, f_r(x_{prev}, x, \cdot), L_e(i))$$



Instant Radiosity

- Second pass
  - Trace a path from the camera to the first non-specular vertex
  - Calculate direct light from each VPL

$$L_o(x \to x_{camera}) = \sum_{i=1}^{N_{VPL}} f_r(x_{prev} \to x_i \to x) G(x_i \leftrightarrow x) f_r(x_i \to x \to x_{camera}) L_{VPL}(i)$$

Instant Radiosity

- Handle direct lighting as usual
- Sum over N_{VPL} lights can be expensive
  - How big should it be?
- Interleaved sampling and filtering can improve

Instant Radiosity

- Unbiased but artifacts
  - Structured noise
  - Weak singularity in the Geometry Term
  - Cannot capture caustics
- Equivalent to Bidirectional Path Tracing for t=1 and shared light paths

Instant Radiosity

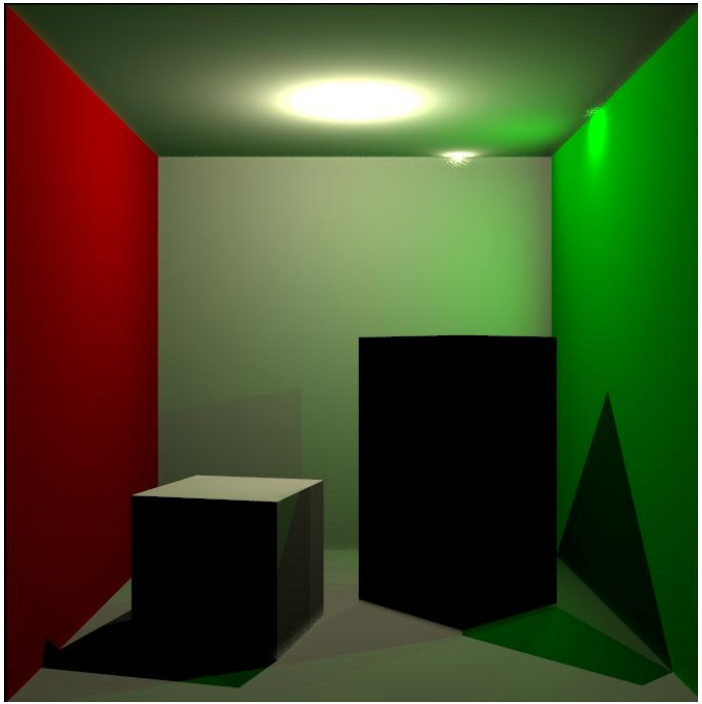
- Implementation
  - Suggest to add a VPL class
    - Store
```
ShadingData shadingData;
Colour Le;
```
    - ShadingData created at each interaction when creating VPLS
    - Or from when creating VPL on light source
      - For area lights, initialize with `ShadingData(p, light->normal(p));`
      - Where `p = light->samplePositionFromLight(sampler, pdfPosition);`

Instant Radiosity

- Implementation
  - Second pass (can be parallelized)
  - For each pixel
    - Trace ray
    - Iterate over all stored VPLs
    - Compute Contribution

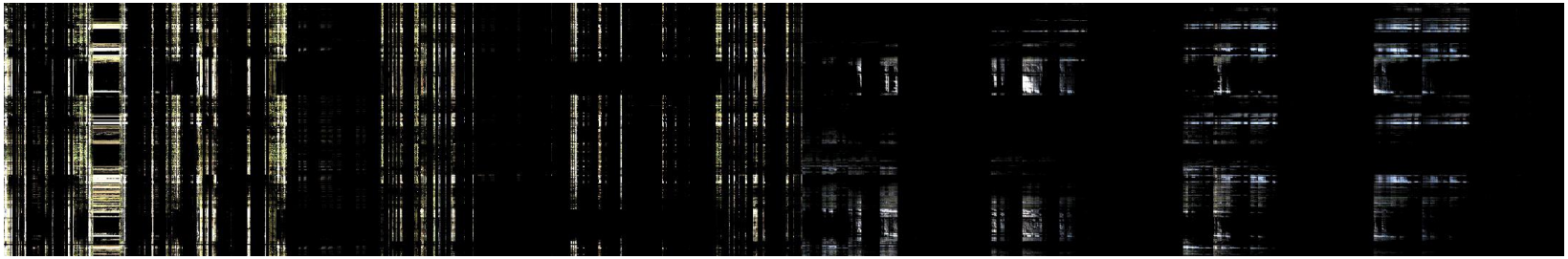
Instant Radiosity

- Artifacts
  - Structured noise if too few VPLS
  - Geometry Term singularities



Instant Radiosity

- Improvements
  - Trace extra bounces
  - Many light methods

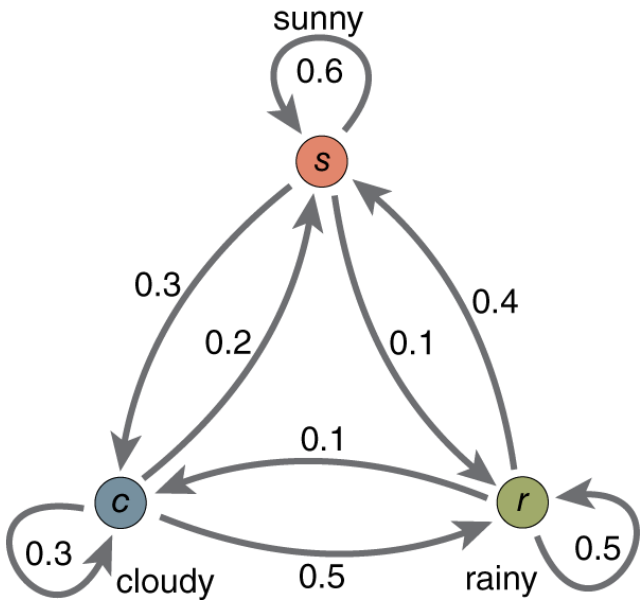


Metropolis Light Transport

- What if we could generate samples distributed proportionally to the integrand?
- Samples could always contribute, images would render faster?
- Can achieve this via the Metropolis-Hastings algorithm

Metropolis Light Transport

- Markov Chains
  - Multiple States $\bar{x}$
  - Transition between states
  - Want to visit each state proportional to its value



Metropolis Light Transport

- Detailed Balance

$$f(\bar{x})T(\bar{x} \to \bar{y})a(\bar{x} \to \bar{y}) = f(\bar{y})T(\bar{y} \to \bar{x})a(\bar{y} \to \bar{x})$$

- Balances the probability of transitioning between states
  - Based on their values $f(\bar{x})$
  - Transition Density $T(\bar{x} \to \bar{y})$

Metropolis Light Transport

- Detailed Balance

$$f(\bar{x})T(\bar{x} \to \bar{y})a(\bar{x} \to \bar{y}) = f(\bar{y})T(\bar{y} \to \bar{x})a(\bar{y} \to \bar{x})$$

- Need extra probability to ensure balance
  - Acceptance Probability $a(\bar{x} \to \bar{y})$
- Need an expression for $a(\bar{x} \to \bar{y})$

Metropolis Light Transport

- Detailed Balance

$$f(\bar{x})T(\bar{x} \to \bar{y})a(\bar{x} \to \bar{y}) = f(\bar{y})T(\bar{y} \to \bar{x})a(\bar{y} \to \bar{x})$$

- Can rearrange, and can be shown that:

$$a(\bar{x} \to \bar{y}) = \min\left(\frac{f(\bar{y})T(\bar{y} \to \bar{x})}{f(\bar{x})T(\bar{x} \to \bar{y})}, 1\right)$$

  – Probability chain will move from state $\bar{x}$ to state $\bar{y}$

Metropolis Light Transport

- If symmetric transition function

$$T(\bar{x} \to \bar{y}) = T(\bar{y} \to \bar{x})$$

- Then

$$a(\bar{x} \to \bar{y}) = \min\left(\frac{f(\bar{y})}{f(\bar{x})}, 1\right)$$

Metropolis Light Transport

- Algorithm
  - Initial state $\bar{x} = \bar{x}_0$
  - For i = 1 to N
    - Mutate $\bar{x}$: $\bar{y} \sim T(\bar{x} \to \bar{y})$
    - Compute $a(\bar{x} \to \bar{y}) = \min\left(\frac{f(\bar{y})T(\bar{y} \to \bar{x})}{f(\bar{x})T(\bar{x} \to \bar{y})}, 1\right)$
    - Record $\bar{x}$
    - if $\xi < a(\bar{x} \to \bar{y})$

      $\bar{x} = \bar{y}$

Metropolis Light Transport

- Updated Path integral formulation to include contribution to j'th pixel

$$I_j = \int_{\mathcal{P}} h_j(\bar{x}) f(\bar{x}) d\mu(\bar{x})$$

- In MLT, samples are generated proportional to their contribution

Metropolis Light Transport

- How do we add colour
  - Path contribution over spectral or RGB values
  - Need a target distribution $f^* : (R, G, B) \mapsto \mathbb{R}^+$
    - Generally use luminance

$$f^*(R, G, B) = 0.2126R + 0.7152G + 0.0722B$$

    - i.e.

$$a(\bar{x} \to \bar{y}) = \min\left(\frac{f^*(\bar{y})T(\bar{y} \to \bar{x})}{f^*(\bar{x})T(\bar{x} \to \bar{y})}, 1\right)$$

Metropolis Light Transport

- Estimator $$E[I_j] = \frac{1}{N} \sum_{i=1}^{N} \frac{h_j(\bar{x}_i) f(\bar{x}_i)}{p(\bar{x}_i)}$$

- Generate samples proportional to integrand so $$p(\bar{x}_i) = \frac{f^*(\bar{x}_i)}{\int_{\mathcal{P}} f^*(\bar{x}) d\mu(\bar{x})} = \frac{f^*(\bar{x}_i)}{b}$$

- Where $b \in \mathbb{R}$ is a normalization constant

Metropolis Light Transport

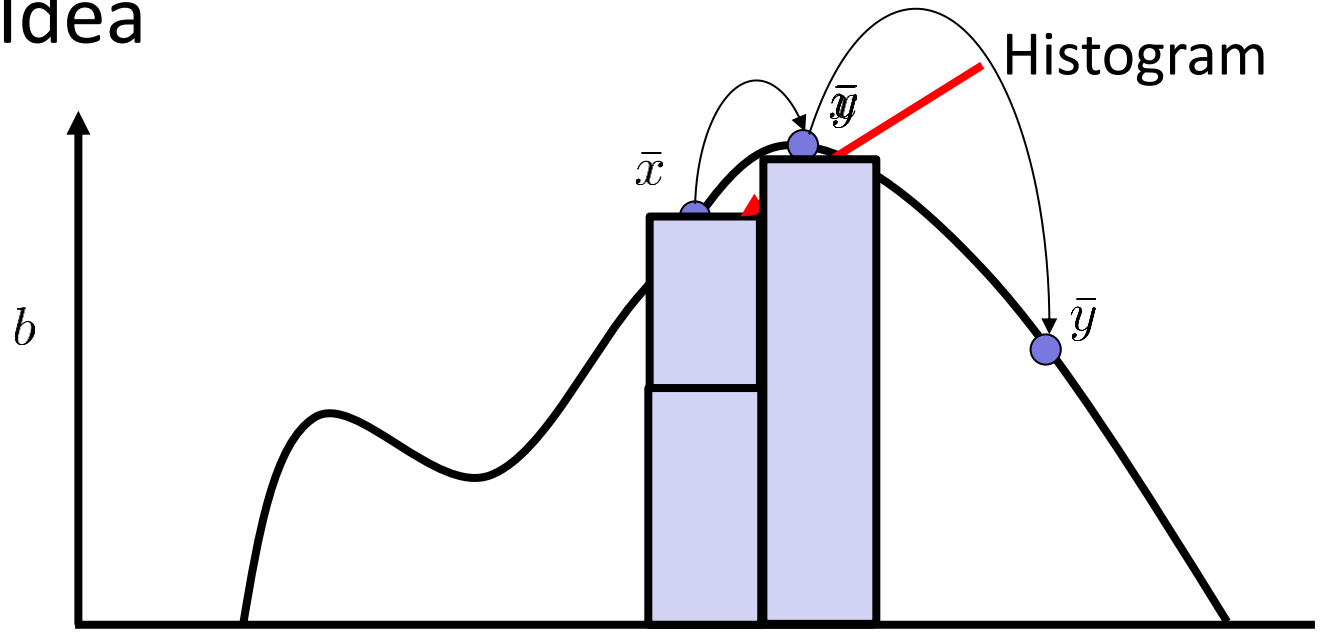
- Substitute into estimator

$$E[I_j] = \frac{1}{N}\sum_{i=1}^{N}\frac{h_j(\bar{x}_i)f(\bar{x}_i)}{\frac{f^*(\bar{x}_i)}{b}} = \frac{b}{N}\sum_{i=1}^{N}\frac{h_j(\bar{x}_i)f(\bar{x}_i)}{f^*(\bar{x}_i)}$$

- Note integrating across screen
- Results in a histogram in screen space
  - b scales it to the correct value

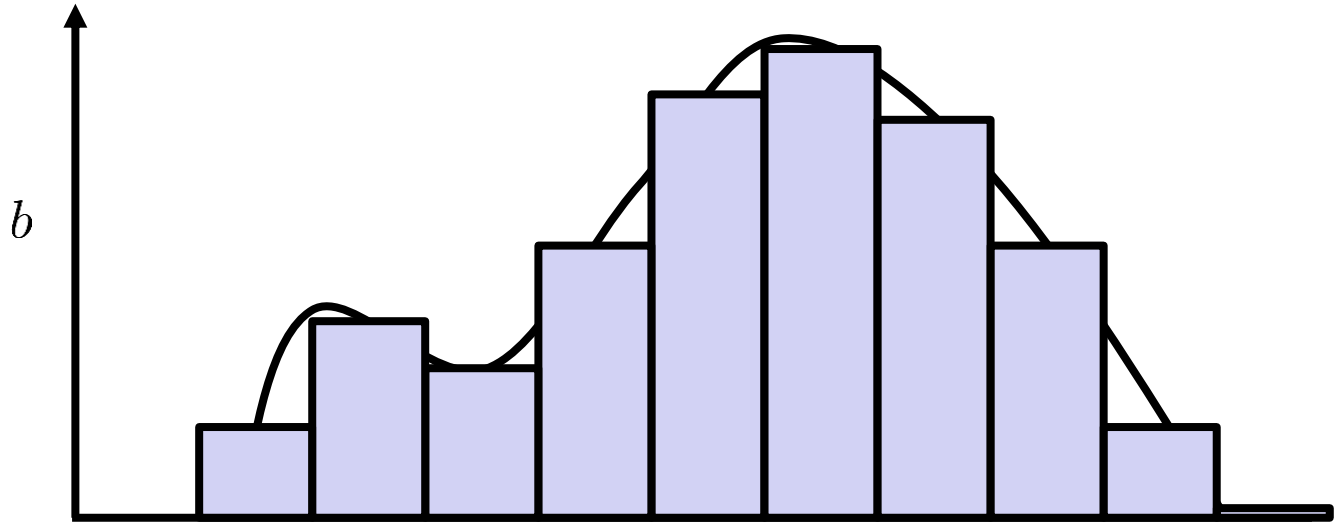
Metropolis Light Transport

- Idea



Metropolis Light Transport

- Idea



Metropolis Light Transport

- Need to estimate b
- Use existing algorithm (e.g. bidirectional path tracing or path tracing)
  - Trace M random paths before rendering
  - Average their contribution to b
- How big should M be? What if b is 0

Metropolis Light Transport

- How do we initialize to the stationary distribution
  - Need initial sample $\bar{x}_0$
- Burn in
  - Run MLT for many iterations, discard previous values, assume at stationary distribution
  - Inefficient

Metropolis Light Transport

- How do we initialize to the stationary distribution
- Resampling
  - Generate set of candidates $X = \{\bar{x}_{init(0)}, \bar{x}_{init(1}, ..., \bar{x}_{init(N}\}$
  - Sample one proportional to its contribution

$$\bar{x}_0 \propto f(\bar{x}_{init(i)})$$

  - Distributed close to detailed balance

Metropolis Light Transport

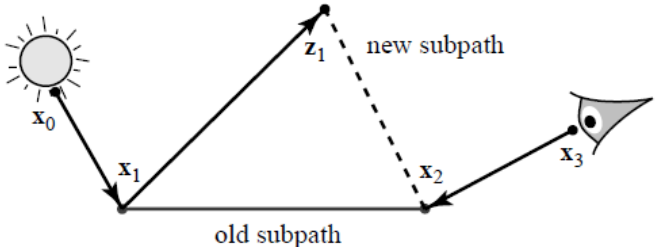
- Transition Densities
  - Can be designed to explore space
  - Need to be:
    - Ergodic
    - Aperiodic
    - Respect Detailed Balance

Metropolis Light Transport

- Transition Densities
  - Bidirectional
  - Lens
  - Caustic
  - Multi-Chain

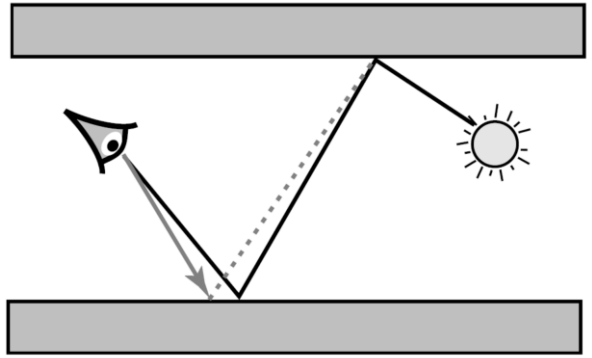
Metropolis Light Transport

- Bidirectional
  - Ensures ergodicity
  - Probabilistically deletes and adds subpaths
  - Traces via BSDF sampling
  - Probability of regenerating whole path or subpaths



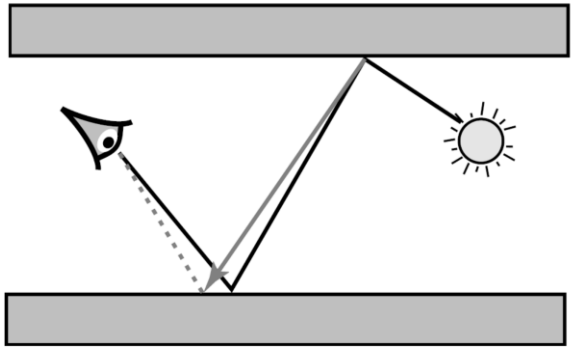
Metropolis Light Transport

- Transition Densities
  - Lens
  - Perturbs point on image plane
  - Traces subpath from camera to the first non-specular vertex
  - Connect to unchanged path



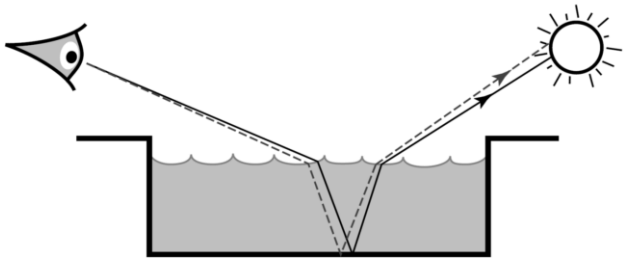
Metropolis Light Transport

- Transition Densities
  - Caustic
  - Perturbs direction on hemisphere on non-specular vertex before specular chain
  - Trace until vertex before camera



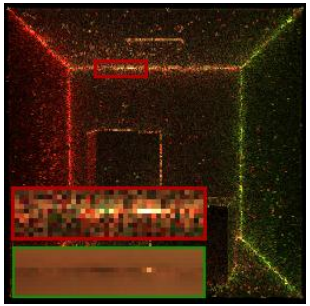
Metropolis Light Transport

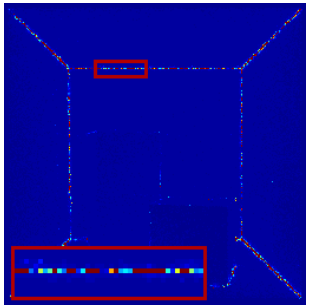
- Transition Densities
  - Multi-Chain
  - Handles chains of specular interactions with non-specular vertices between chains
  - Both perturbations on image plane and hemisphere



Metropolis Light Transport

- Transition Densities
  - Very efficient to compute
  - But usually have an extra geometry term in because of connection to unchanged path





Metropolis Light Transport

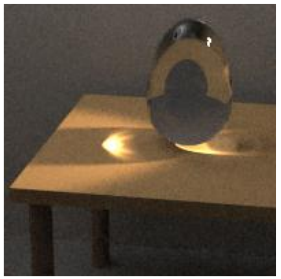
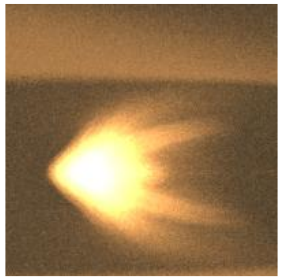
- Reusing accepted values
  - Both new state and old state contain using light path information
  - Can combine via acceptance probability

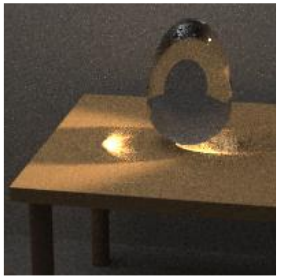
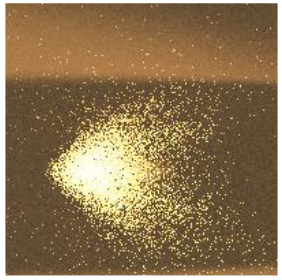
$$E[I_j] = \frac{1}{N}\sum_{i=1}^{N} \frac{a(\bar{(}x) \to \bar{y})h_j(\bar{y}_i)f(\bar{y}_i)}{p(\bar{y}_i)} + \frac{(1 - a(\bar{(}x) \to \bar{y}))h_j(\bar{x}_i)f(\bar{x}_i)}{p(\bar{x}_i)}$$

Metropolis Light Transport

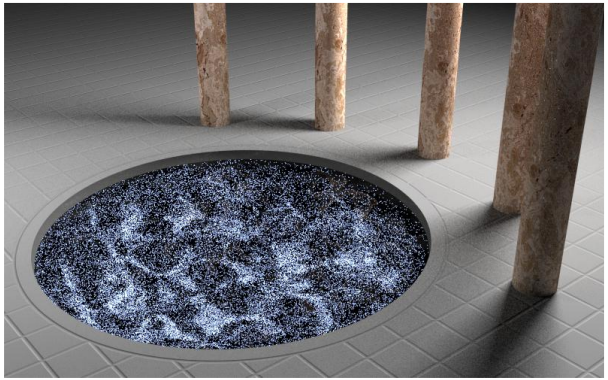
- Reusing accepted values
  - Initial state $\bar{x} = \bar{x}_0$
  - For i = 1 to N
    - Mutate $\bar{x}$: $\bar{y} \sim T(\bar{x} \to \bar{y})$
    - Compute $a(\bar{x} \to \bar{y}) = \min\left(\frac{f(\bar{y})T(\bar{y} \to \bar{x})}{f(\bar{x})T(\bar{x} \to \bar{y})}, 1\right)$
    - Record $\bar{y} \cdot a(\bar{x} \to \bar{y})$
    - Record $\bar{x} \cdot (1 - a(\bar{x} \to \bar{y}))$
    - if $\xi < a(\bar{x} \to \bar{y})$
      $\bar{x} = \bar{y}$

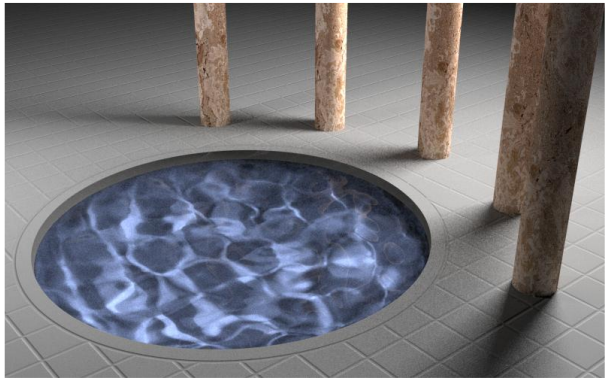
Metropolis Light Transport

Metropolis Light Transport



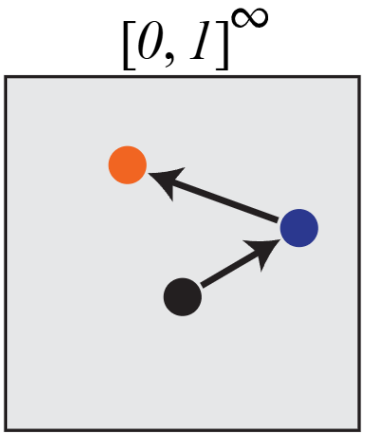


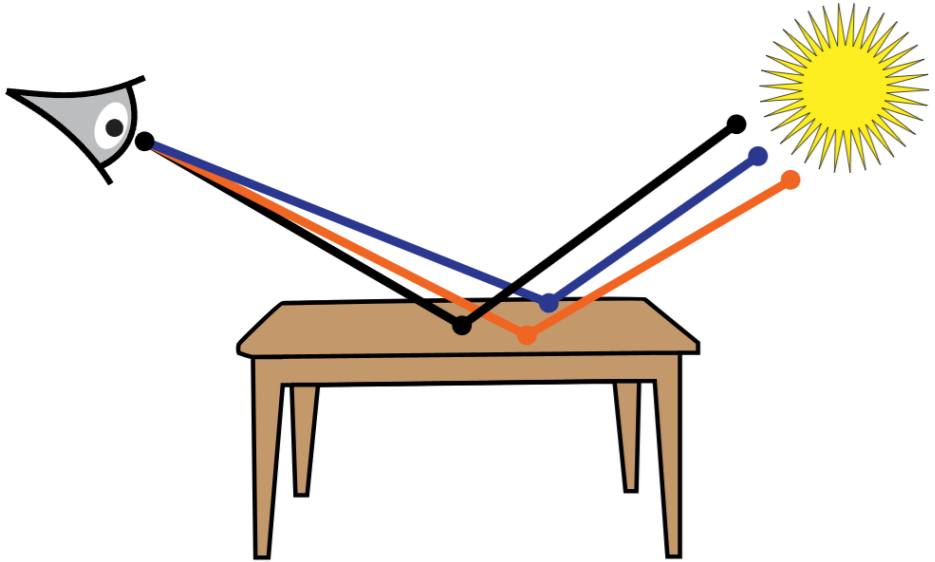
Metropolis Light Transport

- Primary Sample Space (PSS)
  - Recall all sampling depends on random numbers
  - Each path corresponds to a vector of random numbers $\bar{u}$ in the infinite dimensional unit hypercube $\mathcal{U} = [0,1]^\infty$
  - First two (u_0, u_1) are position on screen

Metropolis Light Transport

- Primary Sample Space (PSS)





Metropolis Light Transport

- Primary Sample Space (PSS)
  - Recall path space contribution

$$f(\bar{x}) = L_e(x_0 \to x_1) \left[\prod_{i=1}^{L-2} G(x_{i-1} \leftrightarrow x_i) f_r(x_{i-1} \to x_i \to x_{i+1}) \right] G(x_{L-1} \leftrightarrow x_L) W_e(x_{L-1} \to x_L)$$

  - Need change of variables to PSS

Metropolis Light Transport

- Primary Sample Space (PSS)
  - Map from random numbers to path space

$$\bar{x} = CDF^{-1}(\bar{u})$$

  - Can write (when sampling) $f(\bar{x}) = \frac{f(\bar{x})}{p(\bar{x})} p(\bar{x}) = C(\bar{x}) p(\bar{x})$
    - Recall path throughput

$$C(\bar{x}) = \prod_{j=1} \frac{f_r(x_j, \omega_{i,j}, \omega_o) \cos(\theta_j)}{p_\Omega(\omega_{i,j})}$$

Metropolis Light Transport

- Primary Sample Space (PSS)
  - Map between domains $C(\bar{x}) = C(CDF^{-1}(\bar{u})) = \hat{C}(\bar{u})$
  - So $C(\bar{x})p(\bar{x})d\mu(\bar{x}) = \hat{C}(\bar{u})p(\bar{u})\left|\frac{d\mu(\bar{x})}{d\bar{u}}\right|d\bar{u} = \hat{C}(\bar{u})d\bar{u}$
- Rewrite path integral

$$I_j = \int_{\mathcal{U}} h_j(\bar{u})\hat{C}(\bar{u})d\bar{u}$$

Metropolis Light Transport

- Primary Sample Space (PSS)
- Detailed balance

$$\hat{C}^*(\bar{u})T(\bar{u} \to \bar{u}')a(\bar{u} \to \bar{u}') = \hat{C}^*(\bar{u}')T(\bar{u}' \to \bar{u})a(\bar{u}' \to \bar{u})$$

- Acceptance Probability

$$a(\bar{u} \to \bar{u}') = \min\left(\frac{\hat{C}^*(\bar{u}')T(\bar{u}' \to \bar{u})}{\hat{C}^*(\bar{u})T(\bar{u} \to \bar{u}')}, 1\right)$$

Metropolis Light Transport

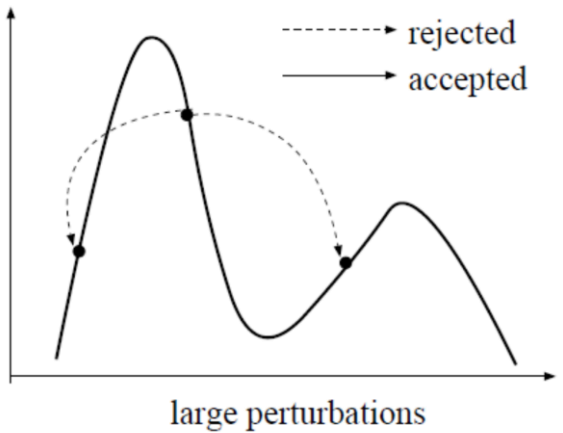
- Primary Sample Space Metropolis Light Transport (PSSMLT)
  - Estimator $$E[I_j] = \frac{b}{N} \sum_{i=1}^{N} \frac{h_j(\bar{u}_i)\hat{C}(\bar{u}_i)}{\hat{C}^*(\bar{u}_i)}$$
    - Where $\hat{C}^*(\bar{u}_i)$ is the target distribution (e.g. luminance of the path as before)

Metropolis Light Transport

- Primary Sample Space Metropolis Light Transport (PSSMLT)
  - Algorithm similar to before
  - Need Transition functions for PSS
  - Need to be ergodic and aperiodic

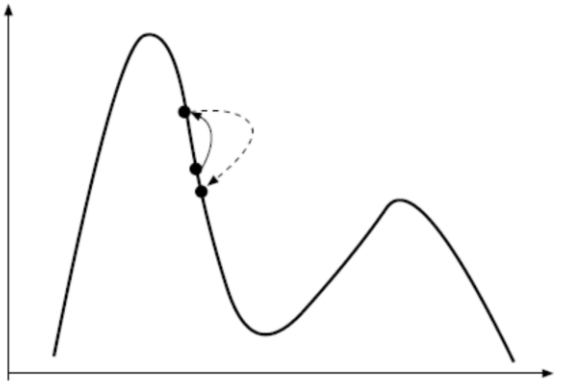
Metropolis Light Transport

- PSSMLT
  - Large Step Perturbation
  - Resample new $\bar{u}$
    - Generates a new path



Metropolis Light Transport

- PSSMLT
  - Small Step Perturbation
  - Explore region around $\bar{u}$
  - Usually symmetric Transition function



$$\Delta u = s_1 \cdot e^{-ln\frac{s_1}{s_2}\cdot\xi_1}$$

$$u' = \begin{cases} (u+\Delta u)\%1 & \text{if } \xi_2 < 0.5 \\ (u-\Delta u)\%1 & \text{otherwise} \end{cases}$$

Metropolis Light Transport

- PSSMLT
  - Small Step Perturbation
  - Mod 1 ensures the values of $\bar{u}$ stay in $\mathcal{U} = [0,1]^\infty$
  - Parameters: $s_1 = \frac{1}{64}$, $s_2 = \frac{1}{1024}$ (original)
  - Can use other perturbations
    - Uniform in range
    - Gaussian

Metropolis Light Transport

- PSSMLT
  - Probabilistically sample large or small steps

$$p_{large} \in [0.3, 0.7]$$

$$p_{small} = 1 - p_{large}$$

$p_{large} = 0.02$ 

$p_{large} = 0.1$ 

$p_{large} = 0.5$ 

$p_{large} = 0.9$ 

$p_{large} = 0.98$ 

Metropolis Light Transport

- PSSMLT
- Acceptance Probability
  - Both Large Step and Small Step perturbations are symmetric

$$a(\bar{u} \rightarrow \bar{u}') = \min\left(\frac{\hat{C}^*(\bar{u}')}{\hat{C}^*(\bar{u})}, 1\right)$$

Metropolis Light Transport

- PSSMLT
  - Implementation details
    - Need to store a vector of random numbers
      - Theoretically infinite length, but in reality only finite needed
      - e.g. `std::vector<float> psscoords;`
    - Each path starts random numbers from index 0
    - Usually implemented as a sampler similar to `MTRandom`
    - Have to store index of current sample used

Metropolis Light Transport

- PSSMLT
  - Implementation details
    - Every time a new random number is needed (e.g. sampling hemisphere)
      - If index in list
        - » Perturb random number at index
      - Else
        - » Generate new random number entry at end of vector

Metropolis Light Transport

- PSSMLT
  - Implementation details
    - All coordinates of path need to be perturbed in small step, even ones not generated yet
    - Usually use a timer

```
std::vector<float> psstimer;
```

      - Reset timer on large step perturbations (i.e. clear vectors)
        - » Note this requires a copy of the original vectors to be stored due to probabilistic acceptance

Metropolis Light Transport

- PSSMLT
  - Implementation details
    - For small steps
      - Increment timer
      - Store current perturbation time for each random number
      - Ensure perturbation time == timer for each used random number

```
while (times[index] < timer)
{
    psscoords[index] = mutate(rc, psscoords[index]);
    timesr[index]++;
}
```

Metropolis Light Transport

- PSSMLT
  - Implementation details
    - Why?
    - Recall each point in unit hypercube is a path
    - In theory need to perturb all coordinates
    - In practice only need to perturb used coordinates, but have to account for previous successful small step perturbations when adding new coordinates

Metropolis Light Transport

- PSSMLT
  - Implementation details
    - Usually implement a PSS class PSS
    - Use via $PSS(\bar{u}') \sim T(PSS(\bar{u}) \rightarrow PSS(\bar{u}'))$
    - With acceptance probability $a(\bar{u} \rightarrow \bar{u}') = \min\left(\frac{\hat{C}^*(\bar{u}')}{\hat{C}^*(\bar{u})}, 1\right)$
    - Record as usual
    - if $\xi < a(\bar{u} \rightarrow \bar{u}')$
      $PSS(\bar{u}) = PSS(\bar{u}')$

Metropolis Light Transport

- PSSMLT
  - Can weight with MIS between large step and small steps, and reuse accepted values
  - Needs information if large step used

$$\mathbb{I}_{large} = \begin{cases} 1 & \text{if} \quad \text{Large Step Selected} \\ 0 & \text{otherwise} \end{cases}$$

Metropolis Light Transport

- PSSMLT
  - Record $\bar{u}'$ with weight $\dfrac{a(\bar{u} \to \bar{u}') + \mathbb{I}_{large}}{\frac{C^*(\bar{u}')}{b} + p_{large}}$
  - Record $\bar{u}$ with weight $\dfrac{(1 - a(\bar{u} \to \bar{u}')}{\frac{C^*(\bar{u})}{b} + p_{large}}$

Metropolis Light Transport

- PSSMLT with Weighting

Metropolis Light Transport

- Several Extensions
  - Participating Media (Pauly et al)
  - Improved handling of specular surfaces (Jakob)
  - Use of local gradients (Li)
  - Use of Half Vectors (Kaplanyan)
  - MIS between techniques (Hachisuka)
  - Combination of MLT and PSSMLT (Bitterli)
  - Use of multiple Markov Chains (Me)

Photon Mapping

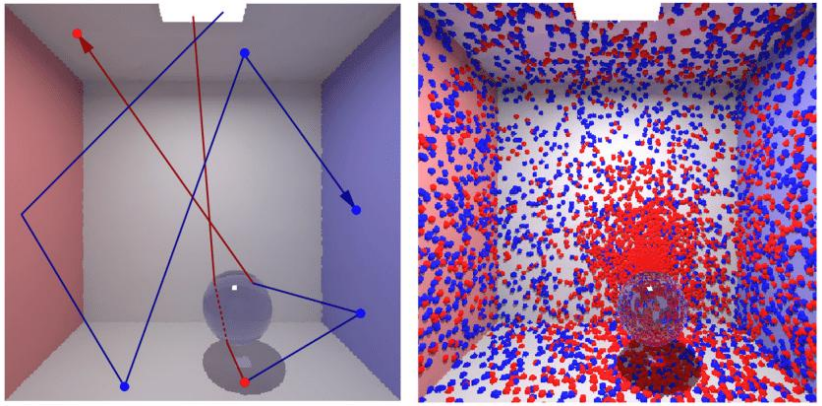
- Two pass method
  - First pass traces particles (photons) from the light
  - Second pass gathers particles at hitpoints from eye and estimates lighting
- Can estimate most illumination effects
- Biased but consistent

Motivation

- Model capable of handling
  - All BRDFs
  - All geometry
- All global illumination effects are supported
- Computationally cheaper than other methods

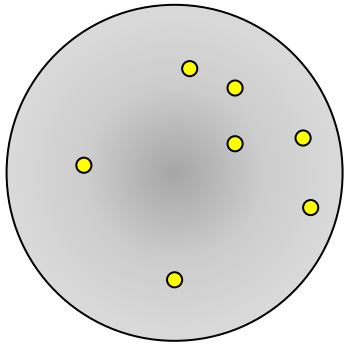
Photon Mapping Overview

- First pass: Shoot photons
  - Emit photons
  - Trace photons
  - Store in Photon Map



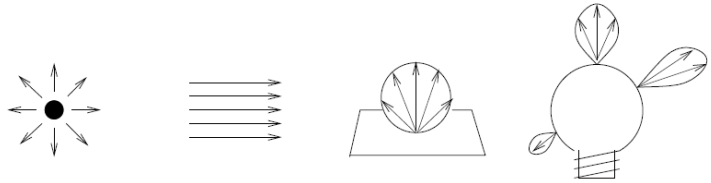
Photon Mapping Overview

- Second Pass: Ray trace from eye
  - Depending on material
  - Trace rays further OR
    - Access photon map
    - Compute Density estimation



Shooting Photons

- Shoot fixed number N_p photons
- For each photon
  - Sample light
  - Sample outgoing direction
  - Recursively trace into the scene
  - Similar to light tracing



Photon Weight

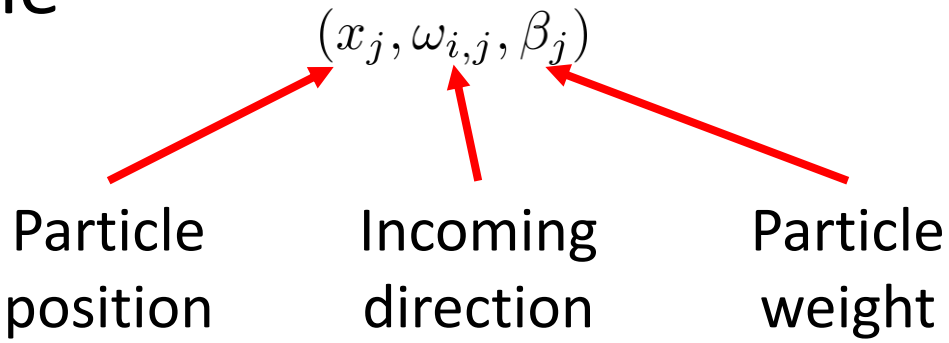
- Weight computed via tracing paths

$$\beta = \frac{L_e(x_0 \to x_1)}{p_A(x_0)} \prod_{i=1}^{L-2} \frac{1}{1-q_i} \frac{G(x_{i-1} \leftrightarrow x_i) f_r(x_{i-1} \to x_i \to x_{i+1})}{p_A(x_{i-1} \to x_i)}$$

- Or via solid angle

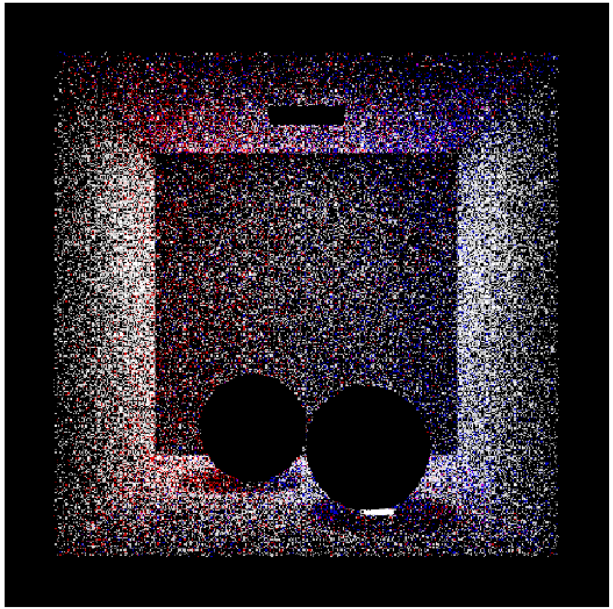
Photon Storage

- At each intersection store tuple for j’th particle



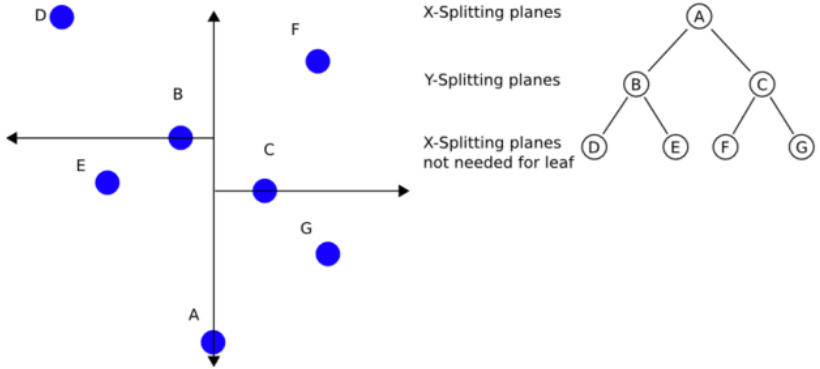
Photon Storing

- Photons stored on non-specular surfaces
  - Position
  - Incident Direction
  - Photon power



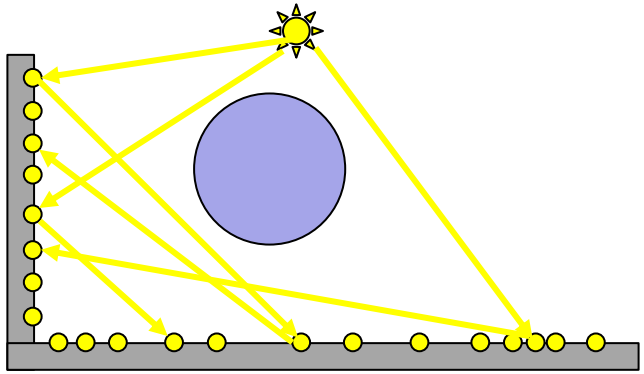
Photon Storing

- Requires quick look up in second stage
- Usually a KD-Tree
- But can use any other structure (e.g. octree)



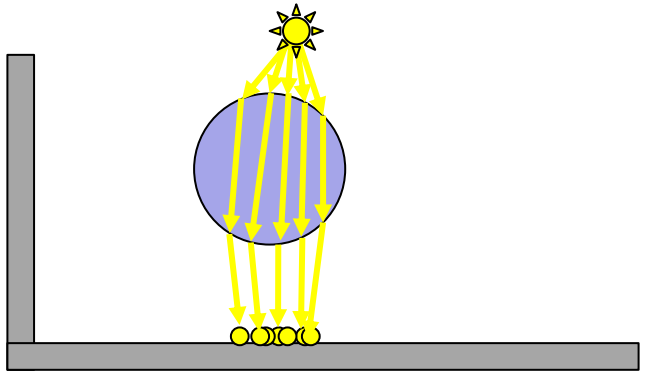
Photon Storing

- 3 Photon Maps
  - Global photon map
    - Any number of reflections before diffuse absorption



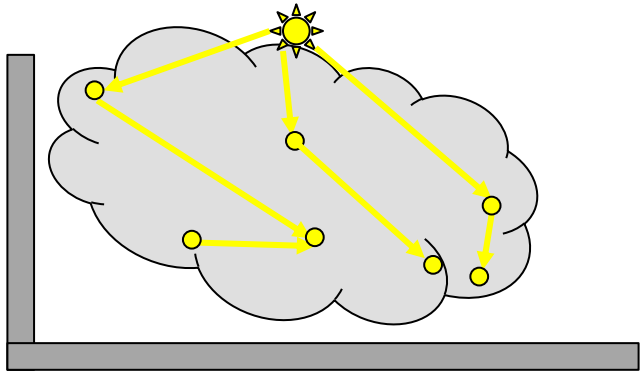
Photon Storing

- 3 Photon Maps
  - Caustic map
    - Any photon that has hit one or more specular surfaces before being absorbed by a diffuse one



Photon Storing

- 3 Photon Maps
  - Volume photon map
    - Any number of reflections before entering volume



Photon Mapping

- Now need to estimate lighting from particles
- Interpolate lighting from nearby particles
- Use Density Estimation

Photon Mapping

- Density Estimation
  - Gather nearby particles and weight by kernel

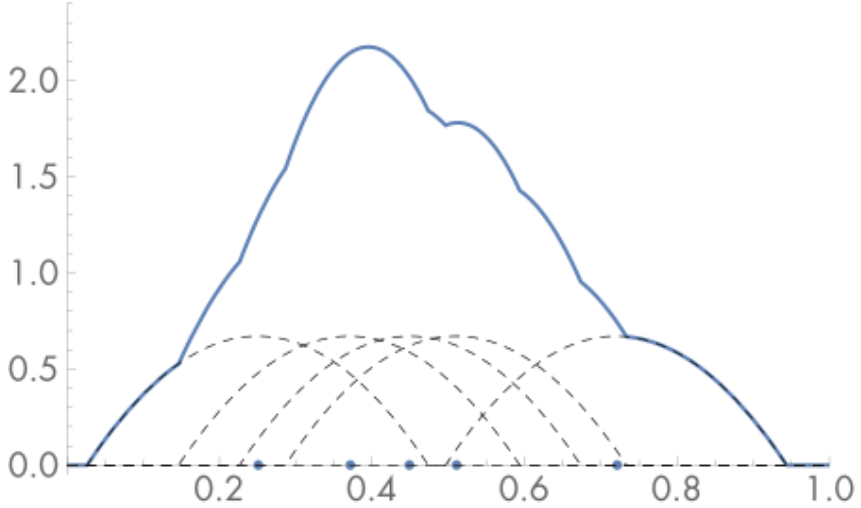
$$k(d)$$

  - Argument is distance to point
  - Must be normalized $\int_{\infty}^{\infty} k(l)dl = 1$
  - Has an associated radius r (width if 1D)

Photon Mapping

- Density Estimation
  - At each point, value is sum of point values weighted by kernel

$$\frac{1}{N_p \cdot r} \sum_{j=1}^{N_p} k\left(\frac{x_j - x}{r}\right)$$



Photon Mapping

- Photon Density Estimation

$$L_o(x, \omega_o) = \frac{1}{N_p \cdot \pi r^2} \sum_{j=1}^{N_p} k\left(\frac{x_j - x}{r}\right) \beta_j f_r(x, \omega_{i,j}, \omega_o)$$

- Blurred biased estimate of lighting
  - From photons on a tangent plane around x
- But computes a smooth image much faster

Photon Mapping

- Photon Density Estimation
  - Most photons will be outside kernel
  - So can choose fixed number of photons (N_{kernel}) to consider and find distance to furthest r_{kernel}

$$L_o(x, \omega_o) = \frac{1}{N_p \cdot \pi r_{kernel}^2} \sum_{j=1}^{N_{kernel}} k\left(\frac{x_j - x}{r_{kernel}}\right) \beta_j f_r(x, \omega_{i,j}, \omega_o)$$

Photon Mapping

- Photon Density Estimation
  - Recall rendering equation (reflectance only)

$$L_o(x, \omega_o) = \int_\Omega f_r(x, \omega_i, \omega_o) L_i(x, \omega_i) \cos\theta_i d\omega_i$$

  - Insert definition of radiance

$$L_o(x, \omega_o) = \int_\Omega f_r(x, \omega_i, \omega_o) \frac{d^2\Phi_i(x, \omega_i)}{dA_i \cos\theta_i \, d\omega_i} \cos\theta_i d\omega_i$$

Photon Mapping

- Photon Density Estimation
  - So $$L_o(x,\omega_o) = \int_\Omega f_r(x,\omega_i,\omega_o)\frac{d^2\Phi_i(x,\omega_i)}{dA_i}$$
  - Estimate $$L_o(x,\omega_o) \approx \frac{1}{N_p}\sum_{j=1}^{N_p} f_r(x,\omega_{i,j},\omega_o)\frac{\Delta\Phi_j(x,\omega_j)}{\Delta A}$$

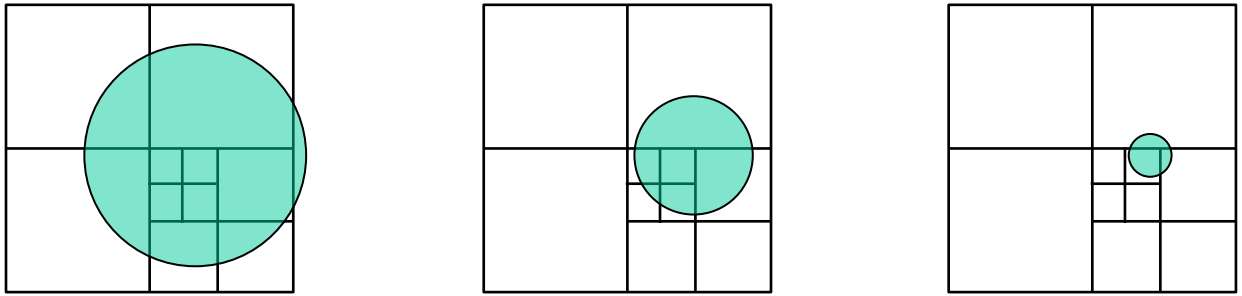
$$L_o(x,\omega_o) = \frac{1}{N_p \cdot \pi r_{kernel}^2}\sum_{j=1}^{N_{kernel}} f_r(x,\omega_{i,j},\omega_o) k\left(\frac{x_j - x}{\pi r_{kernel}^2}\right)\beta_j$$

Photon Mapping

- Photon Density Estimation
  - Lookups from data structure
  - Usually use a priority queue to store N_{kernel} photons
    - Push each photon found
    - Remove furthest photons if more than N_{kernel} in queue

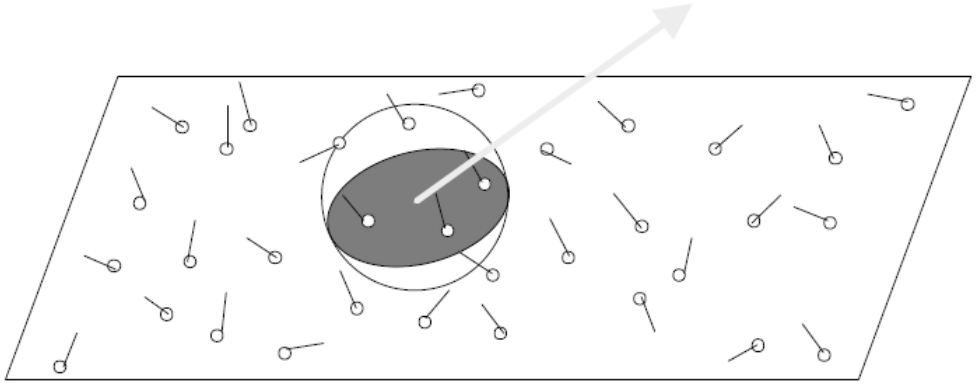
Photon Mapping

- Photon Density Estimation
  - Can adapt data structure search based on the current radius
  - Check distances to split planes



Photon Mapping

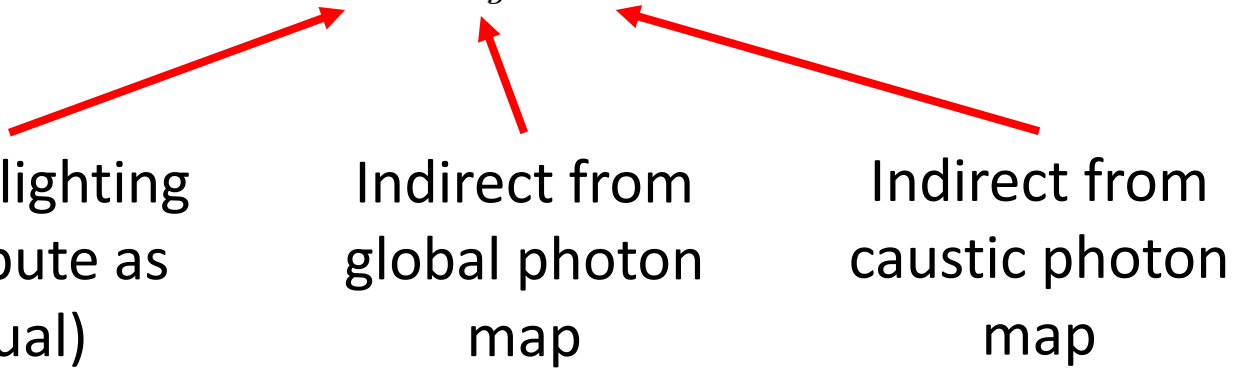
- Photon Density Estimation



Photon Mapping

- Photon Density Estimation
  - Split contributions from different path types

$$L_o = L_d + L_g + L_c$$

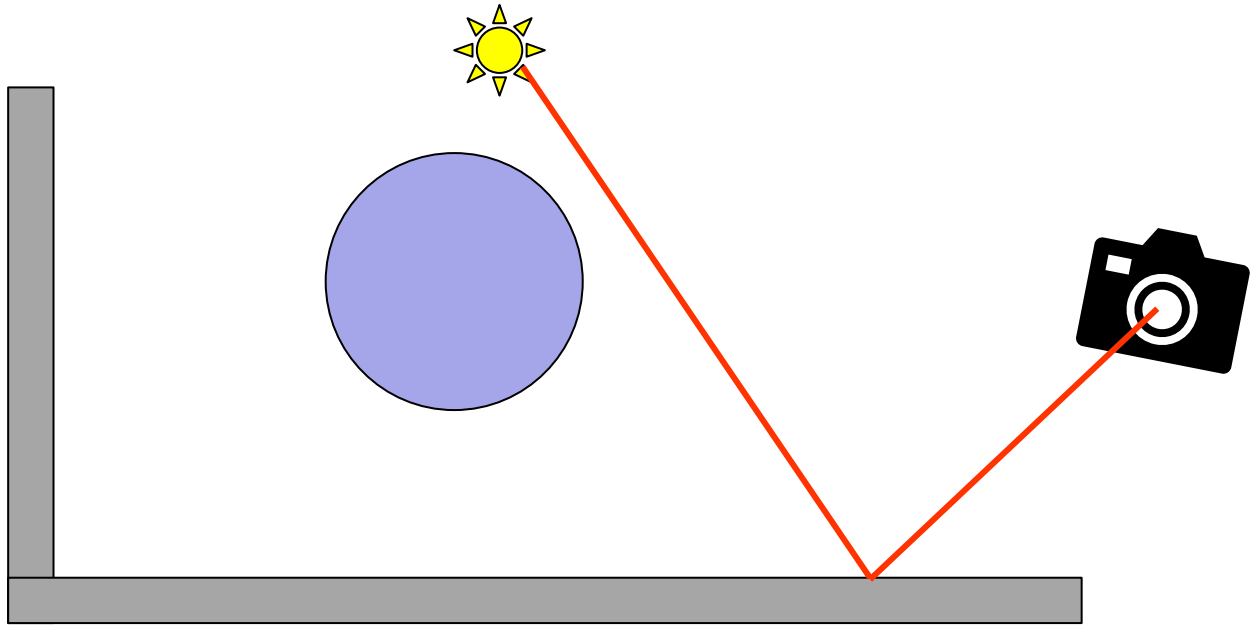


Photon Mapping

- Photon Density Estimation
  - Kernel radius different for different photon maps
  - Global photon map typically larger
  - Caustic smaller to capture details
  - Can be adaptive

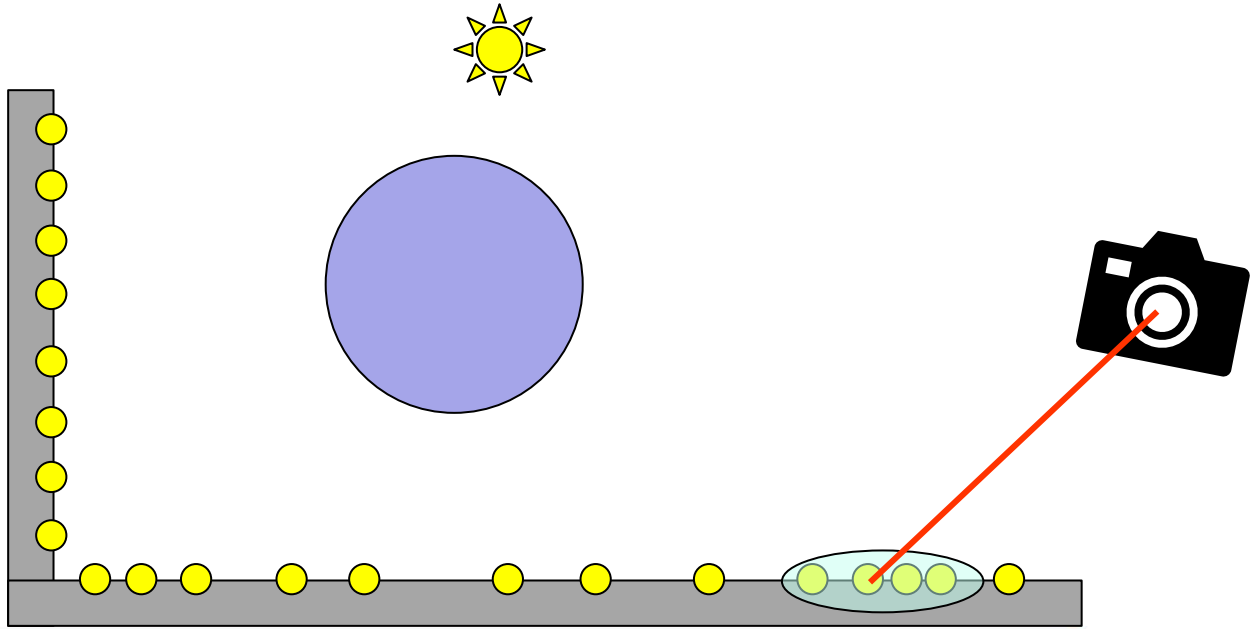
Photon Mapping

- Direct lighting



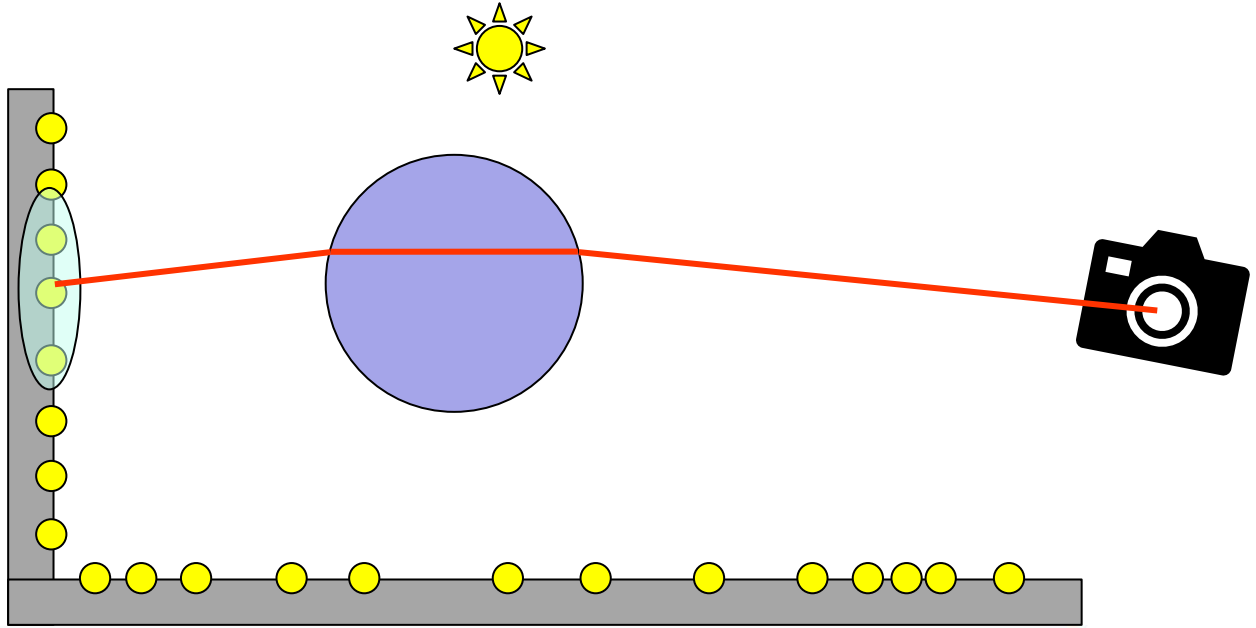
Photon Mapping

- Indirect from global photon map



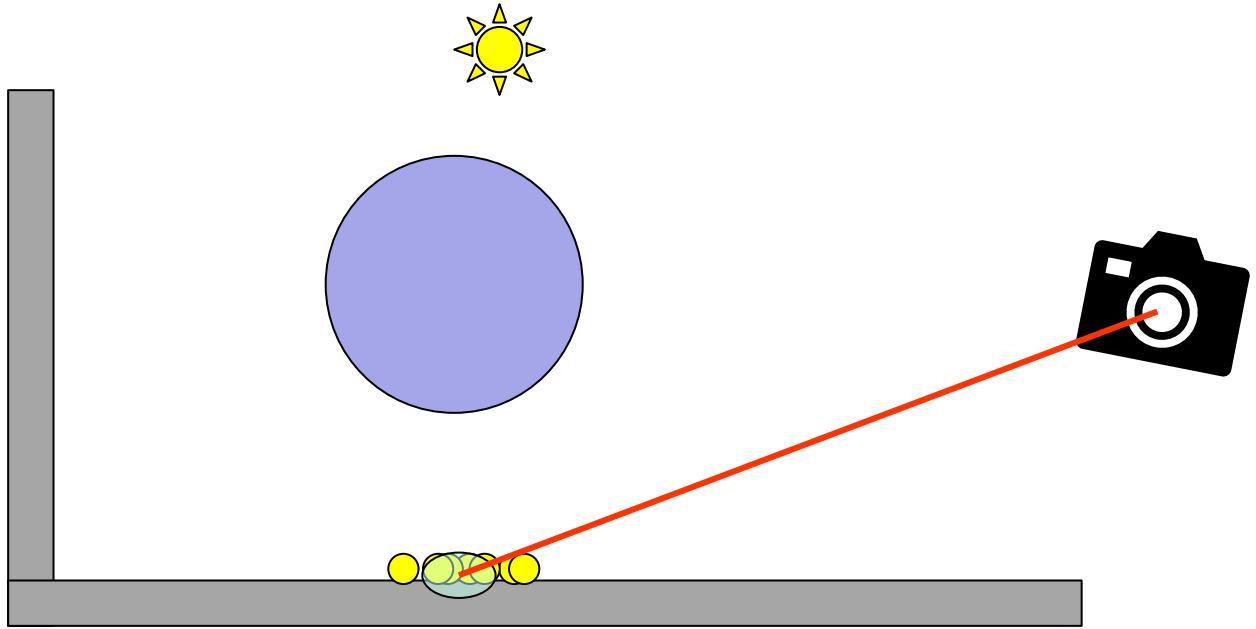
Photon Mapping

- Trace through specular surfaces



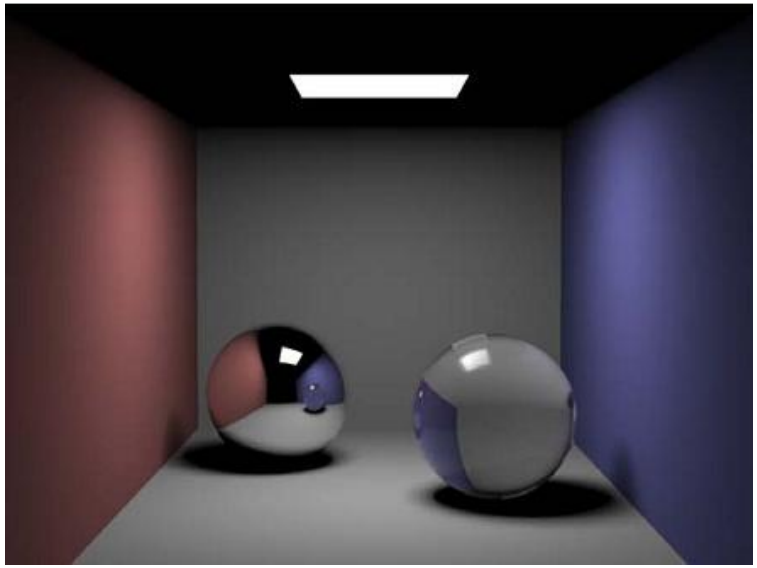
Photon Mapping

- Indirect from caustic photon map



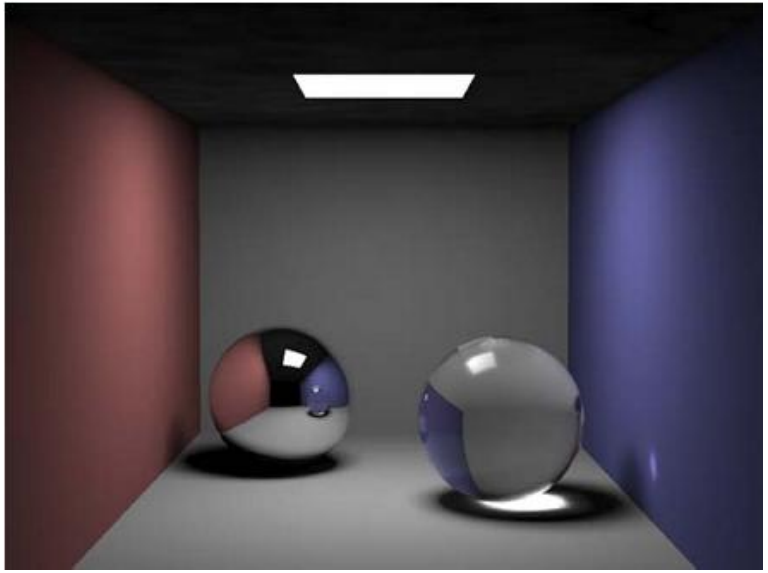
Photon Mapping

- Direct lighting



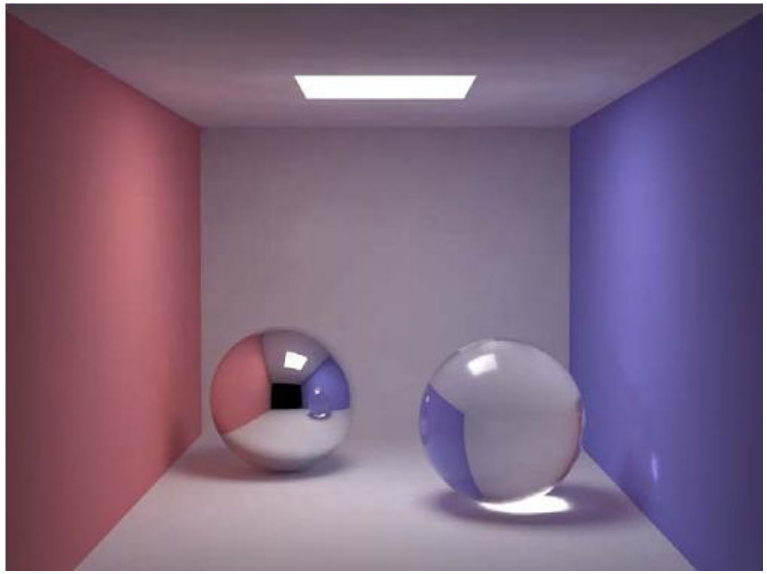
Photon Mapping

- Indirect from caustic photon map
  - 50K Photons
  - 60 in kernel



Photon Mapping

- Indirect from global photon map + caustic
  - 200K Photons
  - 100 in kernel

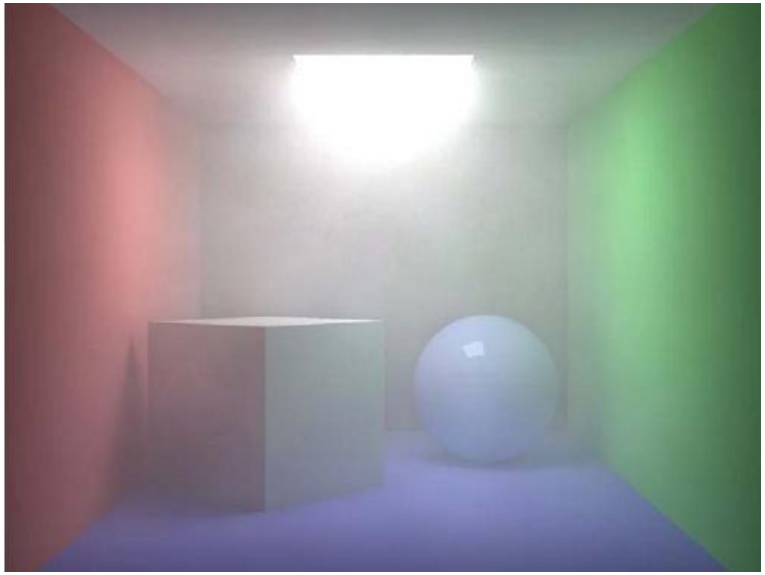


Photon Mapping

- Indirect from global photon map + caustic
  - 500K Photons
  - 100 in kernel

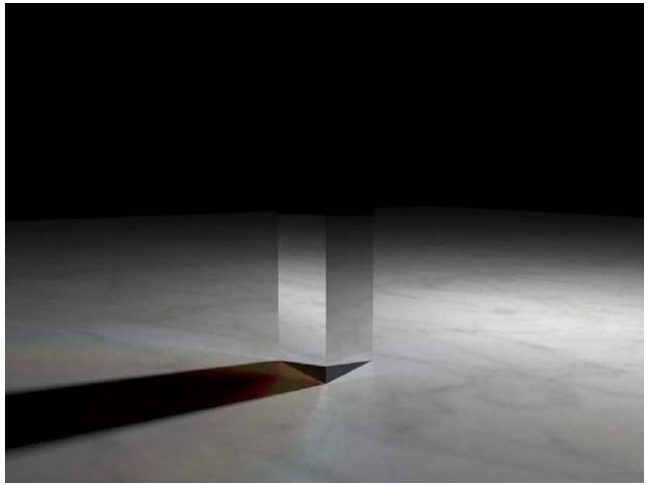
Photon Mapping

- Indirect from volume photon map
  - Global
    - 100K Photons
    - 100 in kernel
  - Volume
    - 150K Photons
    - 100 in kernel



Photon Mapping

- More examples



Photon Mapping

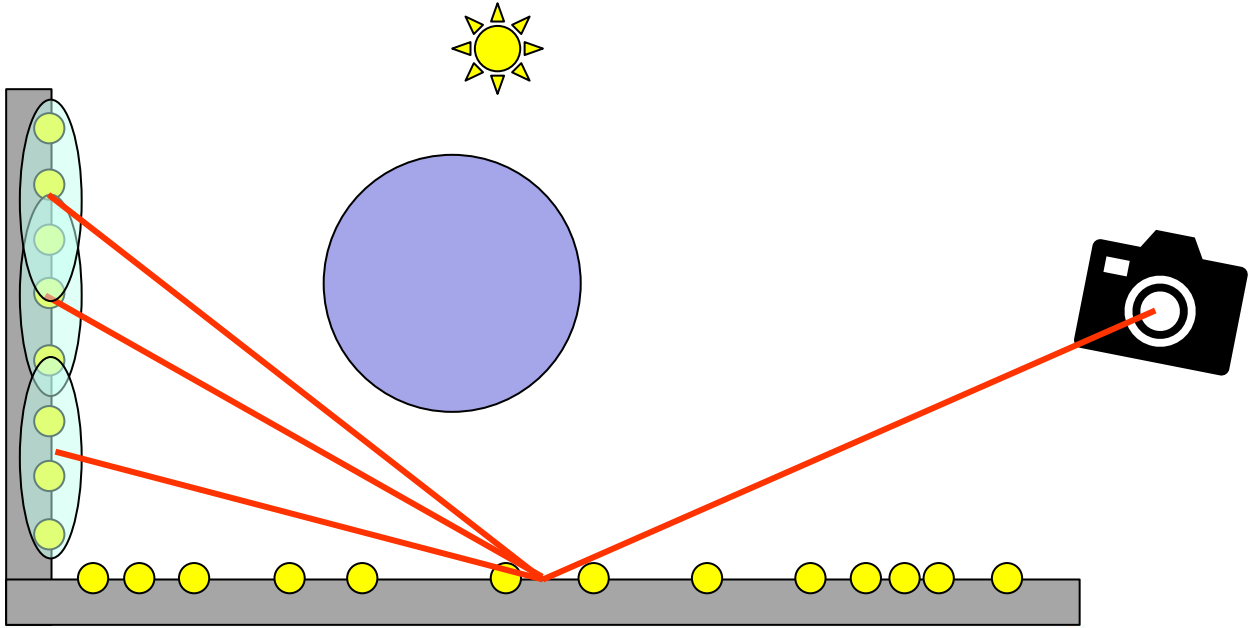
- Problems
  - Biased but consistent estimate as $N_p \to \infty$
    - But cannot store
  - Finite size kernels
  - Leads to blurry images
  - Problems at edges
    - Light leakage (too bright or too dark)

Photon Mapping

- Final gathering
  - Shoot extra rays from first non-specular hit point
  - Can be importance sampled
  - Density estimation at secondary hit points

Photon Mapping

- Indirect from global photon map

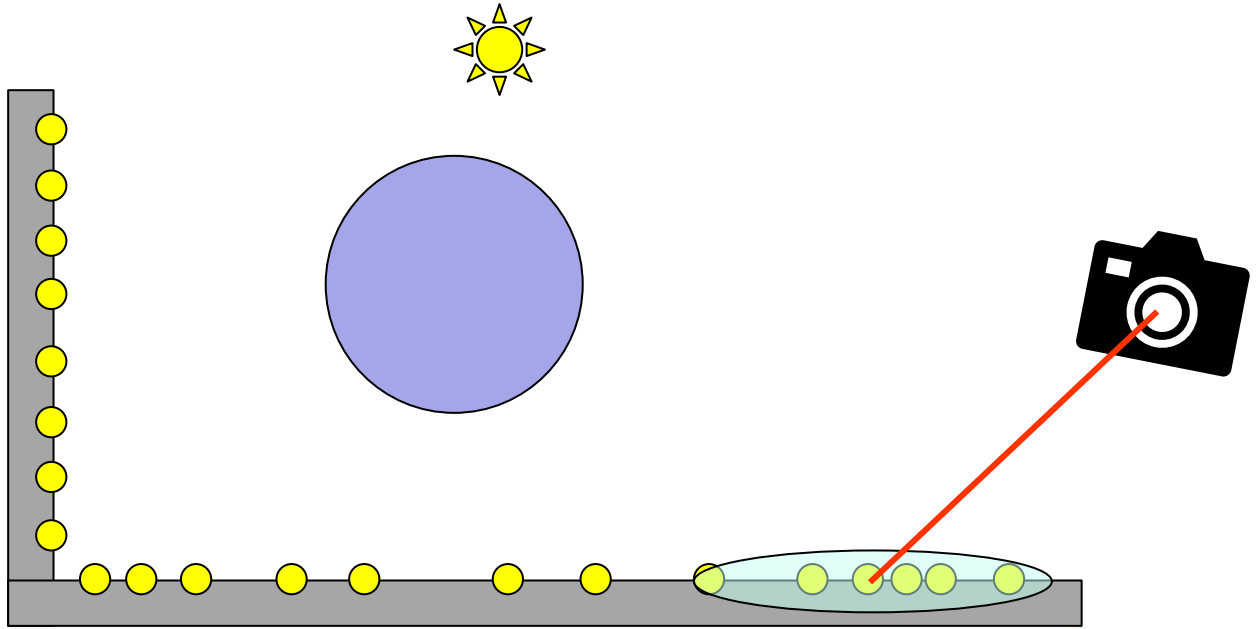


Progressive Photon Mapping

- Solves storage problem
- Pass based algorithm
  - Compute points visible from camera
  - 1. Trace photons and contribute to image
  - Refine parameters
  - Goto 1

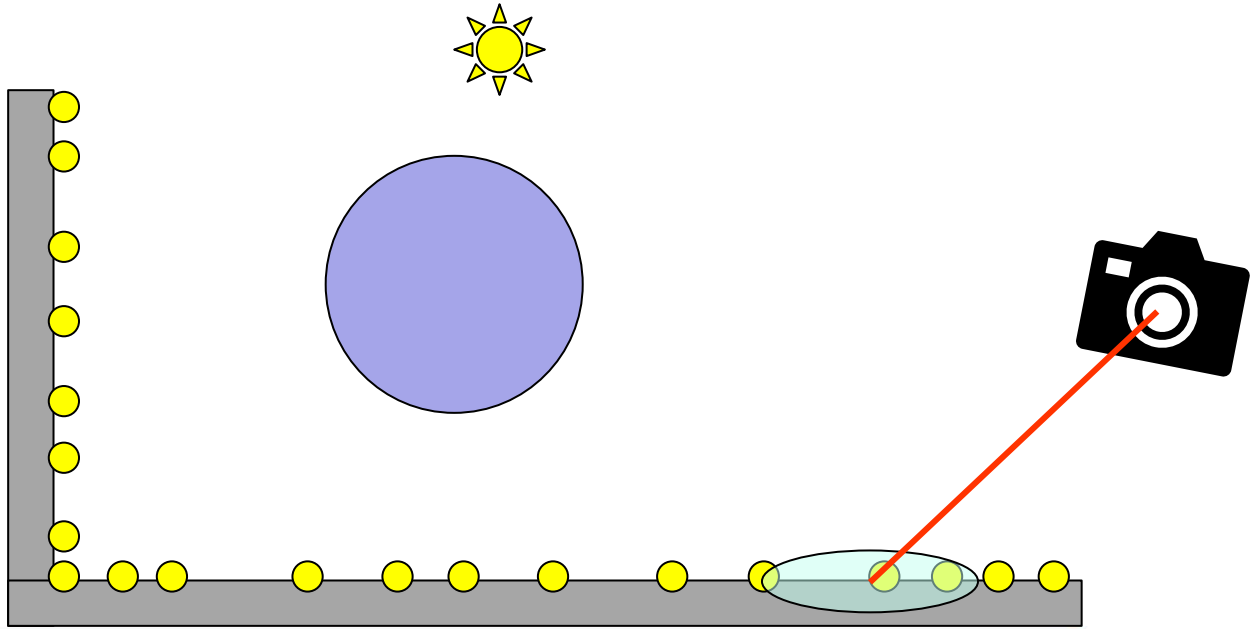
Progressive Photon Mapping

- Multiple passes refine estimate



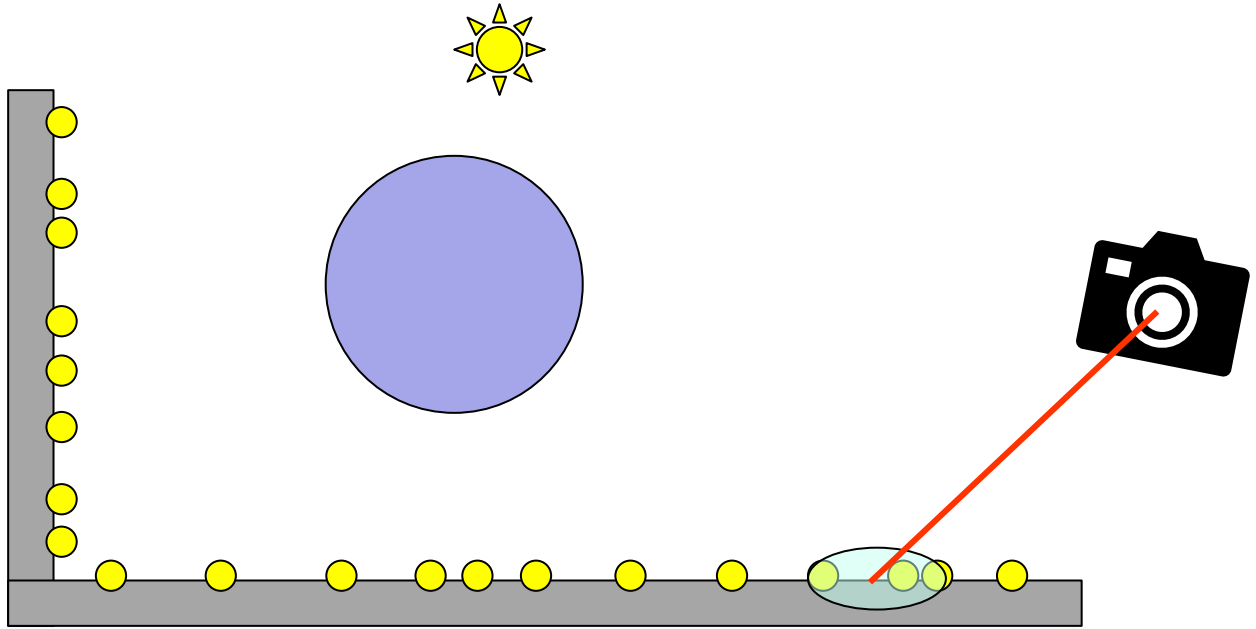
Progressive Photon Mapping

- Multiple passes refine estimate



Progressive Photon Mapping

- Multiple passes refine estimate



Progressive Photon Mapping

- Idea
  - Refine estimate each pass
  - Partially discard previous contribution each iteration
  - Need to store statistics for each pixel independently

Progressive Photon Mapping

- Per pixel estimate
  - Need tuple of quantities

$$(x_{pixel}, r_{pixel}, N_{pixel}, \tau_{pixel})$$

World position of first non-specular point in pixel

Kernel radius

Number of photons contributing

Current estimate

Progressive Photon Mapping

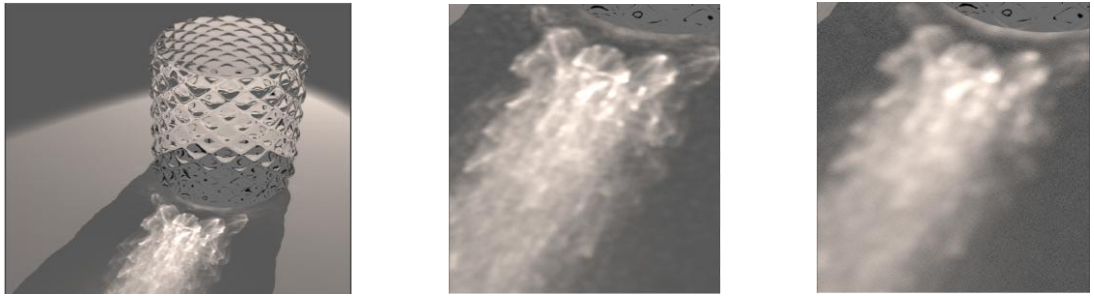
- Per pixel estimate
  - Initialize quantities from a pre-pass
  - Trace from camera until non-specular surface
    - Store initial values in screen sized buffer

$$(x_{pixel}, r_0, N_0 = 0, \tau_0 = (0,0,0))$$

    - r_{init} can be a percentage of scene size

Progressive Photon Mapping

- For each pass
  - Trace M photons
    - Same algorithm as before
    - Balance between image quality and render time



Progressive Photon Mapping

- Per pixel estimate
  - Update quantities each pass

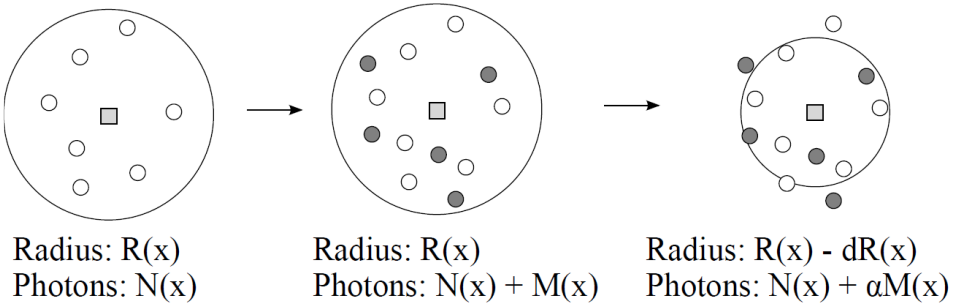
$$N_{i+1} = N_i + \alpha M_i \qquad \text{where } \alpha \in [0,1] \text{, usually } \alpha \approx \frac{2}{3}$$

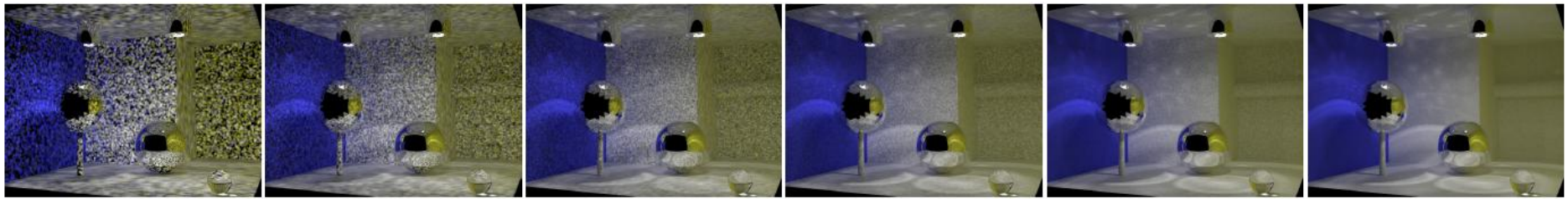
$$r_{i+1} = r_i \sqrt{\frac{N_{i+1}}{N_i + M_i}}$$

$$\tau_{i+1} = (\tau_i + \Phi_i)\frac{r_{i+1}^2}{r_i^2} \qquad \text{where} \quad \Phi_i = \sum_{j=1}^{M_i} \beta_j f_r(x, \omega_{i,j}, \omega_o)$$

Progressive Photon Mapping

- Per pixel estimate
  - Converges as $N_i \to \infty$ and $r_i \to 0$





Progressive Photon Mapping

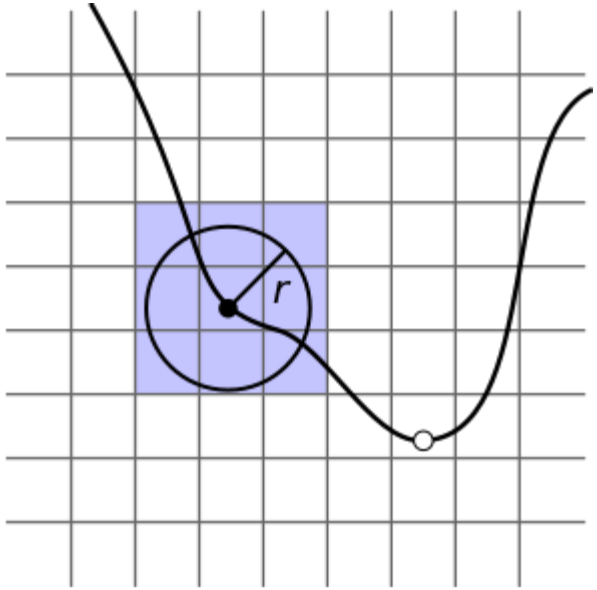
- Per pixel estimate
  - α is the rate previous contributions are discarded
  - How to compute $\Phi_i = \sum_{j=1}^{M_i} \beta_j f_r(x, \omega_{i,j}, \omega_o)$
    - Could build photon map per pass (e.g. in a kd-tree / octree, spatial hash)
    - Extra computation

Progressive Photon Mapping

- Per pixel estimate
  - How to compute $\Phi_i = \sum_{j=1}^{M_i} \beta_j f_r(x, \omega_{i,j}, \omega_o)$
  - Can build data structure over pixel statistics data (i.e. x_{pixel} from $(x_{pixel}, r_{pixel}, N_{pixel}, \tau_{pixel})$)
  - Much faster but needs to store more data

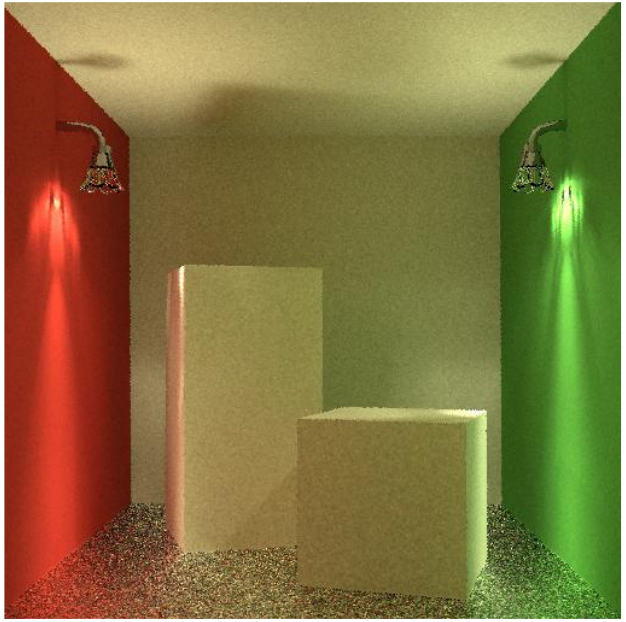
Progressive Photon Mapping

- Per pixel estimate
  - E.g. Uniform grid or spatial hash
  - Reference to pixel added to each cell where the radius overlaps



Progressive Photon Mapping

- Limitations
  - Specular surfaces
  - Inefficient for glossy surfaces

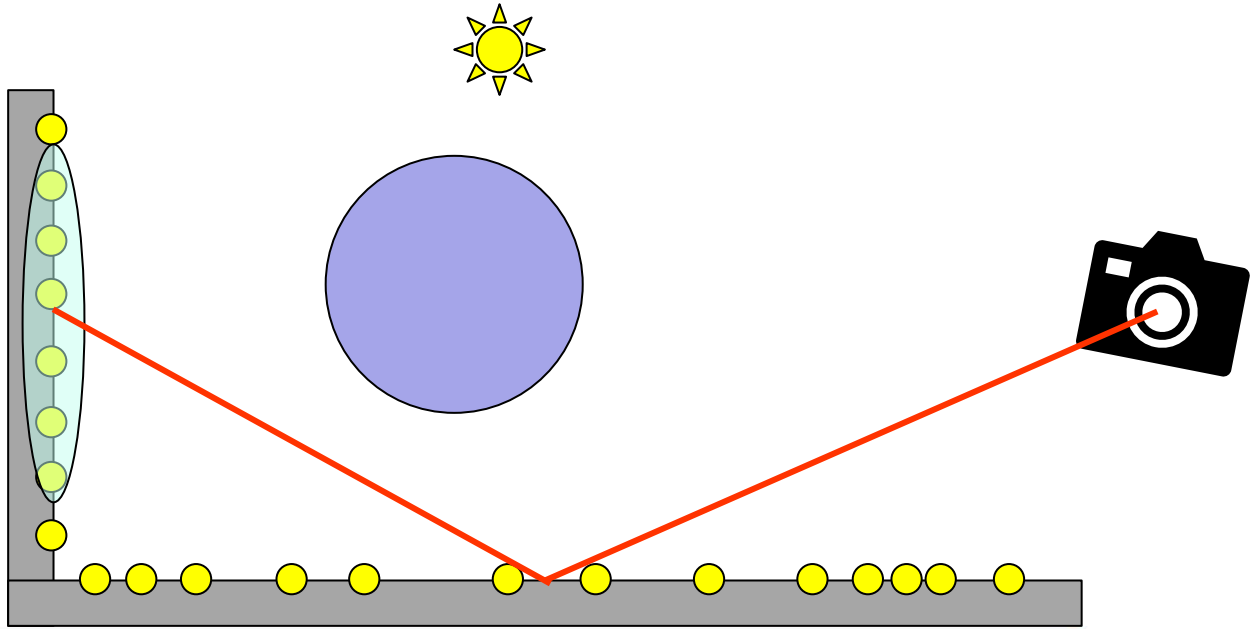


SPPM

- Stochastic Progressive Photon Mapping
- Can extend paths by one bounce similar to final gathering
  - Can still use shared pixel statistics
- Improved aliasing and glossy image quality

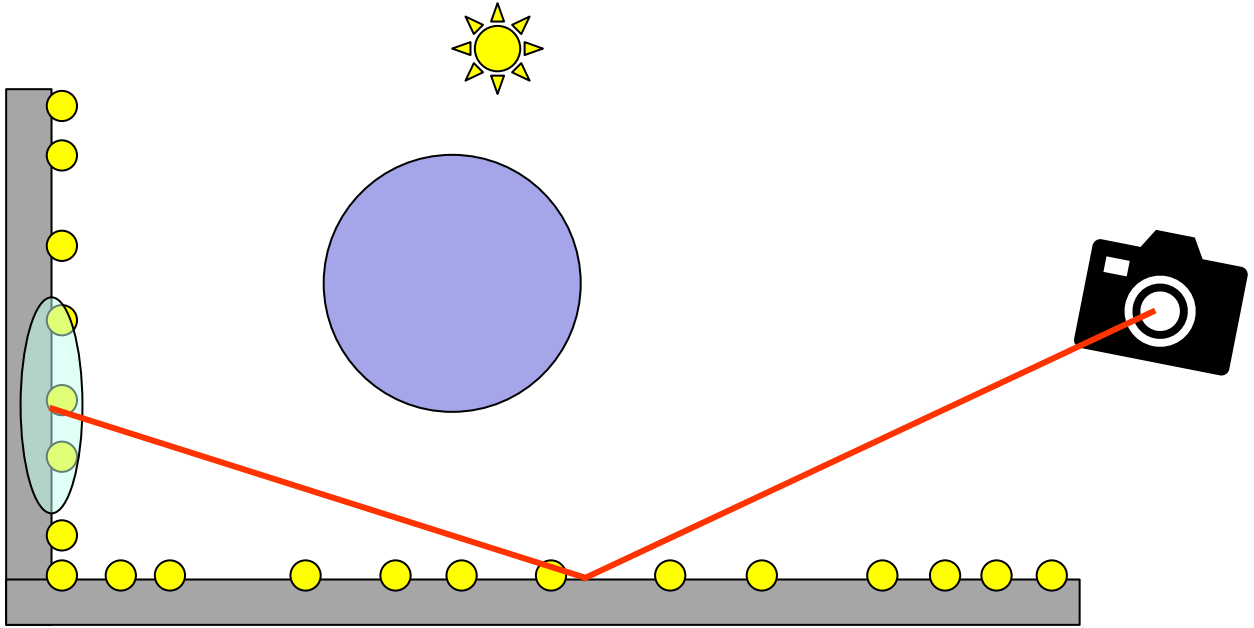
SPPM

- Update photon map and camera sub path



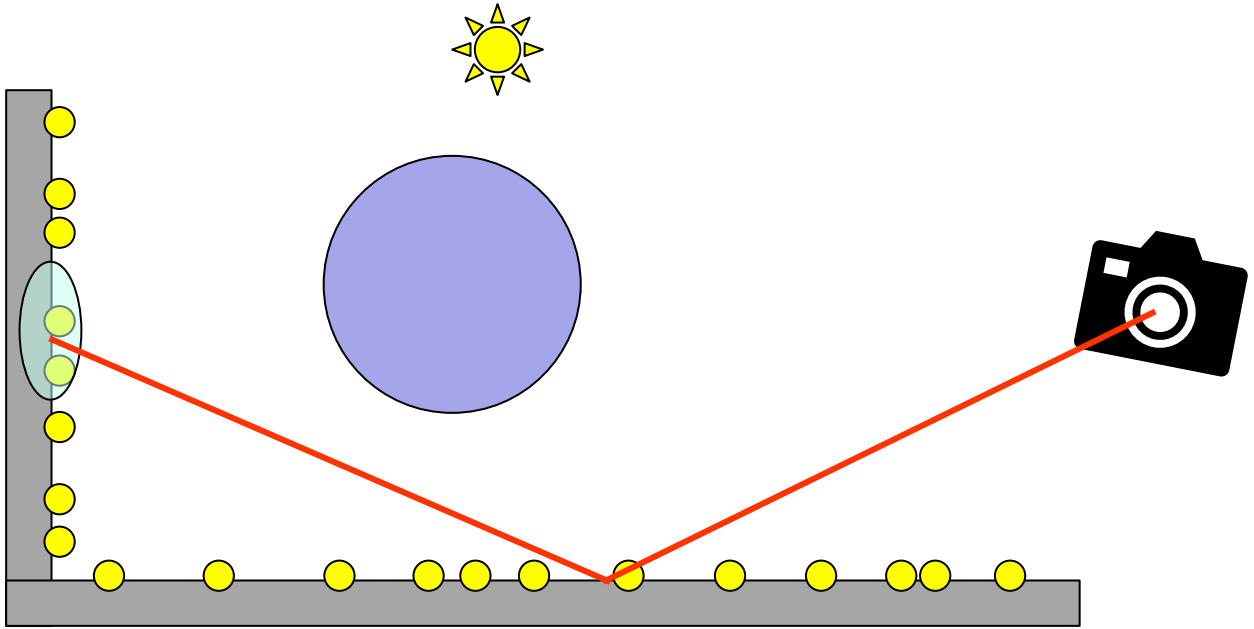
SPPM

- Update photon map and camera sub path



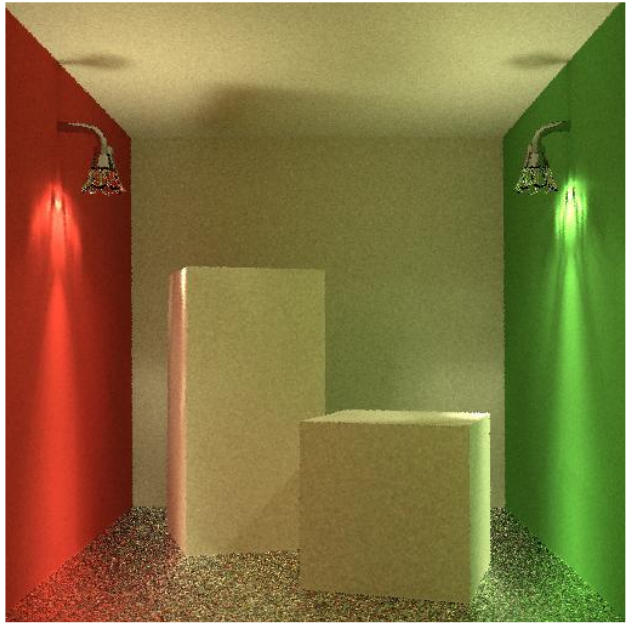
SPPM

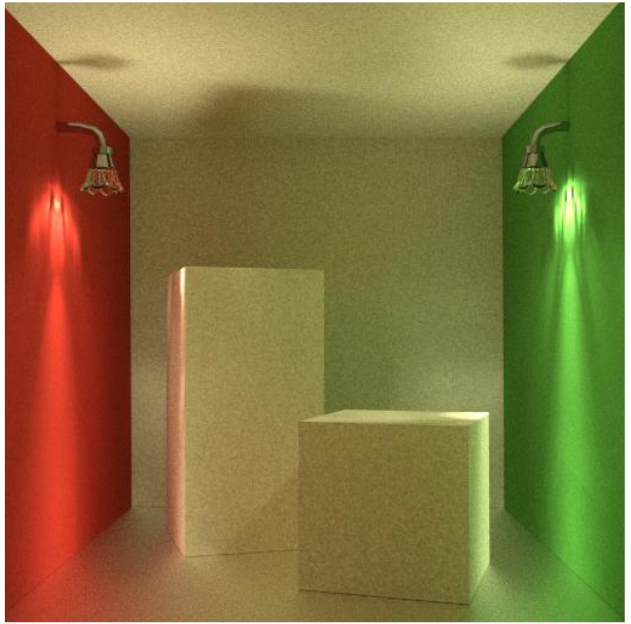
- Update photon map and camera sub path



SPPM

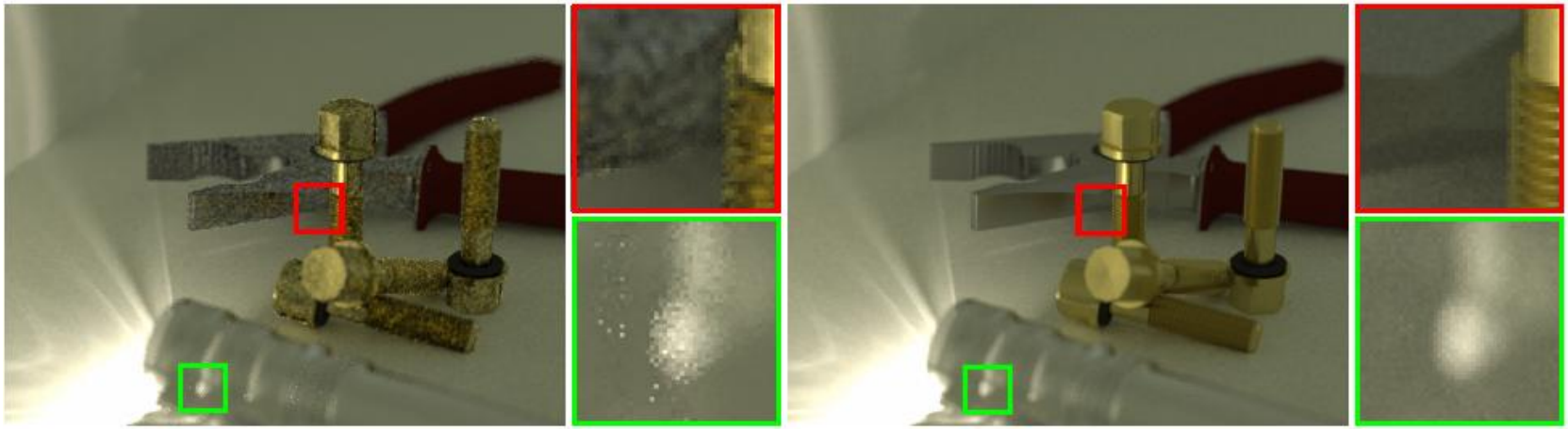
- Improved image quality compared to PPM





SPPM

- Improved image quality compared to PPM



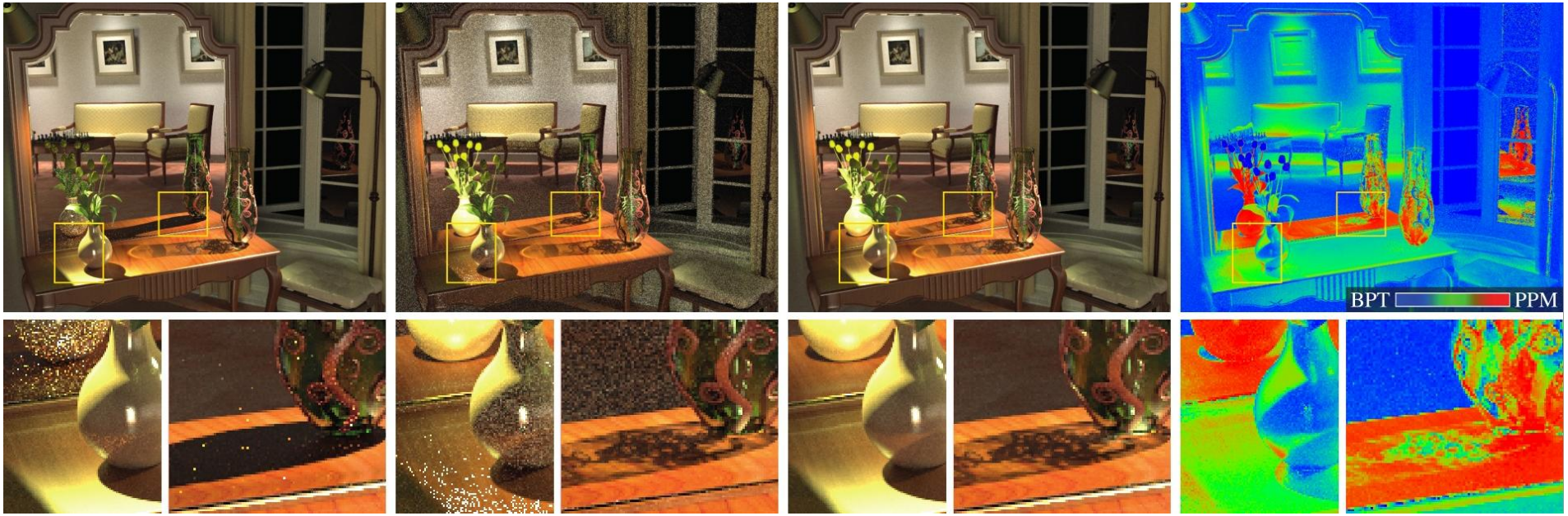
Vertex Connection and Merging

- Combination of Bidirectional Path Tracing and Progressive Photon Mapping
- Trace many paths per pass
  - Camera paths
  - Light paths used for BPT and Density estimation

Vertex Connection and Merging

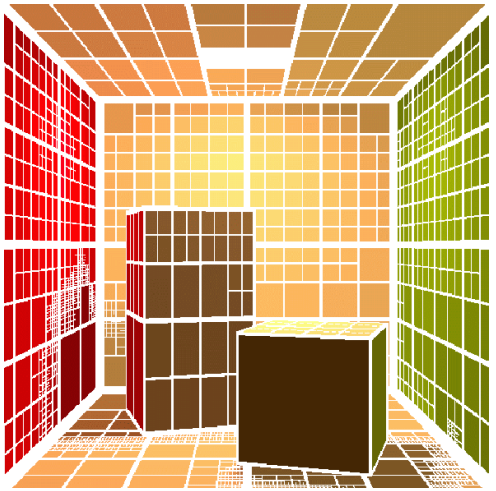
- Vertex Connection
  - Connects paths like BPT
- Vertex Merging
  - Uses kernel density estimation
- Combination
  - MIS between technqiues

Vertex Connection and Merging



Radiosity

- First approach for Global Illumination
- Finite Element Approach to lighting
  - Discretizes the scene into patches
  - Calculate energy leaving each patch
    - Radiosity
  - View independent



Radiosity

- Radiant Exitance
  - Integrate outgoing radiance over the hemisphere

$$b(x) = \int_{\Omega} L_o(x, \omega_o) \cos(\theta) d\omega_o$$

  - Assume diffuse surfaces so constant outgoing radiance $L_o(x, \omega_o) = L(x)$:

$$b(x) = L_o(x) \int_{\Omega} \cos(\theta) d\omega_o = \pi L_o(x)$$

Radiosity

- Assume constant radiosity over patch i

$$b_i = b(x)$$

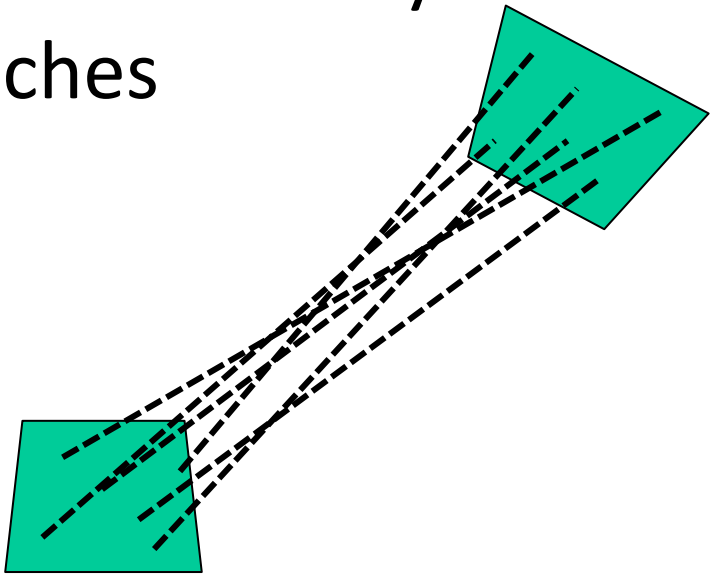
- How to compute
  - Sum incoming radiosity from other patches
  - Included emitted radiosity

$$b_i = e_i + \rho_i \sum_{j=1}^{N} F_{ij} b_j$$

Radiosity

- Need to calculate how much radiosity is transported between patches
- Form Factor

$$F_{ij} = \frac{1}{A_i} \int_{A_i} \int_{A_j} G(x_i \leftrightarrow x_j) dx_j dx_i$$



Radiosity

- Form Factor
  - Estimate from e.g. Monte Carlo
- Write in matrix form $b = e + \mathbf{RF}b$
  - b vector of reflected radiance
  - e vector of emitted radiance

Radiosity

- Form Factor matrix

$$\mathbf{F} = \begin{bmatrix} 0 & F_{12} & \cdots & F_{1N} \\ F_{21} & 0 & \cdots & F_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ F_{N1} & F_{N2} & \cdots & 0 \end{bmatrix}$$

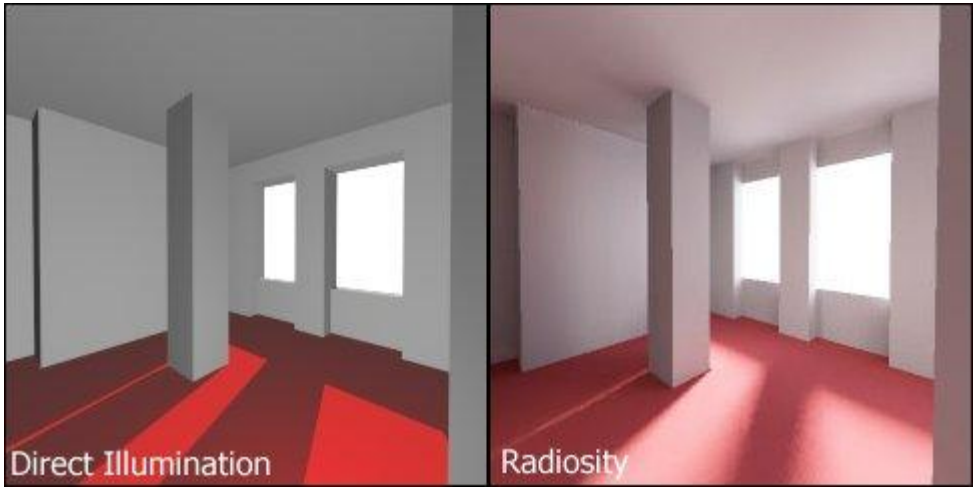
- Reflectance matrix

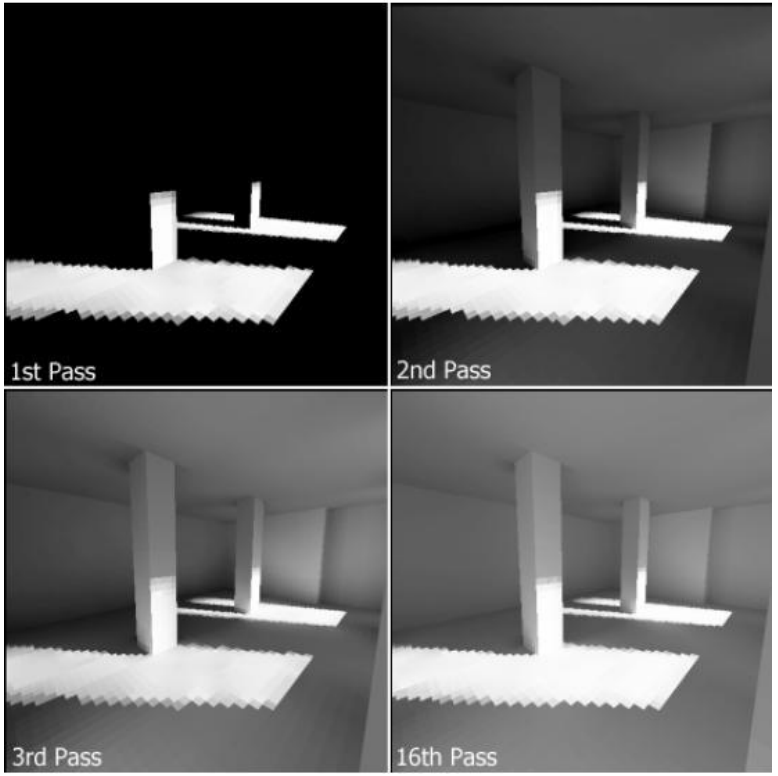
$$\mathbf{R} = \begin{bmatrix} \rho_1 & 0 & \cdots & 0 \\ 0 & \rho_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \rho_N \end{bmatrix}$$

Radiosity

- Solve
  - Invert $b = (\mathbf{I} - \mathbf{R}\,\mathbf{F})^{-1} e$
    - Not feasible when large
  - Gauss-Seidel, Jacobi or Progressive Refinement
    - Initialize $b^{(0)} = e$
    - Refine until converged $b^{(k+1)} = e + \mathbf{R}\,\mathbf{F}\,b^{(k)}$
    - Infinite series $b = e + \mathbf{R}\,\mathbf{F}\,e + (\mathbf{R}\,\mathbf{F})^2 e + (\mathbf{R}\,\mathbf{F})^3 e + \cdots$

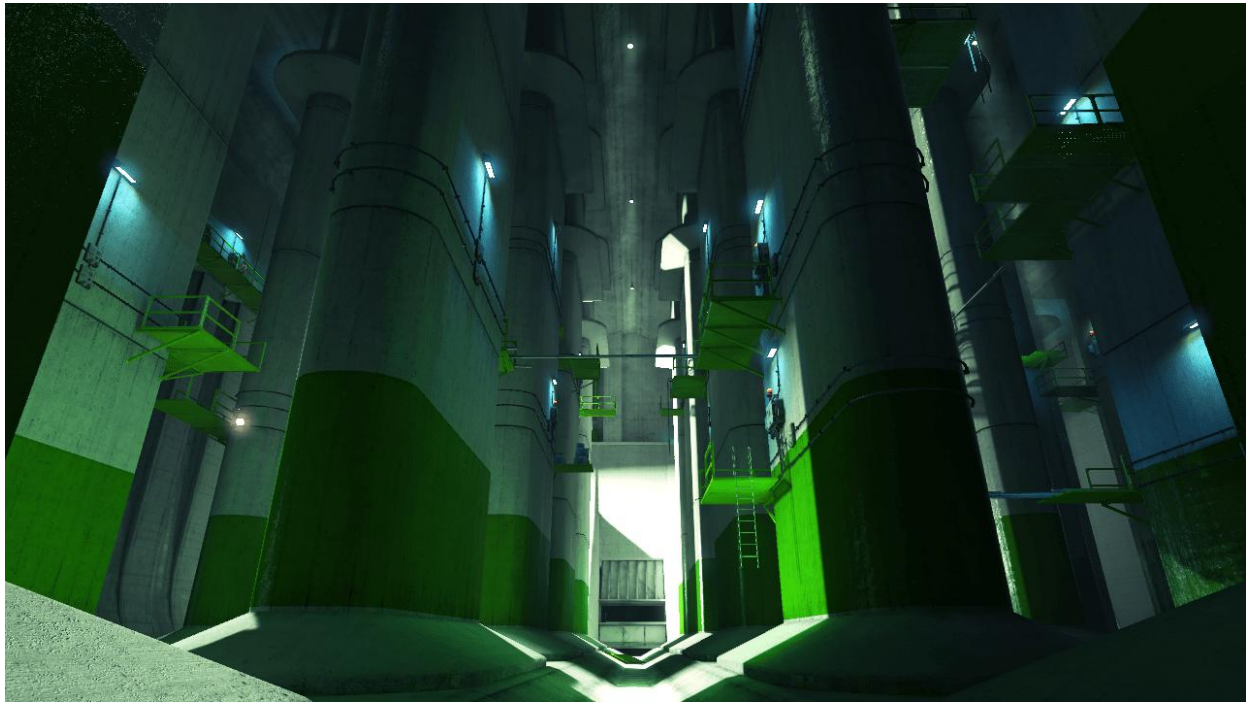
Radiosity


Direct Illumination
Radiosity


1st Pass
2nd Pass
3rd Pass
16th Pass

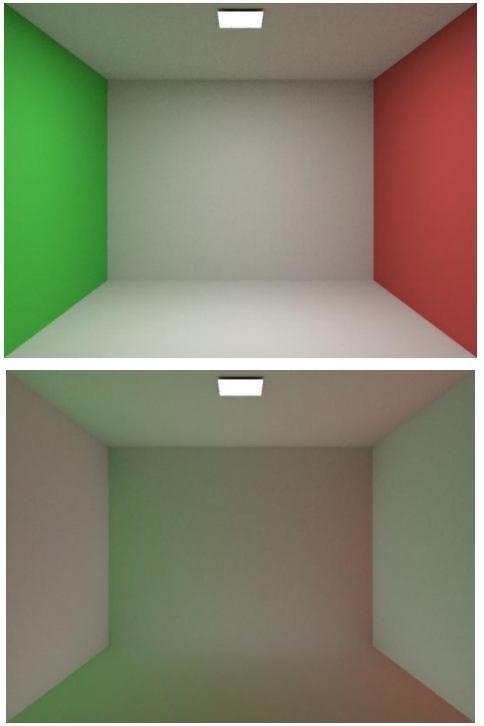
Radiosity

- Lightmaps



Irradiance Caching

- Indirect lighting varies slowly across surfaces
  - Can speed up computations by storing indirect lighting and interpolating



Irradiance Caching

- Start with rendering equation

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_\Omega f_r(x, \omega_i, \omega_o) L_i(x, \omega_i)(\omega_i \cdot n) d\omega_i$$

- Assume diffuse surfaces and no emitted light $f_r(x, \omega_i, \omega_o) = \frac{\rho(x)}{\pi}$

$$L_o(x, \omega_o) = \frac{\rho(x)}{\pi} \int_\Omega L_i(x, \omega_i)(\omega_i \cdot n) d\omega_i = \frac{\rho(x)}{\pi} E(x)$$

  - $E(x)$ is irradiance

Irradiance Caching

- Assume irradiance estimates at positions in world space $\{x_1, x_2, ..., x_N\}$
- Can interpolate irradiance rather than compute

$$L_o(x, \omega_o) \approx \frac{\rho(x)}{\pi} \sum_{i=1}^{S} \frac{w(x, x_i)}{\sum_{j=1}^{N} w(x, x_j)} E(x_i)$$

  - Where $w(x, w_i)$ is an interpolation weight from S nearby cache points

Irradiance Caching

- How to compute $E(x)$?
- How to select $\{x_1, x_2, ..., x_N\}$?
- What should be used for $w(x, x_i)$?
- How should S be selected?
- How should $E(x)$ be stored for efficient access?

Irradiance Caching

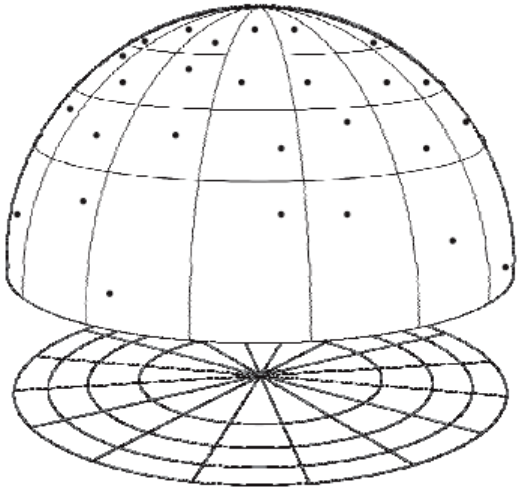
- How to compute $E(x)$?
  - Monte Carlo

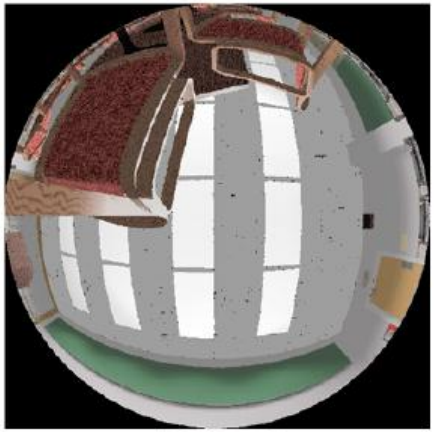
$$E(x) \approx \frac{1}{N} \sum_{i=1}^{N} \frac{L_i(x, \omega_i) \cos(\theta)}{p(\omega_i)}$$

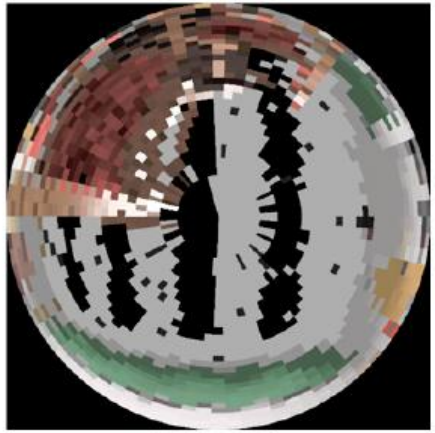
  - Path trace from x into the scene
  - Multiple directions over the hemisphere

Irradiance Caching

- How to compute $E(x)$?
  - Low discrepancy helps







Irradiance Caching

- Radius of influence
  - Needed later
  - Harmonic mean of distances to nearest scene surfaces

$$R_i = \frac{N}{\sum_{i=1}^{N} r(x, \omega_i)}$$

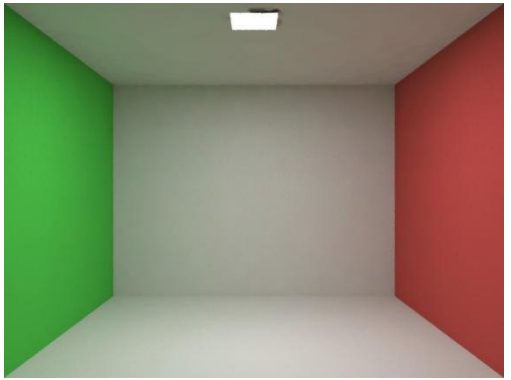
  - Where $r(x, \omega_i)$ is the distance to the first intersection
  - Can be computed during irradiance estimation

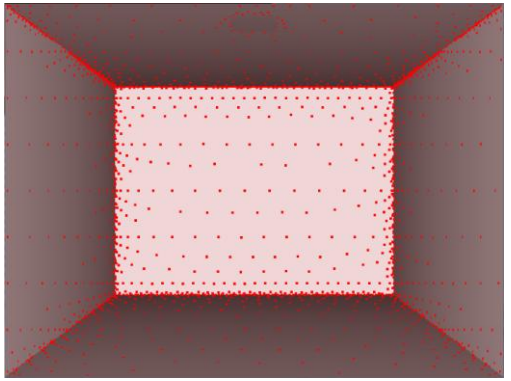
Irradiance Caching

- What to store
  - Irradiance $E(x_i)$
  - Position x_i
  - Normal n_x
  - Radius of influence R_i

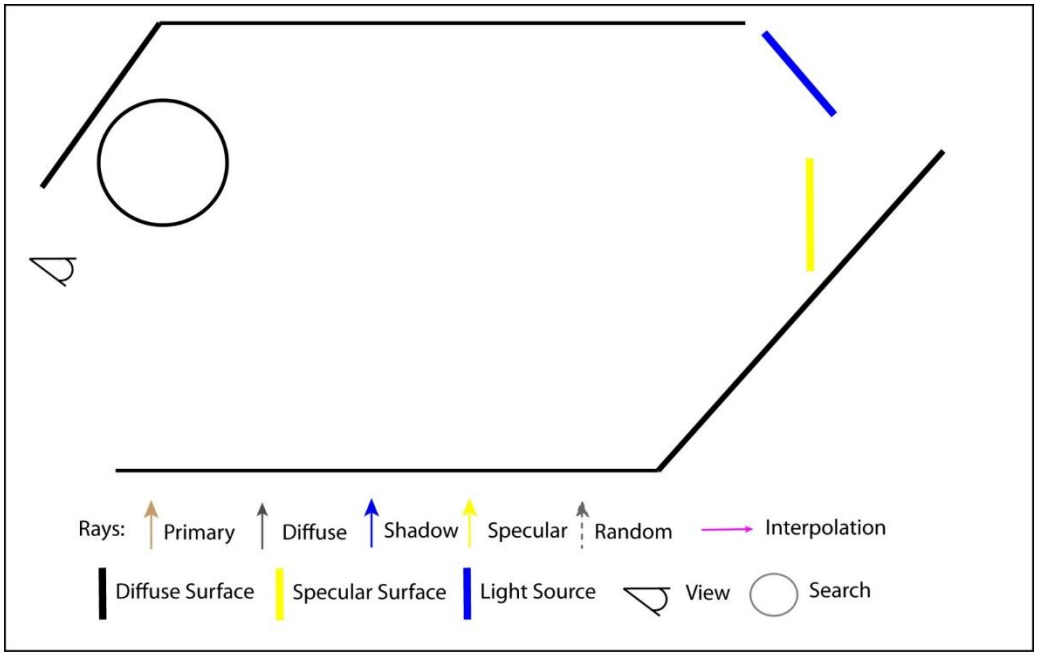
Irradiance Caching

- How to select $\{x_1, x_2, ..., x_N\}$?
  - Want more cache points where irradiance changes quickly
  - Fewer in smooth regions

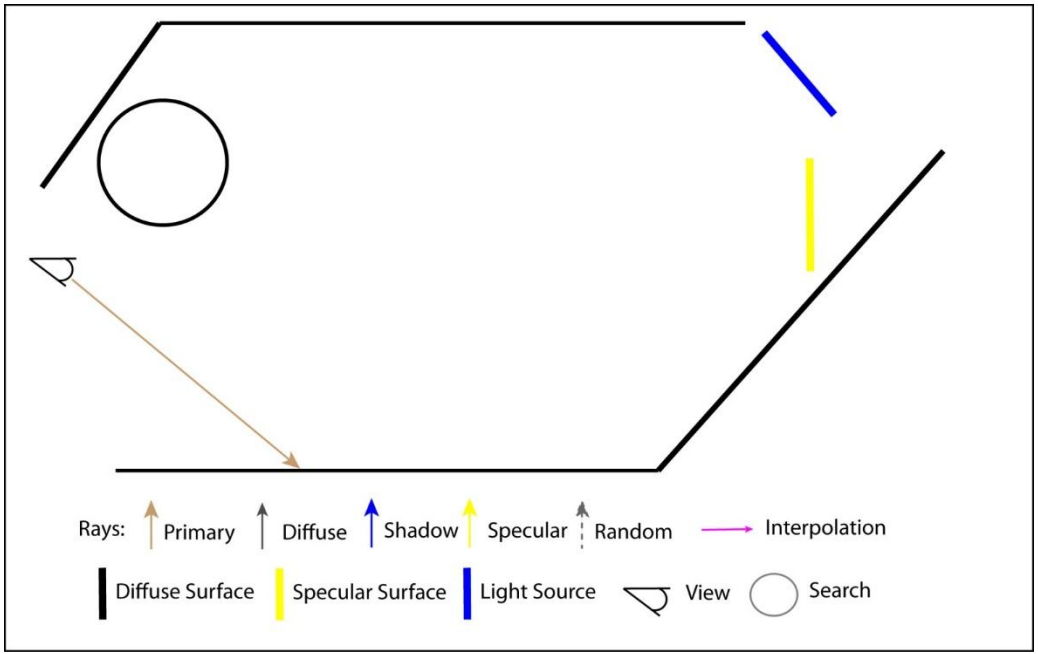




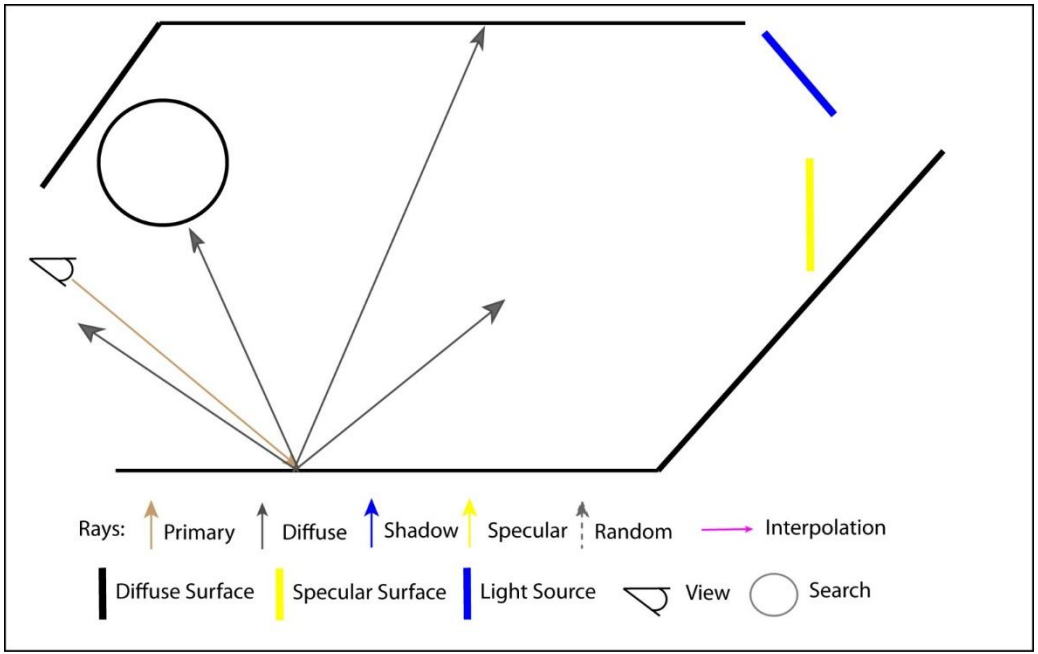
Irradiance Caching



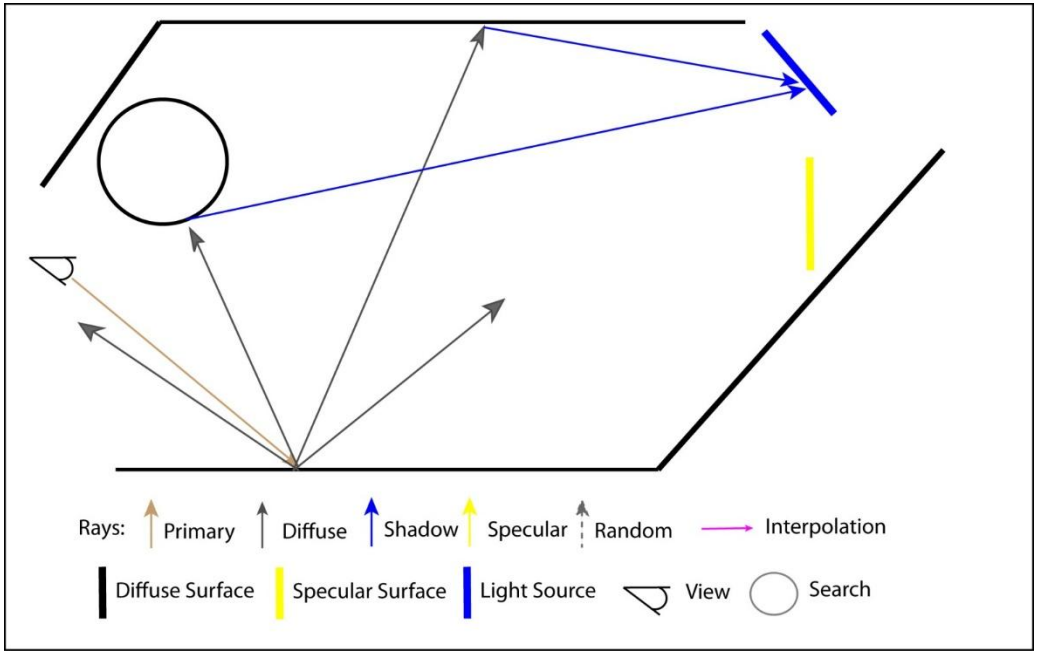
Irradiance Caching



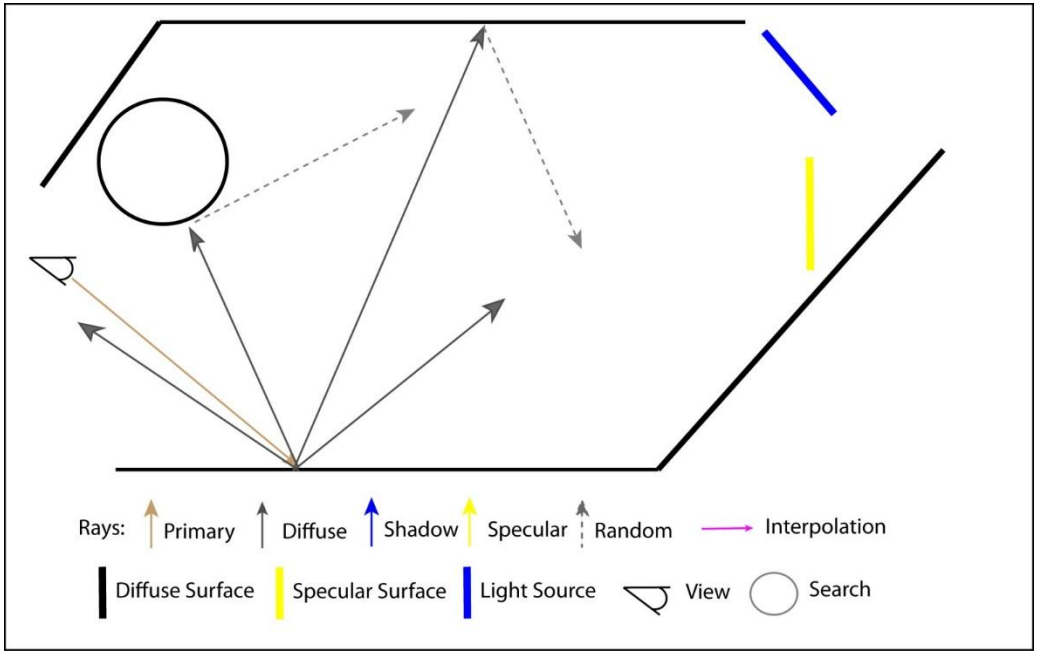
Irradiance Caching



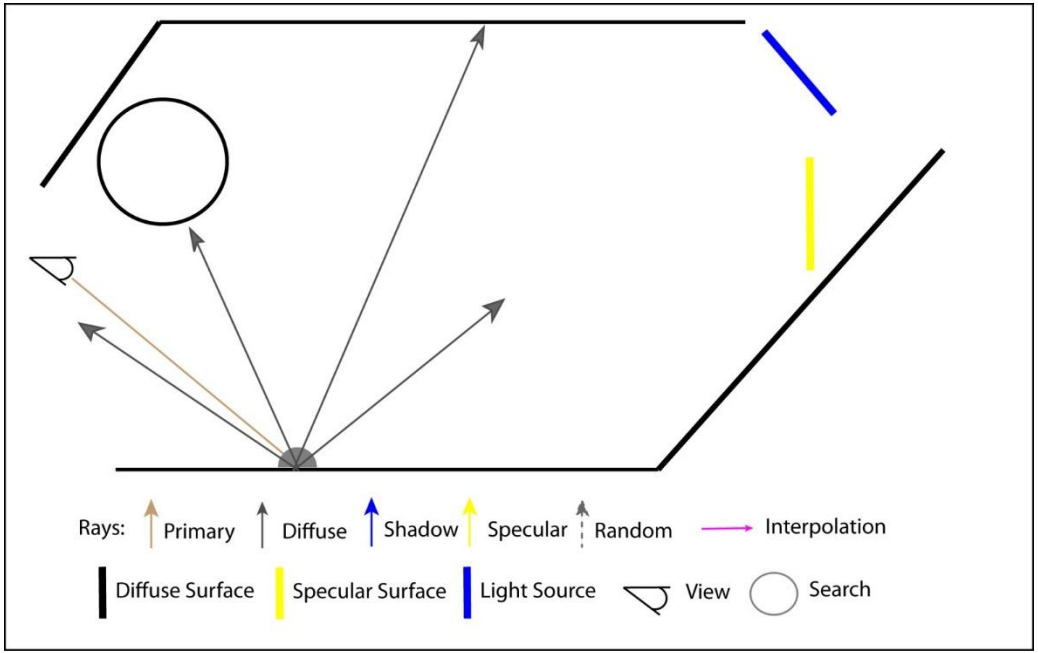
Irradiance Caching



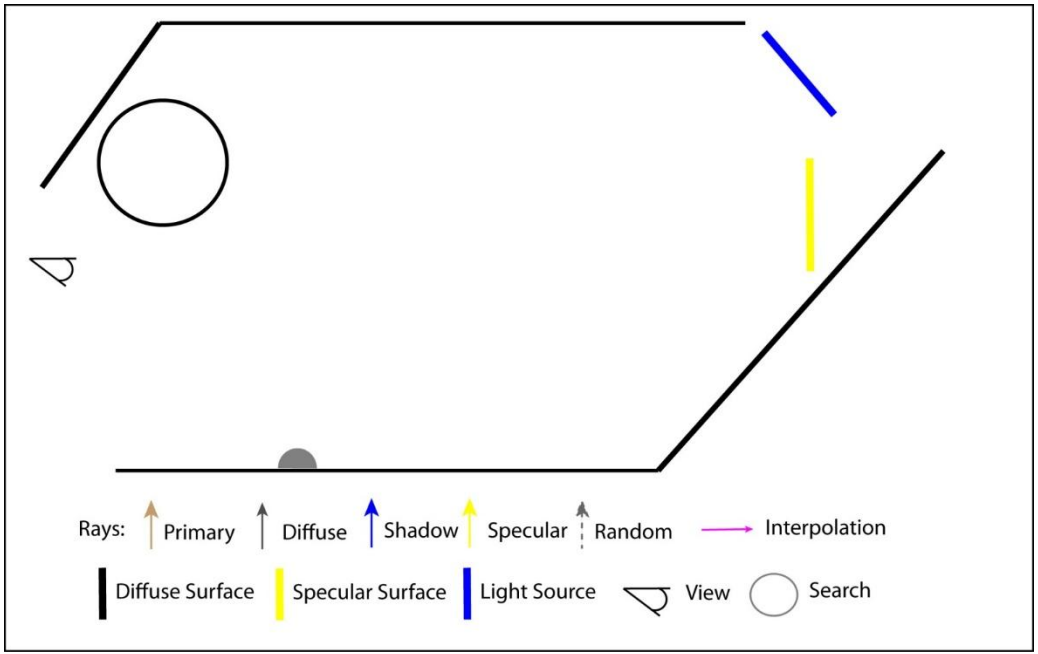
Irradiance Caching



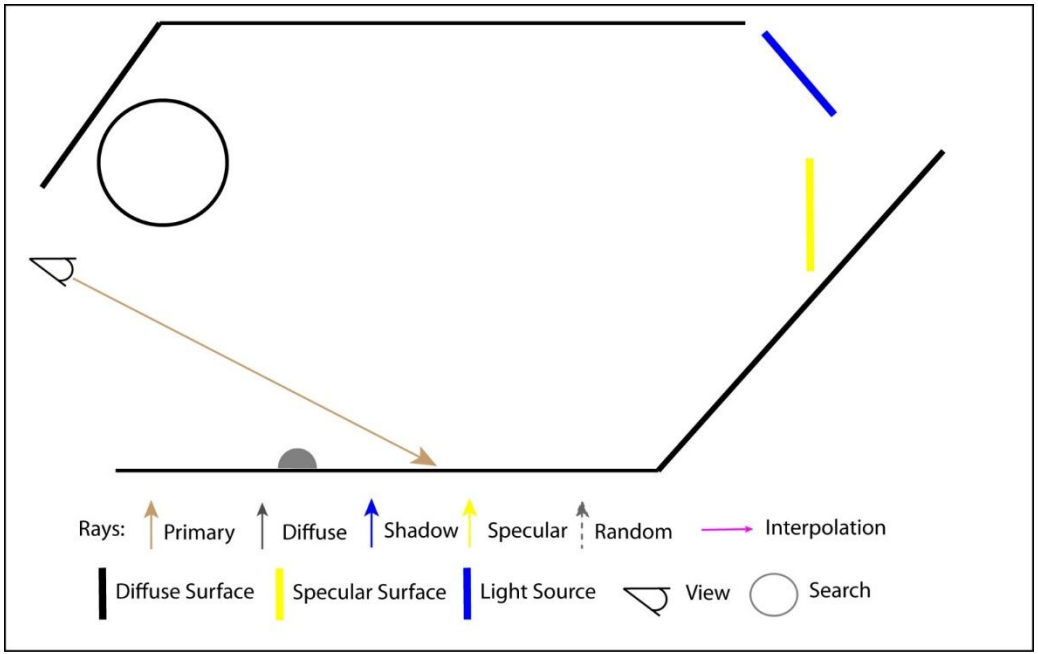
Irradiance Caching



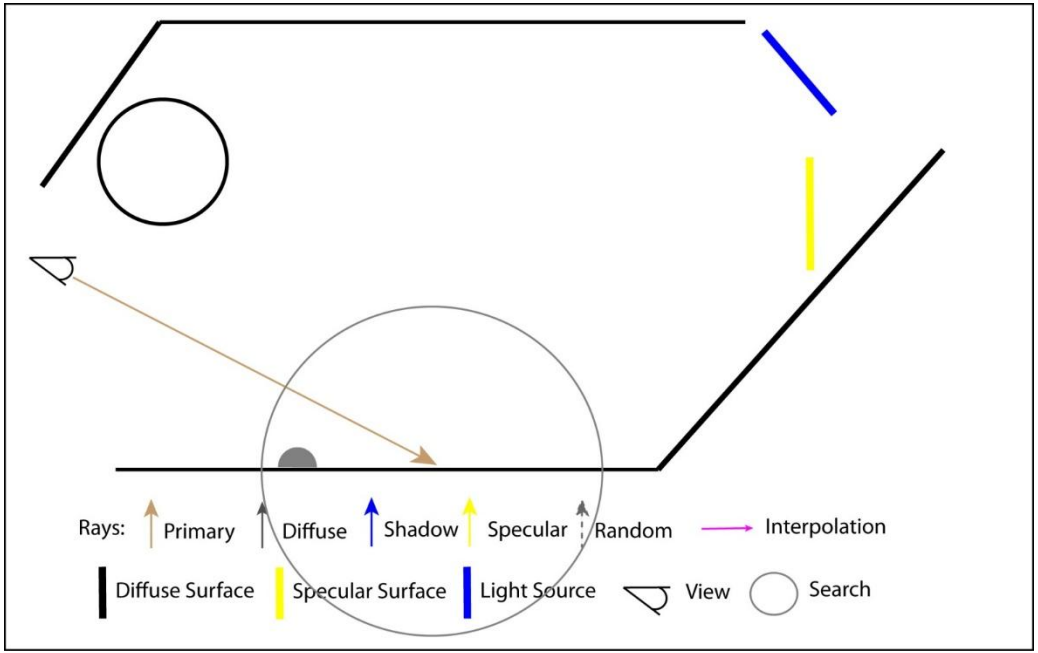
Irradiance Caching



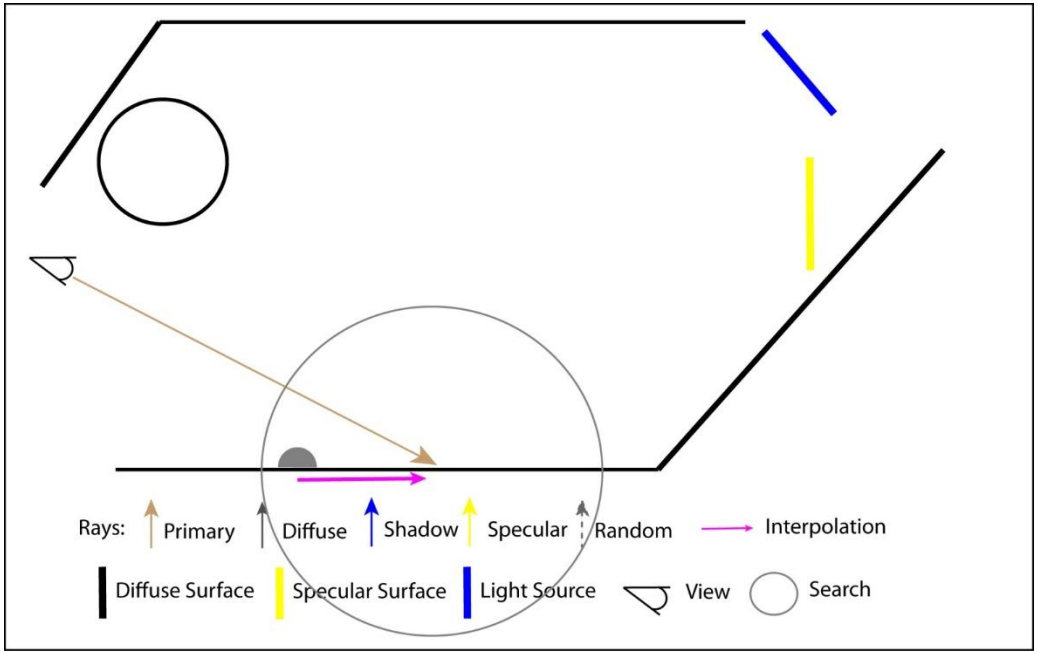
Irradiance Caching



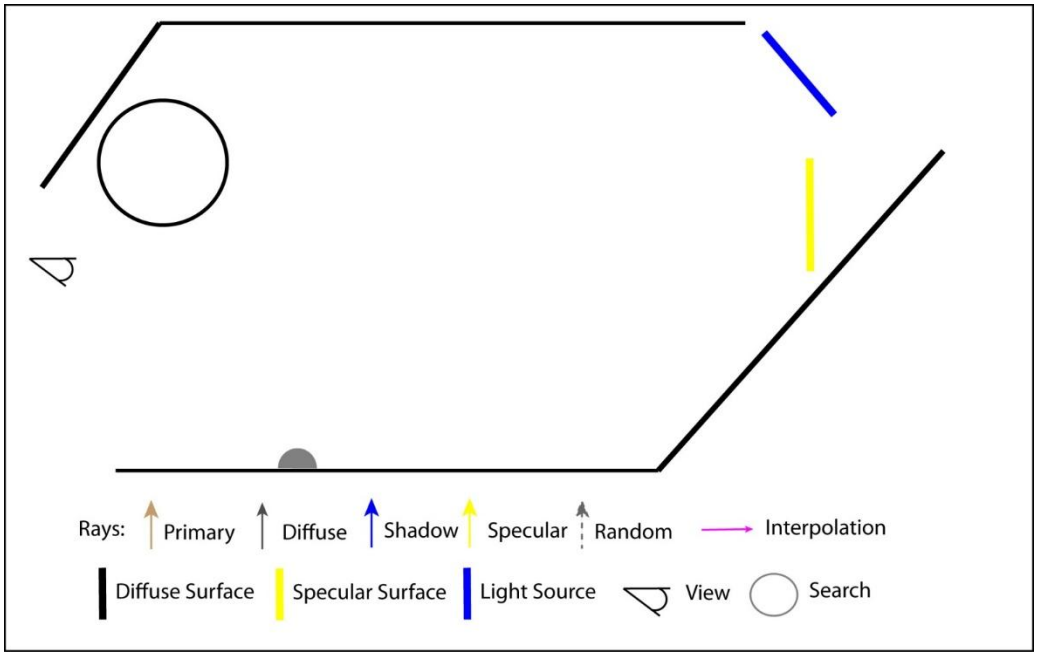
Irradiance Caching



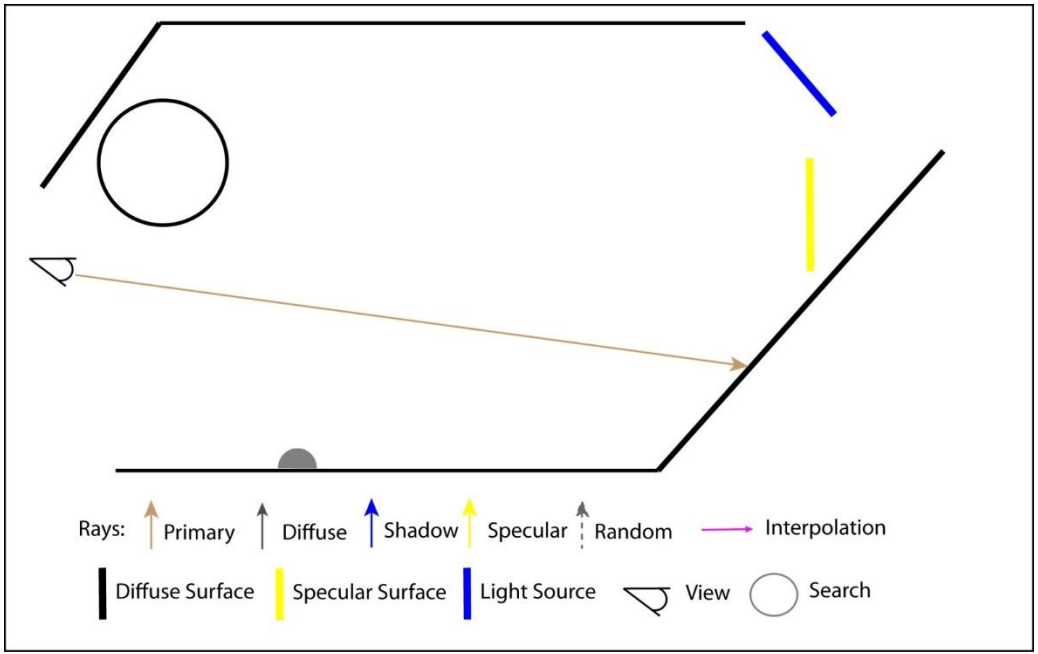
Irradiance Caching



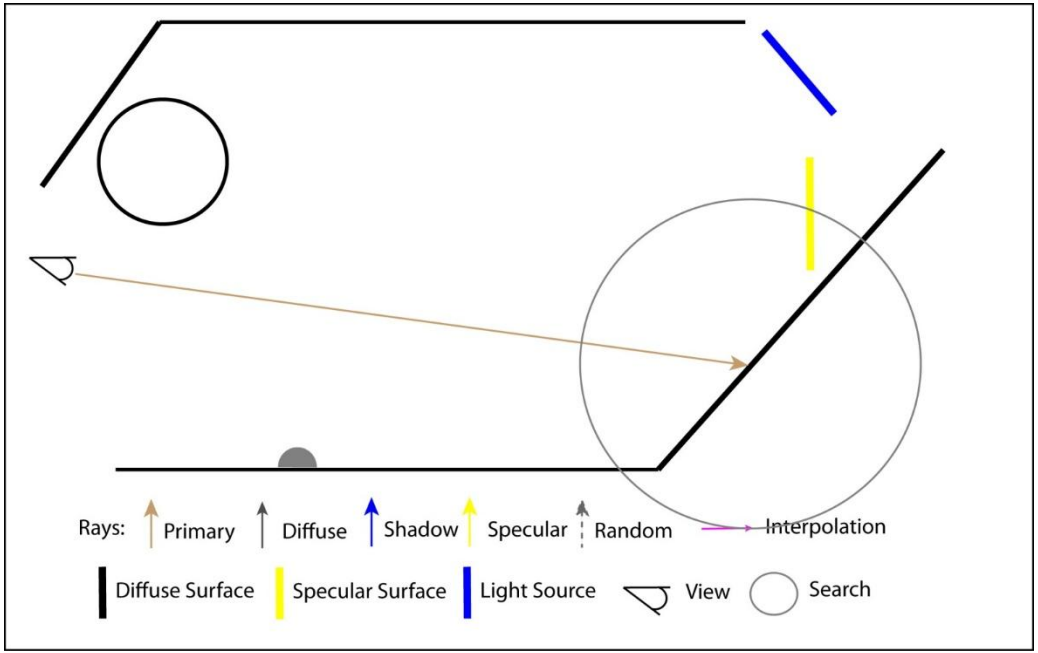
Irradiance Caching



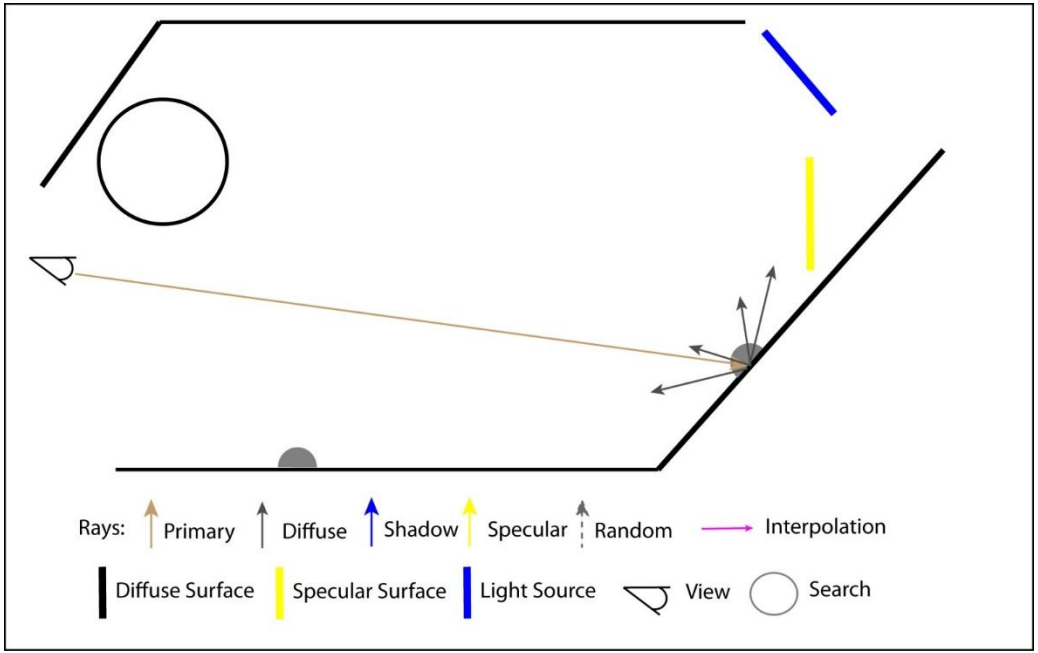
Irradiance Caching



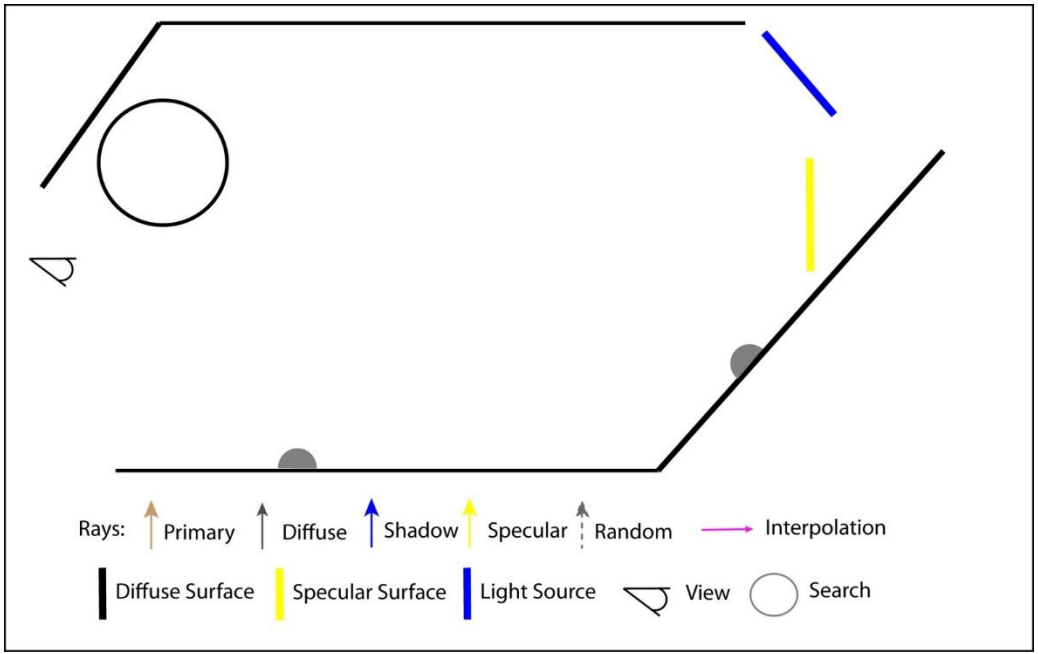
Irradiance Caching



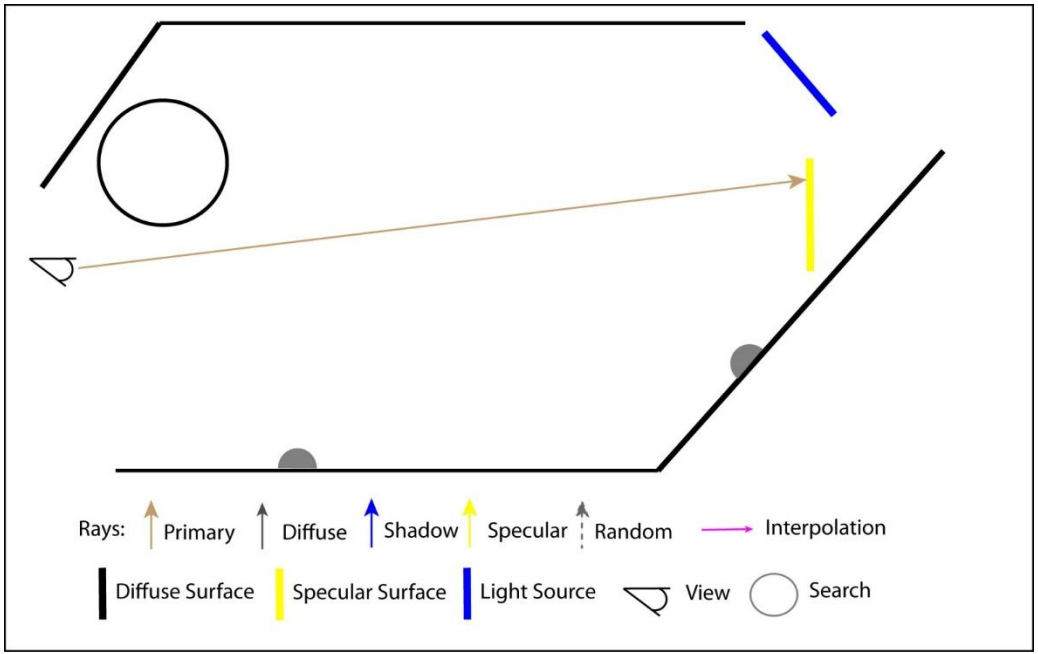
Irradiance Caching



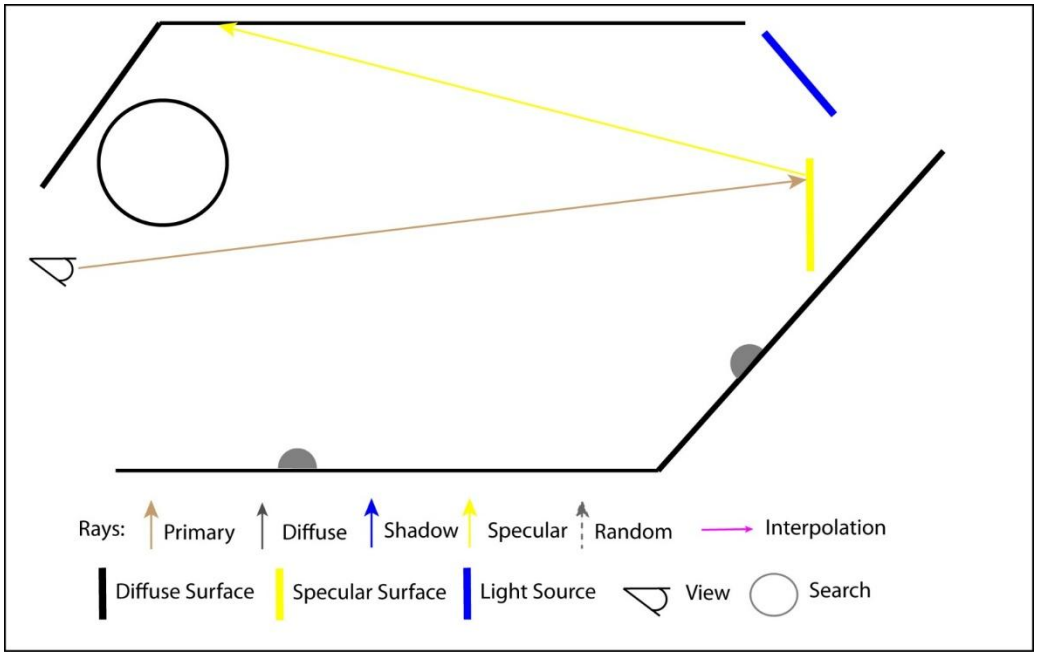
Irradiance Caching



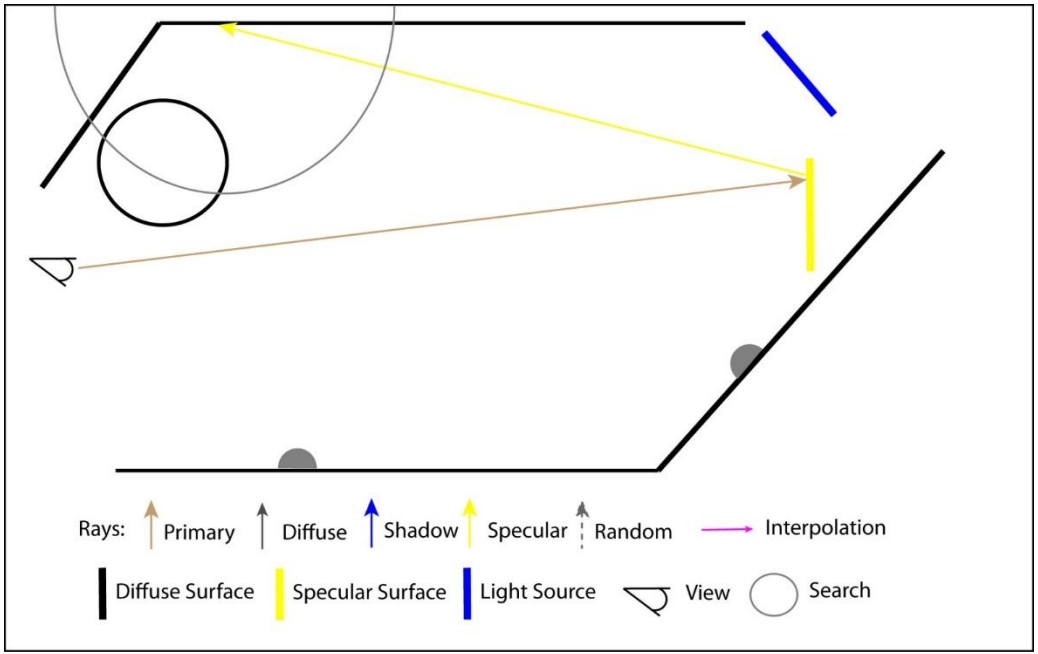
Irradiance Caching



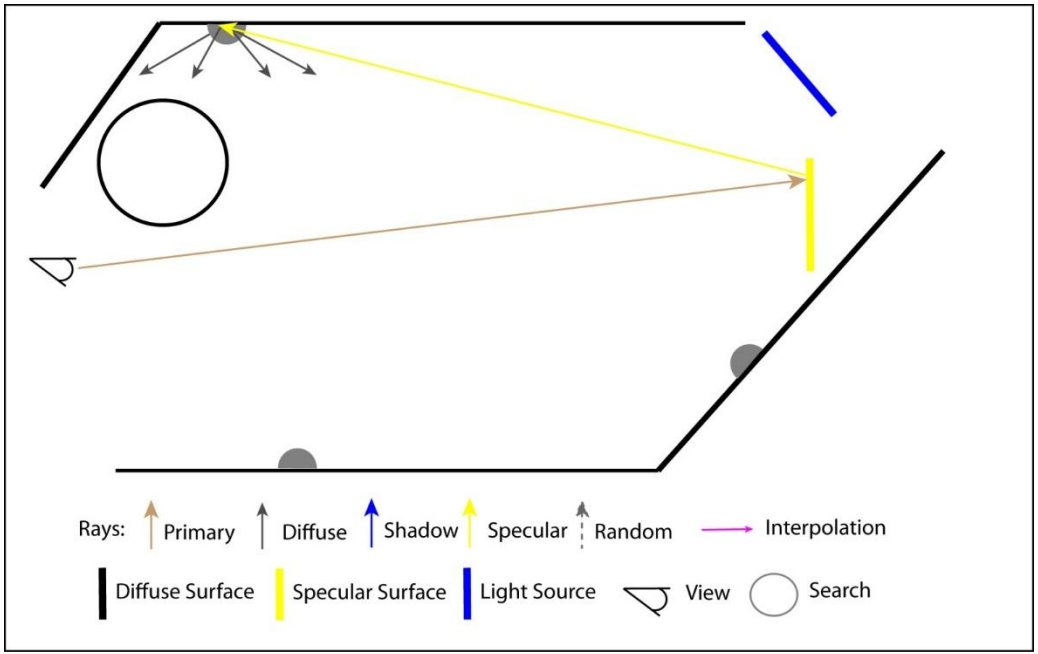
Irradiance Caching



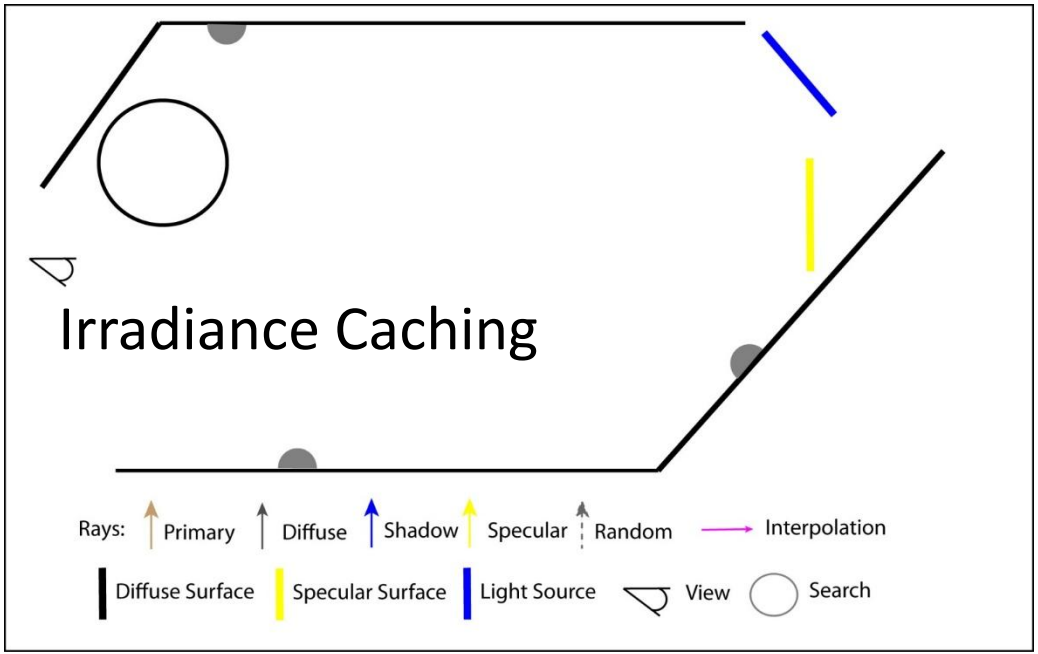
Irradiance Caching



Irradiance Caching



Irradiance Caching



Irradiance Caching

- How to select $\{x_1, x_2, ..., x_N\}$?
- Add a new point if no valid nearby points
  – i.e. if S is empty
- Add a new point if error from interpolating irradiance is too large

Irradiance Caching

- What should be used for $w(x, x_i)$?
  - Want to include distance and angular influence
  - Several expressions in literature
  - Common to use $$w(x, x_i) = \frac{1}{\frac{||x - x_i||}{R_i} + \sqrt{1 - (n_x \cdot n_{x_i})}}$$
    - Have to be careful of singularity
  - Or $$w(x, x_i) = 1 - \frac{1}{a} \max\left(\frac{||x - x_i||}{R_i}, \sqrt{1 - (n_x \cdot n_{x_i})}\right)$$
    - a user defined error parameter (often 1 or [0.5,2])

Irradiance Caching

- What should be used for $w(x, x_i)$?
  - Derived from split sphere model

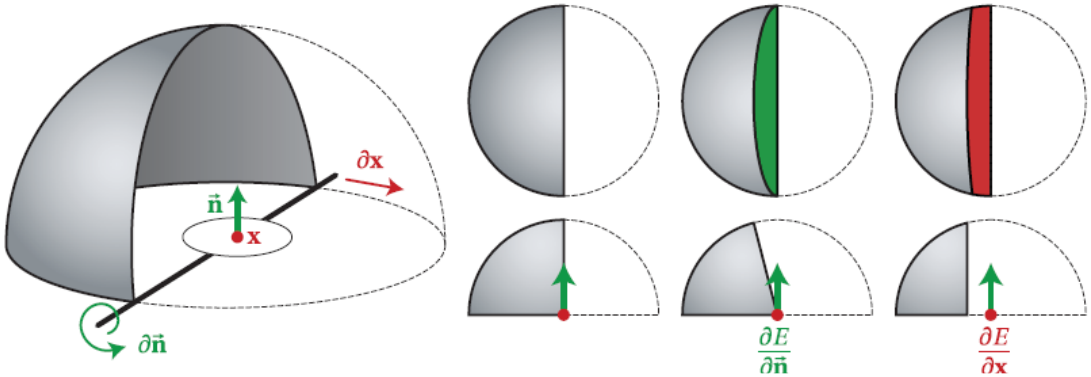


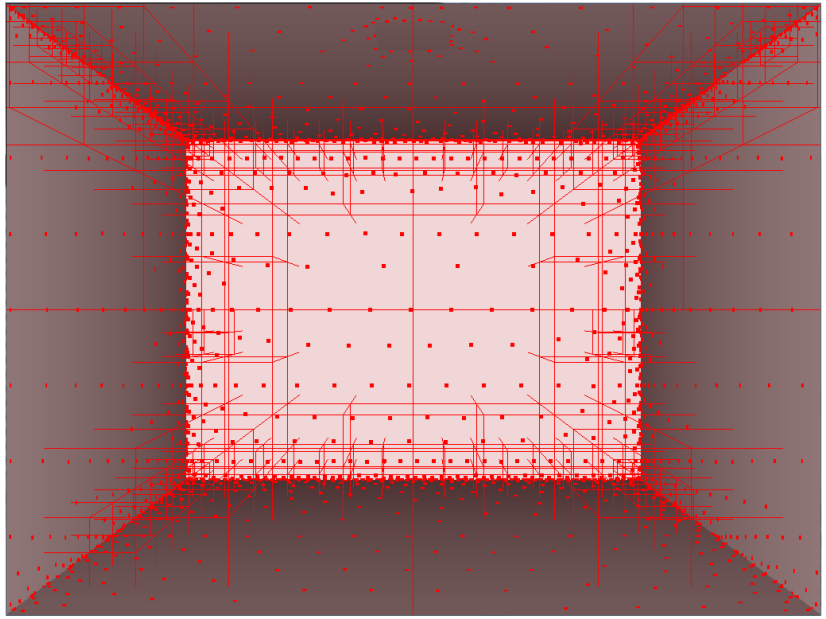
Image From https://cs.dartmouth.edu/~wjarosz/publications/dissertation/

Irradiance Caching

- How should S be selected?
  - Want low error for $$L_o(x,\omega_o) \approx \frac{\rho(x)}{\pi} \sum_{i=1}^{S} \frac{w(x,x_i)}{\sum_{j=1}^{N} w(x,x_j)} E(x_i)$$
  - Choose $$S = \{i : w(x,x_i) < \frac{1}{a}\}$$
  - Need to search all cache points

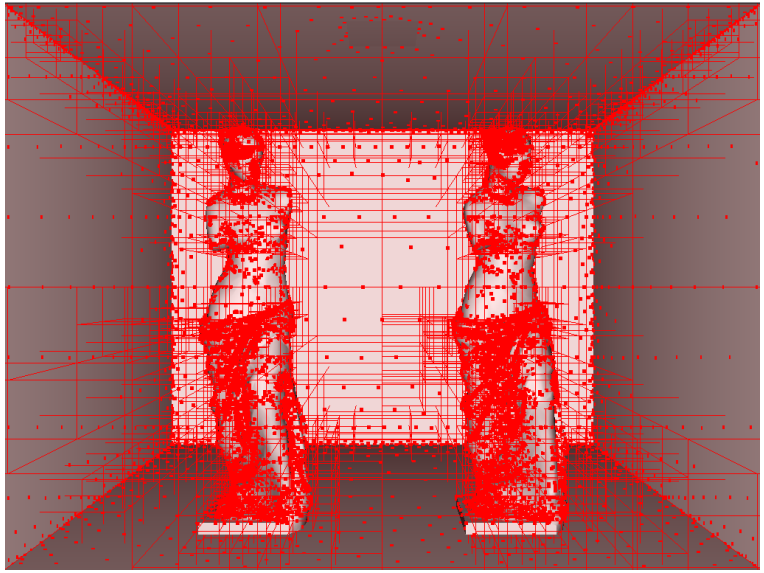
Irradiance Caching

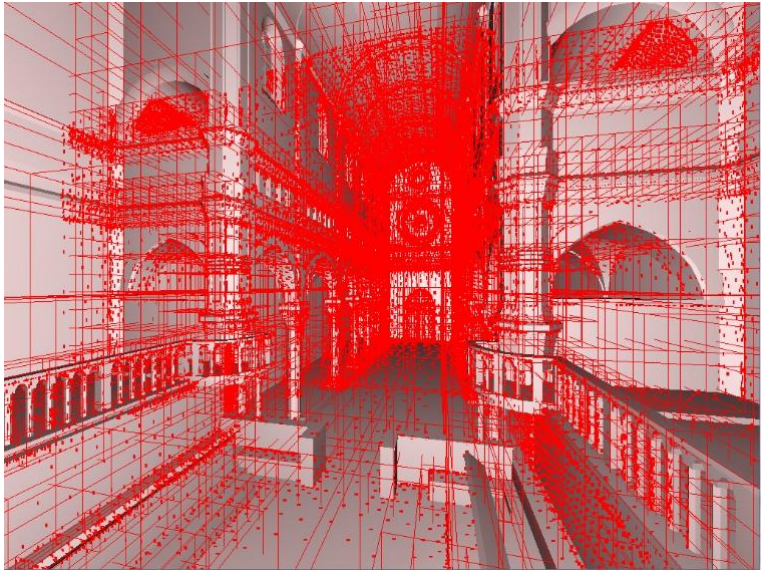
- How should $E(x)$ be stored for efficient access?
  - Octree



Irradiance Caching

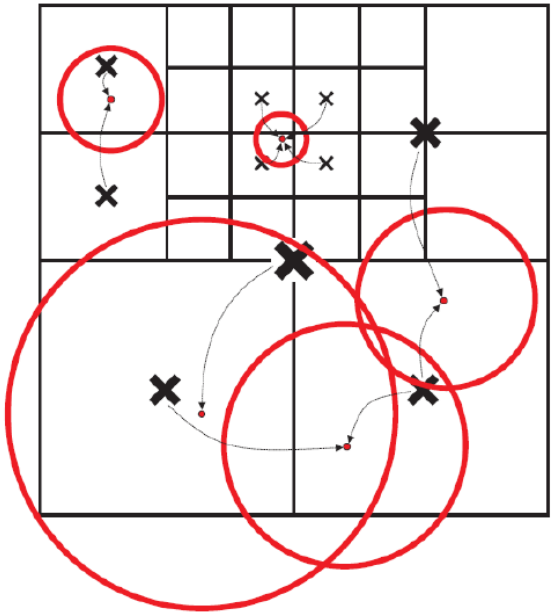
- Octree





Irradiance Caching

- Octree
  - Add point to octree when creating new cache points
  - Search tree when interpolating



Irradiance Caching

- Used to be used in films

Irradiance Caching

- Used to be used in films

Irradiance Caching

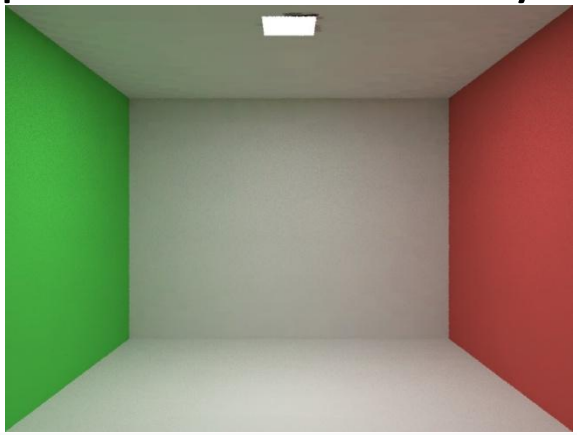
- Can also be used in participating media

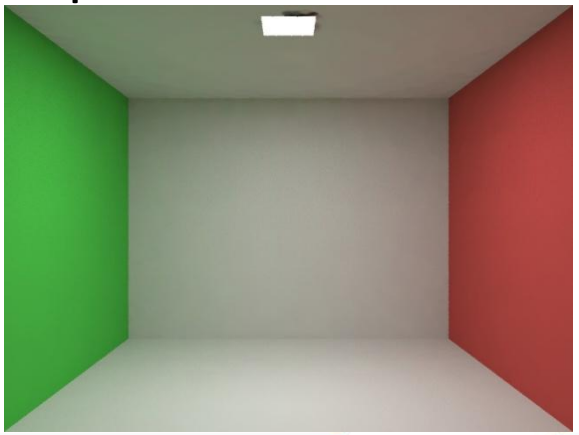
Irradiance Caching

- Limitations
  - Diffuse surfaces only
  - Consistent but not unbiased
  - Visible errors

Irradiance Caching

- Improvements
  - Overture Pass
    - Second pass which uses fully computed cache points





Irradiance Caching

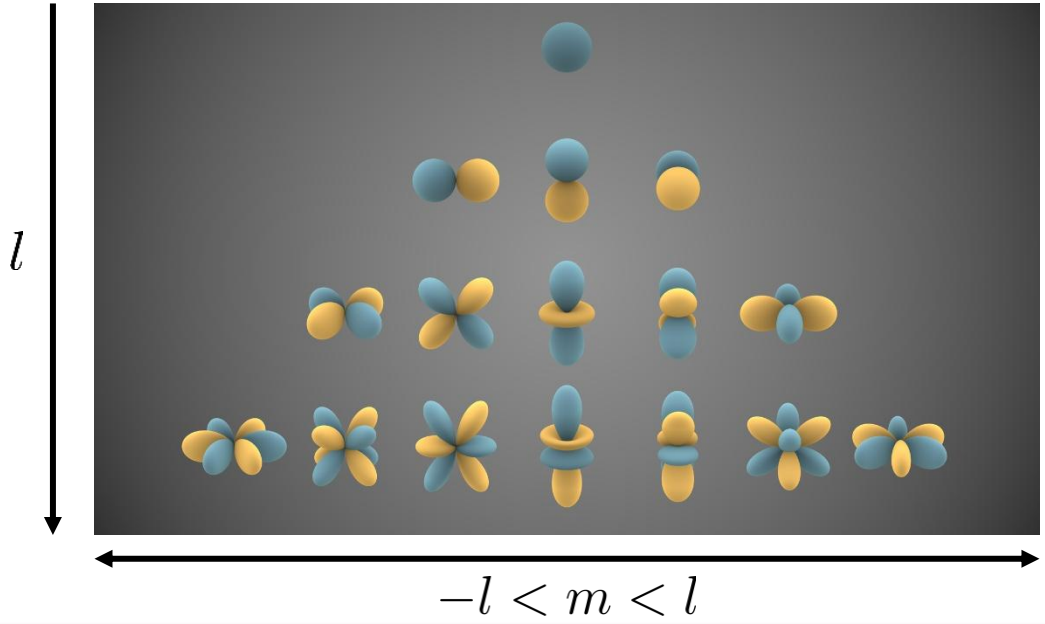
- Improvements
  - Gradients
  - Subtle tweaks (don’t use cache points in front of shading point)
  - Interpolate radiance for glossy surfaces

Radiance Caching

- Interpolate radiance
  - Combine with glossy BSDF
- Need to store radiance and BSDF in a common representation
- Usually use spherical basis

Radiance Caching

- Spherical Harmonics



Radiance Caching

- Spherical Harmonics
  - Function of spherical coordinates $Y(\omega) = Y(\theta, \phi)$
  - For m=0 $Y_{l0}(\theta, \phi) = \sqrt{\frac{2l+1}{4\pi}} P_l(\cos\theta)$
  - For m>0 $Y_{lm}(\theta, \phi) = \sqrt{2} \sqrt{\frac{2l+1}{4\pi} \frac{(l-m)!}{(l+m)!}} P_l^m(\cos\theta) \cos(m\phi)$
  - For m< 0 $Y_{l|m|}(\theta, \phi) = \sqrt{2} \sqrt{\frac{2l+1}{4\pi} \frac{(l-|m|)!}{(l+|m|)!}} P_l^{|m|}(\cos\theta) \sin(|m|\phi)$

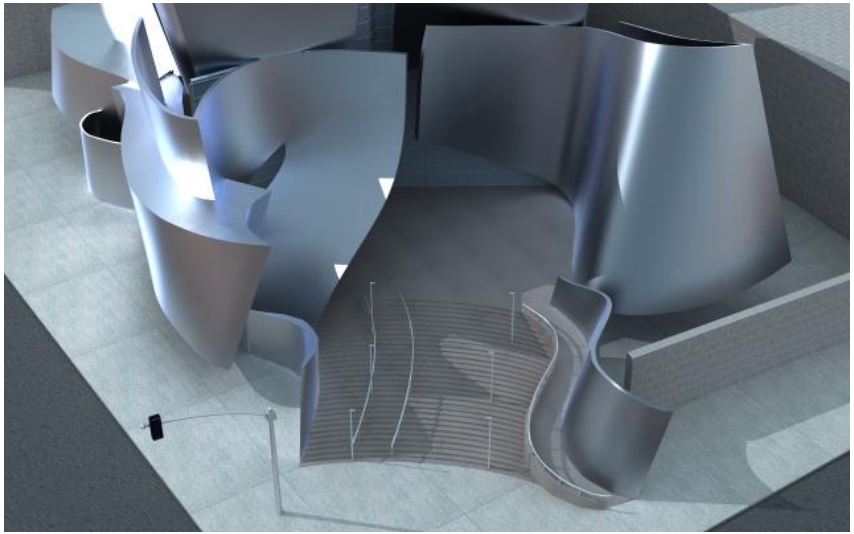
Radiance Caching

- Project incoming radiance over spherical harmonics $$c_{lm}(x) = \int_{\Omega} L(x, \omega_i)\, Y_l^m(\omega_i) d\omega_i$$
- Project BSDF $$b_{lm} = \int_{\Omega} f_r(x, \omega_i, \omega_o) \cos(\theta)\, Y_l^m(\omega_i) d\omega_i$$
- Result product of coefficients

$$L_o(x, \omega_o) \approx \sum_{l=0}^{L} \sum_{-l<m<l} c_{lm}(x) b_{lm}$$

Radiance Caching

- Can interpolate $c_{lm}(x)$

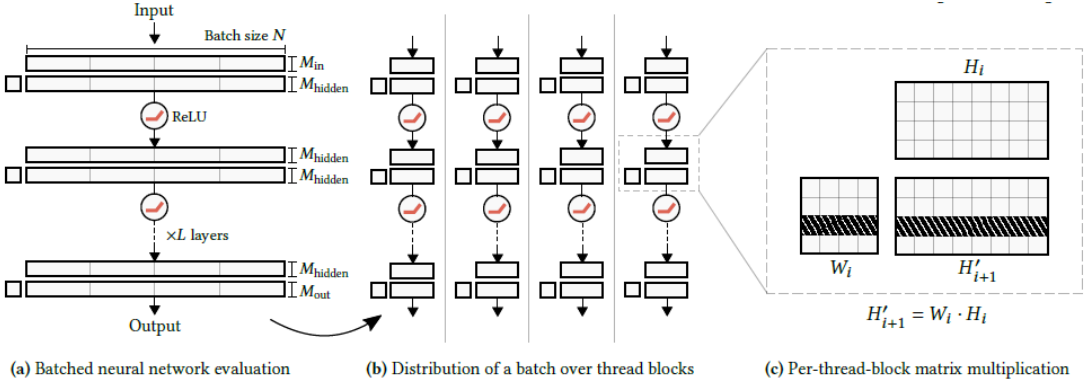


Neural Radiance Caching

- Can represent incident radiance with neural networks
  - Neural Radiance Caching
  - Learns incident radiance
  - And product with BSDF
  - Learns from small number of paths

Neural Radiance Caching

- Use small neural network with positional encoding (frequency) and other features
- Uses efficient GPU Matrix multiplication for FC Network



(a) Batched neural network evaluation (b) Distribution of a batch over thread blocks (c) Per-thread-block matrix multiplication

Neural Radiance Caching

Questions?

Computer Graphics

Rendering Algorithms

WARWICK

Dr. Tom Bashford-Rogers